c-for-emb

Автор Имя Фамилия

Содержание

Предисловие . . . 1
Кто такие эмбеддеры? . . . 1
О чем эта книга? . . . 2
Для кого эта книга? . . . 3
1. Краткая история языка C . . . 4
1.1. Предыстория: от ALGOLa к B . . . 5
1.2. Рождение языка C . . . 7
1.3. Переносимость кода . . . 8
1.4. Эволюция языка и стандарты . . . 10
1.5. Влияние языка C на экосистему встраиваемых систем . . . 16
2. Концепции . . . 17
2.1. Интерпретация и компиляция . . . 17
2.2. Языки общего назначения и предметно-ориентированные языки . . . 19
2.3. Парадигмы . . . 22
2.4. Типизация в языках программирования . . . 26
2.5. Резюме . . . 32
3. Первая программа . . . 33
3.1. Программа «Hello World!» . . . 33
3.2. Комментарии . . . 34
3.3. Включение заголовочного файла. . . . 35
3.4. Определение текстовой строки через макрос . . . 35
3.5. Главная функция программы . . . 36
3.6. Открывающая и закрывающая функцию скобки . . . 37
3.7. Вывод приветствия на консоль . . . 37
3.8. Разделитель . . . 37
3.9. Возврат в операционную систему . . . 38
4. Компиляция и выполнение кода C . . . 39
4.1. Выбор компилятора . . . 39
4.2. Фазы компиляции . . . 41
4.3. Независимая компиляция модулей . . . 44
4.4. Использование библиотек . . . 44
4.5. Полезные артефакты компиляции . . . 46
4.6. Выполнение программы на C . . . 53
5. Препроцессор и макрокоманды . . . 75
5.1. Директивы препроцессора . . . 75
5.2. Объектные макросы (макросы без аргументов) . . . 75
5.3. Функциональные макросы (макросы с аргументами) . . . 76
5.4. Операторы # и ## . . . 77

5.5. Предопределённые макросы . . . 77
5.6. Условная компиляция . . . 78
5.7. Многострочные макросы . . . 81
5.8. Достоинства и недостатки макросов . . . 81
5.9. Когда следует применять макросы . . . 82
5.10. Резюме . . . 84
6. Типы в языке C . . . 85
6.1. Понятие типа . . . 85
6.2. Тип, объект, значение . . . 86
6.3. Базовые и производные типы . . . 87
6.4. Квалифицированные типы . . . 88
6.5. Общая картина типов в C . . . 91
7. Скалярные типы . . . 93
7.1. Целочисленные типы (`int`) . . . 93
7.2. Целочисленные типы произвольной разрядности (`_BitInt(N)`) . . . 97
7.3. Перечисления . . . 98
7.4. Вещественные типы . . . 110
7.5. Булев тип . . . 112
7.6. Указатели . . . 113
7.7. Обычные арифметические преобразования . . . 118
7.8. Преобразования между скалярными типами . . . 119
7.9. Сводная таблица скалярных типов . . . 120
7.10. Резюме . . . 120

Предисловие

Эта книга - первая из задуманной серии учебных пособий для начинающих эмбеддеров.

Кто такие эмбеддеры?

Термин «эмбеддер» происходит от английского **embedded systems** (встраиваемые системы).

Что такое "Встраиваемая система"

Встраиваемая система (от англ. **embedded system**) — это **специализированная вычислительная система**, которая предназначена для выполнения одной или нескольких строго определенных задач в составе более крупного устройства или комплекса.

В отличие от универсальных компьютеров (ноутбуков, десктопов, серверов), на которых можно запускать разные программы, встраиваемая система «заточена» под одну конкретную функцию и часто вовсе скрыта от глаз пользователя.

Ключевые особенности встраиваемых систем:

1. **Специализация**: система решает одну закрепленную за ней задачу (например, поддержание температуры в холодильнике, управление впрыском топлива в двигателе, отслеживание уровня воды в резервуаре).
2. **Ограниченные ресурсы**: такие системы часто работают на микроконтроллерах с небольшим объемом оперативной памяти, низкой тактовой частотой процессора и строгими лимитами энергопотребления (например, работа от батарейки в течение нескольких месяцев).
3. **Надежность и отказоустойчивость**: они должны работать стабильно длительное время, часто в экстремальных условиях (перепады температур, электромагнитные помехи, вибрация), без необходимости явной перезагрузки.
4. **Работа в реальном времени (Real-Time)**: многие из них обязаны гарантированно реагировать на внешние события за строго определенное время (например, система ABS в автомобиле должна сработать за миллисекунды при блокировке колес).

Состав типичной встраиваемой системы:

- **Аппаратная часть**: микроконтроллер (микропроцессор), память (Flash для кода, RAM для данных), периферия: датчики (температуры, давления, движения), исполнительные механизмы (сервомоторы, реле, клапаны), человеко-машинный интерфейс (дисплеи, индикаторы, кнопки, энкодеры).
- **Программная часть**: встроенное программное обеспечение, которое часто называют **прошивкой** (firmware). Оно может быть написано:
  - как «голый» код (bare-metal), работающий напрямую с оборудованием;

    - на базе операционной системы реального времени (RTOS);
    - на базе операционной системы Linux.

Соответственно, эмбеддерами на профессиональном жаргоне называют разработчиков встраиваемых систем, либо, в более узком смысле, разработчиков **код** для этих систем.

Работа эмбеддера находится на стыке «железа» и софта, поэтому спектр необходимых для этого знаний чрезвычайно широк:

- уверенное владение языками программирования встраиваемых систем: C (основной на данный момент), C++ (с существенными ограничениями, специфическими для эмбеддинга), Rust (быстро набирает популярность);
- знание архитектуры микроконтроллеров;
- умение читать и понимать электрические схемы,
- знание основ электроники;
- знание протоколов обмена данными (на низком уровне - UART, SPI, I2C, USB, Ethernet; на высоком - TCP/IP, Modbus, MQTT);
- понимание принципов работы операционных систем реального времени (RTOS);
- умение оптимизировать код под ограниченные ресурсы (низкая вычислительная поизводительность, малый объем RAM/ROM).

Этот список далеко не полон и приведен лишь для иллюстрации.

О чем эта книга?

Эта книга - о языке программирования C. Но не просто о языке, ведь подобных за более чем полувековую историю языка уже написано немало. Как уже говорилось ранее, серия предназначена для эмбеддеров. Поэтому язык рассматривается в первую очередь с их точки зрения. Что это значит? Какие такие особенные потребности есть у них?

Приведу лишь один пример. Выше упоминался код для bare-metal. Этот код работает без поддержки операционной системы (а она весьма значительна, даже если не видна с первого раза), поэтому должен заботиться о всех низкоуровневых деталях сам: инициализировать оборудование, распределить системные ресурсы и т. д. Если вы не имеете дело с микроконтроллерами, вы вряд ли когда-нибудь столкнетесь с таким кодом. Во-первых, вам это просто не нужно, операционная система сделает это за вас в лучшем виде. Во-вторых, даже если вам вдруг захочется поэкспериментировать в этом направлении, операционная система просто не позволит вам сделать это, поскольку на корню рубит любые притязания на системныересурсы в обход системного API. Поэтому можно десятилетиями писать на C и в лучшем случае лишь понаслышке знать о работе с автономным кодом.

Еще одной особенностью книги является привязка материала к действующим стандартам языка (на момент написания это был ISO/IEC 9899:2024, также известный как C23). Многие учебные пособия, особенно для начинающих, обходят стандарты молчанием, и совершенно

напрасно. Конечно, легким чтением стандарт языка не назовешь. Однако он является последней инстанцией в случае, когда нужно разобраться с каким-то тонким моментом. Я старался достичь баланса между легкостью материала для понимания и строгостью соответствия стандарту.

Конечно, при желании эту книгу может читать любой желающий познакомиться с легендарным языком программирования. Я надеюсь, он найдет здесь много полезного для себя. И все же наибольшую пользу по замыслу она должна принести именно эмбеддерам.

Для кого эта книга?

У этой книги есть заранее определенная аудитория. Это студенты, обучающиеся в созданной специально с этой целью учебной лабораторией под названием "Лектроника".

Лаборатория создана организацией с длинным названием "ФГАНУ НИИ Специализированные вычислительные устройства защиты и автоматика", в народе "Спецвуз", совместно с физическим факультетом Южного федерального университета (г. Ростов-на-Дону). Задача лаборатории - готовить квалифицированные кадры для быстро развивающегося Спецвуза, конкретно - тех самых эмбеддеров.

Учащиеся лаборатории слушают лекции, выполняют лабораторные работы, работают над проектами. Лучших ждет приз: работа в "Спецвузе" (в который, объективно говоря, попасть не так уж просто, нужно выдержать серьезное собеседование).

Я читаю ученикам лекции по программированию микроконтроллеров, основам электроники, сетям передачи данных и ряд других (мы уже знаем, что круг знаний хорошего эмбеддера должен быть весьма широким). График учебы напряженный, поэтому весьма остро ощущается нехватка учебных пособий для дополнительной самостоятельной работы. Первым их числа таких пособий должна стать эта книга. Это и побудило меня приняться за работу.

Конечно, нет причин ограничивать круг читателей лишь учащимися "Лектроники". Книга может оказаться полезной множеству читателей, независимо от опыта. Однако по ходу изложения я буду придерживаться учебного курса, который устоялся за несколько лет работы студенческой лаборатории. Поэтому, если кому-то покажется непонятным, почему речь зашла о каком-либо предмете, ответ вы уже знаете: это нужно для нашей учебной программы.

Итак, если вы решили, что книга вам подойдет, либо вы - учащийся "Лектроники" (в этом случае у вас просто нет иного выбора), - приступим.

Chapter 1. Краткая история языка C

Для чего вообще нужно знать историю языка программирования?

В действительности многие обходят историю языка, который они изучают, стороной. И, на мой взгляд, делают это совершенно напрасно.

Я даже не говорю о том, что это само по себе интересно. Помимо обычного любопытства, знание истории языка имеет и весьма практическое значение.

Во-первых, большинство языков программирования возникают в конкретный момент времени и для конкретных задач. Не все; иногда язык создается для экспериментальной проверки некоторой концепции и представляет чисто академический интерес (однако к языку C это никоим образом не относится). Зная, кто, когда и с какой целью создал язык, можно понять, почему он получился именно таким. Окажется, что некоторые конструкции, которые могутт показаться на первый взгляд странными и нелогичными, на самом деле наилучшим образом выполняют свое назначение (по крайней мере, на момент создания).

Во-вторых, язык редко возникает ниоткуда во всей полноте и целостности. Он проходит несколько стадий взросления до того, как достигнет окончательной зрелости. По мере использования на практике некоторые черты языка будут изменяться, некоторые окажутся устаревшими и будут исключены из его состава, а некоторые, наоборот, добавлены. Глядя на статичный снимок языка в конкретный момент времени, иногда трудно полностью понять, почему фрагмент кода выглядит именно так, а не иначе.

В-третьих, сама окружающая среда оказывает на язык большое влияние. Например, разработка C начиналась на мини-ЭВМ PDP-11 (не дайте слову "мини" одурачить себя; мини-ЭВМ той эпохи с трудом размещались в небольшой комнате), и некоторое время в начале жизненного пути из конструкций языка повсюду "торчали уши" этого оборудования. Понимая это, задаешь себе меньше вопросов, почемучто-то работает именно так, а не иначе.

Немалое влияние на развитие языка оказало также то, что он создавался в первую очередь для системного программирования, а конкретно - для разработки ОС Unix.

Знание подобных деталей объясняет, **почему** язык устроен именно так, а не иначе: почему в нём есть не просто указатели, а оснащенные средствами специальной арифметики, почему отсутствуют встроенные средства ввода-вывода, почему типы имеют платформенно-зависимый размер (парадокс, потому что целью было как раз создание языка, позволяющего разрабатывать портабельный, кроссплатформенный код), и многие другие особенности. Это знание позволяет **понимать** язык, а не зазубривать.

Получившийся в итоге язык обладает уникальными свойствами: с одной стороны, он достаточно мощный и выразительный, чтобы на нем можно было писать высокоуровневый

код общего назначения (для отдельных экологических ниш выбор C будет не оптимальным, но в принципе большинство задач решить можно); с другой стороны, он позволяет писать код самого нижнего уровня, непосредственно взаимодействующий с оборудованием, не прибегая к использованию ассемблера (или в крайнем случае сводящий это использование к необходимому минимуму).

В разных источниках вы найдете самые разные мнения касательно уровня языка C:

- C - язык высокого уровня (поскольку мы не работаем напрямую с регистрами и инструкциями процессора);
- C - язык низкого уровня (поскольку при необходимости мы можем обращаться к функциям оборудования напрямую без использования ассемблера);
- C - язык высокого уровня с возможностями низкого;
- C - язык низкого уровня с возможностями высокого.

В общем, перебраны все возможные варианты. К счастью, этот разнобой в мнениях ни на что существенно важное не влияет. Гораздо важнее то, что в результате появился язык программирования, на котором были написаны:

- ядра ОС общего назначения (Unix, Linux);
- ядра ОС реального времени (FreeRTOS, Zephyr, RT-Thread);
- стеки сетевых протоколов;
- загрузчики (U-Boot, Das U-Boot);
- многочисленные драйверы периферии;
- библиотеки от производителей микроконтроллеров (STM32 HAL, ESP-IDF, Nordic SDK);
- и еще огромное множество кода.

В данной главе мы проследим путь языка C от его предшественников 1960-х годов до современного стандарта C23, уделив особое внимание тем аспектам, которые непосредственно влияют на повседневную работу embedded-программиста.

1.1. Предыстория: от ALGOLa к B

Как и большинство других языков программирования, C возник не на пустом месте. При его создании учитывался опыт нескольких предшествующих ему языков.

Одним из оставивших наиболее значительный след в истории был ALGOL 60.

1.1.1. ALGOL 60

В 1960 году международный комитет учёных представил язык **ALGOL 60** (Algorithmic Language). Он стал революционным для своего времени: ввёл блочную структуру, лексическую область видимости переменных, рекурсию и статическую сильную типизацию. ALGOL 60 заложил основу синтаксиса, который мы сегодня называем «ALGOL-подобным» (фигурные скобки для блоков, ключевые слова).

Однако ALGOL 60 был ориентирован на научные вычисления и оказался не самым подходящим для системного программирования — написания компиляторов, операционных систем, утилит. Ему была уготована другая судьба: по сути долгое время он играл в информатике ту же роль, что латынь - в медицине (использовалсядля публикации алгоритмов).

1.1.2. CPL

Учёные из Кембриджского и Лондонского университетов попытались развить идеи ALGOL, создав язык **CPL** (Combined Programming Language или Cambridge Programming Language, 1963). Разработчики попытались объединить в нем выразительность, блочную структуру и строгость ALGOL 60 (для написания сложных алгоритмов) с возможностью низкоуровневого управления аппаратурой, свойственную ассемблеру (для системного программирования). Фактически эти цели во многом схожи с теми, которые позднее преследовали создатели C.

Язык CPL включал:

- Строгую статическую типизацию.
- Блочную структуру и лексическую область видимости переменных.
- Возможность работы с указателями и прямой адресации памяти.
- Поддержку как высокоуровневых абстракций, так и машинно-ориентированных конструкций.

В результате язык получился мощным, но чрезмерно сложным: его компилятор было практически невозможно реализовать на машинах того времени.

1.1.3. BCPL

В 1967 году Мартин Ричардс из Кембриджа взял CPL и основательно его упростил, создав **BCPL** (Basic CPL).

Ключевая особенность BCPL — **отсутствие типов данных**. В языке существовал единственный тип — машинное слово (обычно 36 бит, довольно распространенная разрядность для компьютеров того времени). Переменная могла хранить и число, и адрес, и несколько литер — интерпретация зависела от контекста.

В отлисие от предшественника, BCPL был успешно реализован на множестве машин и активно использовался для разработки различного кода, в том числе написания компиляторов и системных утилит.

1.1.4. B

В 1969 году **Кен Томпсон** при участии **Денниса Ритчи** из легендарной лаборатории **Bell Labs** (подразделение AT&T) адаптировал BCPL для работы на списанном миникомпьютере **DEC PDP-7**, создав язык **B**. Свое название новый язык получил по первой букве BCPL. Помимо буквы из названния, язык B унаследовал также одну из главных особенностей своего предшественника: **бестиповость**. Все данные трактовались как машинные слова и интерпретировались в зависимости от контекста.

Именно на PDP-7 Томпсон написал первую версию операционной системы, которую его коллега Брайан Керниган (Brian Kernighan) в шутку назвал **UNICS** (Uniplexed Information and Computing Service) — как ироничную противоположность амбициозному проекту MULTICS. Позже название трансформировалось в ныне известное всем **UNIX**.

1.2. Рождение языка C

В 1970 году Bell Labs закупила новый, значительно более мощный миникомпьютер — **DEC PDP-11**. Этот компьютер обладал двумя важными аппаратными особенностями:

- **Байтовой адресацией памяти** (в отличие от PDP-7 с его 18-битной словесной адресацией).
- **Аппаратной поддержкой чисел с плавающей точкой** и различных форматов целых чисел.

Язык B, спроектированный для старой архитектуры, оказался неэффективен на PDP-11.

1.2.1. Новый язык Денниса Ритчи

Кен Томпсон и его коллега **Деннис Ритчи** (Dennis Ritchie) столкнулись с дилеммой: либо смириться с ограничениями языка B, либо создать новый язык, способный эффективно использовать новые возможности PDP-11. Они выбрали второе.

В 1971–1972 годах Деннис Ритчи создал язык, взяв за основу синтаксис B, но добавив критически важные нововведения:

1. **Типы данных** (`char`, `int`, `float`). Компилятор теперь знал размер каждой переменной и мог генерировать оптимальный машинный код.
2. **Структуры** (`struct`) для группировки связанных данных.
3. **Типизированные указатели**. Указатель на `int` и указатель на `char` вели себя по-разному при арифметике, что элегантно связывало указатели и массивы.
4. **Препроцессор** и макросы для создания абстракций без перегрузки синтаксиса.

Сначала язык получил рабочее название **NB** (New B). Однако по мере расширения он настолько отдалился от оригинала, что Ритчи взял следующую букву алфавита. Так появился **язык C**.

1.2.2. C и UNIX

До 1973 года в индустрии царила догма: **ядра операционных систем пишутся только на ассемблере**. Языки высокого уровня считались слишком медленными, неэффективными и не дающими нужного контроля над аппаратурой.

Ритчи и Томпсон отошли от этой догмы. В 1973 году они **переписали ядро UNIX на языке C**. В ассемблере остались лишь самые низкоуровневые фрагменты: код начальной загрузки и обработчики прерываний, напрямую работающие с регистрами процессора.

Строго говоря, Ритчи и Томпсон не были первыми, кто смог реализовать основную часть операционной системы на языке высокого уровня. Такие попытки предпринимались и до них.

Первопроходцем была операционная система MCP (Master Control Program), созданная для мейнфреймов Burroughs B5000, анонсированных в 1961 году. Изначально она разрабатывалась на языке ESPOL (Executive Systems Programming Oriented Language), специально расширенном для этой цели диалекте ALGOL 60. Позже он был заменен на язык NEWP.

Впрочем, архитектура Burroughs была слишком проприетарной и закрытой, чтобы эта попытка оказала существенное влияние на общие тенденции.

Второй прецедент - MULTICS (Multiplexed Information and Computing Service), над которой работали MIT, General Electric и Bell Labs в середине 1960-х. Значительная часть этой системы (в том числе некоторые подсистемы и утилиты) была написана на языке PL/I (Programming Language One), который также является высокоуровневым.

В результате система получилась слишком сложной, медленной и дорогой, что в итоге заставило Bell Labs выйти из проекта. Однако полученный в ходе резработки MULTICS опыт в итоге подтолкнул Кена Томпсона и Денниса Ритчи к созданию UNIX, и к тому же оказал влияние на название новой ОС.

UNIX стала первой в истории широко распространенной операционной системой, написанной на языке высокого уровня, определив дальнейшее развитие системного программирования.



Симбиоз C и UNIX — классический пример совместной эволюции инструмента и продукта. UNIX нужен был C, чтобы стать мобильной ОС, не привязанной к конкретной архитектуре вычислительной машины. C нужен был UNIX, чтобы получить первую масштабную, сложную и практически значимую область применения. Без UNIX язык C остался бы одним из многих академических экспериментов; без C UNIX никогда не стала бы кроссплатформенной.

1.3. Переносимость кода

Именно переносимость кода стала ключевым фактором широкой популярности языка C в системном программировании и, в частности, в эмбеддинге.

1.3.1. Что такое переносимость?

Переносимость (portability) — это свойство программы, позволяющее ей работать на различных аппаратных платформах без существенной модификации исходного кода.

До появления C переносимость операционных систем была практически нереальна. Чтобы перенести ОС с одной платформы на другую, потребовалосьбы полностью переписать

миллионы строк **ассемблерного кода** под другую архитектуру процессора.

1.3.2. Концепция переносимости

Благодаря языку C переносимость стала реально достижимой. Чтобы перенести UNIX на новый компьютер, достаточно было выполнить две операции:

1. Написать **компилятор языка C** для нового процессора.
2. Скомпилировать исходный код UNIX этим новым компилятором.

Почти вся остальная операционная система переносилась автоматически. Язык C стал «переносимым ассемблером»: он давал доступ к памяти и регистрам (через указатели и битовые операции), но при этом абстрагировал программиста от **конкретных** команд **конкретного** процессора.

1.3.3. Значение переносимости для встраиваемых систем

Для встраиваемых систем переносимость имеет особое значение. Мир микроконтроллеров чрезвычайно разнообразен:

- **Архитектуры:** ARM Cortex-M (M0, M3, M4, M7, M33), AVR, RISC-V, MSP430, PIC, 8051, Xtensa (ESP32), MIPS и др.
- **Производители:** STMicroelectronics, NXP, Microchip, Texas Instruments, Infineon, Espressif, Nordic Semiconductor, GigaDevice и десятки других.
- **Разрядность:** 8-битные, 16-битные, 32-битные, 64-битные.
- **Порядок байтов:** little-endian, big-endian.

Написание отдельной версии драйвера или библиотеки для каждой платформы было бы экономически нецелесообразно. Поэтому разработка встраиваемых систем построена вокруг языка C:

- **HAL-библиотеки** (Hardware Abstraction Layer) производителей пишутся на C и компилируются для микроконтроллеров определенного семейства. HAL образует своего рода оболочку, которая позволяет абстрагироваться от деталей конкретной реализации.
- **RTOS** (Real-Time Operating Systems, операционные системы реального времени) имеют переносимое ядро на C и платформенно-зависимый порт на ассемблере. Примеры популярных RTOS: FreeRTOS, Zephyr, embOS.
- **Сторонние библиотеки** (mbedTLS, lwIP, FatFS, LVGL, cJSON) пишутся на C и работают на любых платформах с подходящим компилятором.

1.3.4. Цена переносимости: платформенно-зависимые типы

Переносимость имеет свою цену. Стандарт языка C **намеренно не фиксирует** размеры базовых типов (`char`, `short`, `int`, `long`), оставляя это на усмотрение конкретной реализации. Это позволяет компилятору выбирать наиболее эффективные размеры для данной архитектуры.

Тип	8-бит AVR	16-бит MSP430	32-бит ARM	64-бит x86-64
`char`	1 байт	1 байт	1 байт	1 байт
`short`	2 байта	2 байта	2 байта	2 байта
`int`	2 байта	2 байта	4 байта	4 байта
`long`	4 байта	4 байта	4 байта	8 байт (Linux) / 4 байта (Windows)
`long long`	8 байт	8 байт	8 байт	8 байт
Указатель	2 байта	2 байта	4 байта	8 байт



Эта неопределённость — частый источник ошибок при портировании кода между платформами. Выражение `sizeof(int) == 4`, справедливое для ARM, может оказаться ложным для AVR. Именно поэтому эмбеддеры используют типы из заголовка `<stdint.h>` (`uint8_t`, `uint16_t`, `uint32_t`, `int64_t`), введённые стандартом C99.

1.4. Эволюция языка и стандарты

Язык C не был создан одномоментно. Он развивался на протяжении более чем 50 лет, и каждый стандарт отражал потребности своего времени. Для embedded-разработчика знание эволюции стандартов критически важно: от выбранного стандарта зависит, какие возможности языка доступны, какие конструкции считаются устаревшими, и насколько код будет переносим между разными платформами и компиляторами.

1.4.1. K&R C (1978) — «ранний» C

В 1978 году Брайан Керниган и Деннис Ритчи выпустили книгу **«The C Programming Language»**. Эта книга стала **де-факто** стандартом языка. Версию C, описанную в первом издании, называют **K&R C**.

Характерные особенности K&R C:

- Отсутствие прототипов функций. Объявления функций не содержали информации о типах аргументов.
- Отсутствие ключевого слова `void`. Функции без указанного типа возвращаемого значения по умолчанию возвращали `int`.
- Отсутствие ключевых слов `unsigned char`, `long`, `struct` в современном понимании.
- Присваивание структур было запрещено.

```
/* Пример функции в стиле K&R C */
int max(a, b)
int a, b;
{
    return a > b ? a : b;
```

```
}
```

Обратите внимание на синтаксис объявления параметров функции, который сегодня выглядит непривычным.



K&R C сегодня считается устаревшим. Современные компиляторы (GCC, Clang, IAR, Keil ARM Compiler) либо не поддерживают его, либо поддерживают с предупреждениями. Однако в legacy-коде такой синтаксис можно найти.

1.4.2. ANSI C / C89 / C90 (1989/1990) — первая стандартизация

К концу 1970-х язык C стал массовым, но появилось множество несовместимых компиляторов от разных производителей (IBM, DEC, Microsoft, Borland). Код, написанный для одного компилятора, мог не компилироваться на другом. Такая ситуация вообще-то нередко встречается; например, почти каждая реализация языка Pascal имела свои особенности. Однако для языка, сам смысл создания которого заключался в переносимости кода, это было явно недопустимо.

В **1989 году** Американский национальный институт стандартов (ANSI) выпустил стандарт **ANSI C**, также известный как **C89**. В **1990 году** этот стандарт был принят международной организацией ISO как **ISO/IEC 9899:1990 (C90)**. Оба стандарта практически идентичны.

Ключевые нововведения C89/C90:

- **Прототипы функций** — компилятор получил возможность проверять типы аргументов при вызове.
- Ключевые слова `void`, `const`, `volatile`, `signed`, `enum`.
- Стандартизированные библиотечные модули (`<stdio.h>`, `<stdlib.h>`, `<string.h>` и др.).
- Расширенные возможности препроцессора (операторы `и#`).

```
/* Пример функции в стиле C89/C90 */
int max(int a, int b)
{
    int result;           /* Объявление в начале блока */
    result = a > b ? a : b;
    return result;
}
```



Стандарт C89/C90 остаётся важнейшей вехой в истории языка. Многие компиляторы для микроконтроллеров (особенно старые версии IAR, Keil, Green Hills) до сих пор по умолчанию ориентированы именно на C99 с элементами C90. Огромный массив legacy-кода в embedded-индустрии написан в соответствии с C89.

1.4.3. C99 (1999) — современное лицо языка

Стандарт **ISO/IEC 9899:1999 (C99)** стал крупнейшим обновлением языка за два десятилетия. Он внёс множество улучшений, критически важных для embedded-разработки.

Ключевые нововведения C99:

- **Заголовок `<stdint.h>`** с типами **фиксированной разрядности**: `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`, `uintptr_t` и соответствующие им знаковые варианты. Это, пожалуй, самое важное нововведение для embedded-программиста, постоянно работающего на битовом уровне.
- **Заголовок `<stdbool.h>`** с типом `bool` и макросами `true`, `false`.
- **Заголовок `<inttypes.h>`** с макросами форматирования для `printf`/`scanf` (`PRIu32`, `SCNu16` и т.д.).
- **Переменные можно объявлять в любом месте блока**, а не только в начале.
- **Однострочные комментарии в стиле C++** (`// …`).
- **Ключевое слово `inline`** для встраиваемых функций.
- **Функции с переменным числом аргументов** (variadic functions) стали строже типизированы.
- **Массивы переменной длины** (VLA, Variable-Length Arrays) — **опциональная** возможность.
- **Составные литералы** и **назначенные инициализаторы** (designated initializers).
- **Тип `long long`** (гарантированно не менее 64 бит).
- **Функция `snprintf`** — безопасная альтернатива `sprintf`.

```
/* Пример кода в стиле C99 */
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint32_t id;
    char     name[16];
    bool     active;
} Device_t;

/* Назначенные инициализаторы */
Device_t dev = {
    .id     = 42,
    .name   = "UART1",
    .active = true
};

/* Объявление переменной в середине блока */
void process(void) {
    int x = 10;
```

```
    x += 5;
    int y = x * 2;   /* В C89 это было бы ошибкой */
}
```



Массивы переменной длины (VLA) были исключены из **обязательной** части стандарта C11 и переведены в разряд опциональных возможностей. Многие embedded-компиляторы (IAR, Keil) не поддерживают VLA из-за проблем с предсказуемостью использования стека. В профессиональной embedded-разработке VLA **не рекомендуются**.

1.4.4. C11 (2011) — многопоточность и безопасность

Стандарт **ISO/IEC 9899:2011 (C11)** внёс ряд дополнений и изменений, касающихся в том числе работы с потоками. потребности многопоточных систем и современные требования к безопасности.

Ключевые нововведения C11:

- **Модель памяти и многопоточность** — ключевые слова `_Atomic`, `_Thread_local`, заголовок `<stdatomic.h>`, `<threads.h>`.
- **Анонимные структуры и объединения** — упрощение вложенных структур.
- **Статические утверждения** (`_Static_assert`) — проверка условий на этапе компиляции.
- **Функции `_Alignof` и `_Alignas`** — управление выравниванием данных.
- **Макрос `STDC_VERSION`** обновлён до `201112L`.
- **Удаление функции `gets`** из стандартной библиотеки (источник уязвимостей, связанных с переполнением приемного буфера).
- **Поддержка Unicode** в строковых литералах.

```
/* Пример использования C11 */
#include <stdint.h>
#include <stdatomic.h>

/* Статическое утверждение: проверка размера структуры */
_Static_assert(sizeof(uint32_t) == 4, "uint32_t must be 4 bytes");

/* Атомарная переменная */
atomic_uint counter = 0;

void increment(void) {
    atomic_fetch_add(&counter, 1);
}

/* Анонимная структура */
typedef struct {
    uint32_t x;
    struct {           /* Анонимная вложенная структура */
```

```
        uint16_t a;
        uint16_t b;
    };
} Point_t;
```



Поддержка многопоточных возможностей C11 в embedded-среде ограничена. Большинство микроконтроллеров работают под управлением простых планировщиков или bare-metal, где многопоточность реализуется средствами RTOS (FreeRTOS, Zephyr), а не языковыми конструкциями. Однако `_Static_assert` и анонимные структуры активно используются в современных embedded-проектах.

1.4.5. C17 (2018) — «техническая коррекция»

Стандарт **ISO/IEC 9899:2018 (C17)**, также известный как C18, не внёс существенных изменений в язык. Это была «техническая коррекция» — устранение неоднозначностей, уточнение формулировок, исправление ошибок в **самом тексте** стандарта C11, а не в языке.

Макрос `STDC_VERSION` установлен в значение `201710L`.

Для embedded-разработчика C17 практически неотличим от C11. Большинство современных компиляторов (GCC 8+, Clang 7+, IAR 8+, Keil ARM Compiler 6+) по умолчанию или опционально поддерживают C17.

1.4.6. C23 (2023/2024) — современная модернизация

Стандарт **ISO/IEC 9899:2024 (C23)** — наиболее значительное обновление со времён C99. Он вносит множество улучшений, делающих язык более безопасным и удобным.

Ключевые нововведения C23:

- **Ключевые слова `true`, `false`, `nullptr`** — теперь встроены в язык, заголовок `<stdbool.h>` больше не требуется.
- **Атрибуты** в стиле C++ (, , ,).
- **Улучшенная работа с типами:** `typeof`, `typeof_unqual`, `nullptr_t`.
- **Улучшенный `enum`:** можно явно задавать базовый тип.
- **Удаление устаревших возможностей:** функция `gets` окончательно удалена, триграфы удалены.
- **Улучшения препроцессора:** оператор `#embed` для встраивания двоичных данных, `#warning`.
- **Расширенная поддержка `constexpr`** для константных выражений.
- **Улучшения в `<stdio.h>` и `<stdlib.h>`.**

```
/* Пример кода в стиле C23 */
#include <stdint.h>
```

```c
/* Атрибуты */
[[nodiscard]] int32_t calculate_checksum(const uint8_t *data, size_t len);

/* nullptr вместо NULL */
void *ptr = nullptr;

/* typeof */
#define MAX(a, b) \
    ({ typeof(a) _a = (a); \
       typeof(b) _b = (b); \
       _a > _b ? _a : _b; })

/* Улучшенный enum */
enum Status : uint8_t {
    STATUS_IDLE    = 0,
    STATUS_RUNNING = 1,
    STATUS_ERROR   = 2
};
```



Поддержка C23 в embedded-компиляторах находится на ранней стадии. В production-проектах использование C23 пока преждевременно.

1.4.7. Сводная таблица стандартов

Стандарт	Год	`STDC_VERSION`	Ключевые особенности для embedded
K&R C	1978	—	Базовый синтаксис, отсутствие прототипов
C89/C90	1989/1990	`199409L`	Прототипы, `void`, `const`, `volatile`, стандартная библиотека
C99	1999	`199901L`	`<stdint.h>`, `<stdbool.h>`, `inline`, VLA, назначенные инициализаторы
C11	2011	`201112L`	`_Atomic`, `_Thread_local`, `_Static_assert`, анонимные структуры
C17	2018	`201710L`	Технические исправления C11
C23	2024	`202311L`	`nullptr`, атрибуты, `typeof`, улучшения `enum`

1.4.8. Выбор стандарта в embedded-разработке

На практике embedded-разработчик редко выбирает стандарт языка произвольно. Выбор определяется:

1. **Компилятором.** Каждый компилятор поддерживает определённый набор стандартов. IAR C/C++ Compiler, ARM Keil, GCC для ARM, XC32 (Microchip), AVR-GCC — все они имеют

различную степень поддержки C99, C11, C17.

2. **Требованиями проекта.** Некоторые стандарты кодирования (MISRA C:2012, CERT C) накладывают ограничения на использование определённых возможностей языка.
3. **Legacy-кодом.** Если проект развивается более 10 лет, переход на новый стандарт может потребовать масштабного рефакторинга.
4. **Сертификацией.** В отраслях с высокими требованиями к безопасности (автомобильная ISO 26262, авиационная DO-178C, медицинская IEC 62304) использование новых возможностей языка требует дополнительного обоснования.



На 2026 год разумным выбором для нового embedded-проекта является **C11** или **C17** с включёнными расширениями компилятора. Это обеспечивает доступ к современным возможностям (`_Static_assert`, `<stdint.h>`, назначенные инициализаторы) при сохранении широкой совместимости с инструментами и библиотеками.

1.5. Влияние языка C на экосистему встраиваемых систем

Язык C оказал существенное влияние на всю экосистему встраиваемых систем:

1. **Доминирование в системном программировании.** Ядра ОС Linux и FreeBSD, RTOS, загрузчики, драйверы — всё это написано на C.
2. **Порождение семейства языков.** C++, Objective-C, C#, Java, JavaScript, Go, Rust используют C-подобный синтаксис.
3. **Инструментарий.** Компиляторы GCC, Clang, отладчики GDB, анализаторы кода изначально создавались для C.

Несмотря на появление более современных и безопасных языков (в первую очередь Rust), язык C по-прежнему удерживает позиции в embedded-разработке благодаря:

- **Зрелости инструментария** — компиляторы отлажены за десятилетия.
- **Предсказуемости** — отсутствие скрытых механизмов (сборщик мусора, исключения) критично для систем реального времени.
- **Обратной совместимости** — код 30-летней давности, как правило, компилируется современными компиляторами без переписывания.
- **Экосистеме** — миллионы строк проверенного кода, библиотеки, документация.

Chapter 2. Концепции

Прежде чем погрузиться в детали языка C, полезно будет ознакомиться с некоторыми общими концепциями программирования и применить их конкретно к C. Мой опыт проведения собеседований показывает, что не все эмбеддеры хорошо с ними знакомы. При этом знание концепций позволяет лучше понять, почему C именно такой в некоторых аспектах.

В этой главе мы познакомимся с несколькими теоретическими концепциями, которые не имеют непосредственного отношения к деталям программирования на C, а скорее определяют его философию: со специализацией языка, типизацией в языках программирования, способами выполнения кода программы на вычислительной машине, а также основными парадигмами программирования. В заключение определим, какие из этих концепций имеют отношение к C.

2.1. Интерпретация и компиляция

В принципе существует два принципиально разных способа выполнить программу на компьютере: **интерпретация** и **компиляция**. У каждого подхода имеются свои плюсы и минусы, поэтому оба имеют право на существование, каждый в своей нише.

2.1.1. Интерпретируемые языки

Интерпретация — это выполнение программы «на лету». Специальная программа — **интерпретатор** — читает исходный код строка за строкой и сразу же выполняет его.

Примеры интерпретируемых языков:

- Python
- JavaScript
- Ruby
- PHP
- Bash

Схематично этот процесс можно представить так:

```
Исходный код  →  [Интерпретатор]  →  Выполнение строка за строкой
```

Плюсы интерпретации:

- Код можно запустить сразу, без этапа компиляции (который может занимать довольно существенное время); это удобно для быстрого прототипирования.
- Программа переносима: если на компьютере есть подходящий интерпретатор, код заработает без изменений[1].
- Ошибки видны сразу при выполнении конкретной строки.

Минусы:

- Скорость выполнения значительно ниже, поскольку интерпретатор тратит время на разбор каждой строки.
- Исходный код нужно передавать пользователю (его сложно скрыть); это не всегда соответствует желаниям автора кода.
- Многие ошибки обнаруживаются только во время выполнения, когда программа уже запущена и выполняет содержащий ошибку фрагмент. Поэтому некоторые ошибки могут долгое время оставаться незамеченными.

2.1.2. Компилируемые языки

Компиляция — это перевод всей программы с языка высокого уровня (например, C) на машинный код **заранее**, до её запуска. На выходе получается готовый исполняемый файл (`.exe` (PE) в Windows, ELF в Linux, двоичный файл прошивки в случае bare-metal). Он содержит машинный код, который процессор может выполнять аппаратно.

Примеры компилируемых языков:

- C
- C++
- Go
- Rust
- Fortran
- Pascal

Упрощенная схема процесса:

```
Исходный код  →  [Компилятор]  →  Машинный код  →  Запуск
```

Плюсы компиляции:

- Высокая скорость выполнения — программа превращается с машинный код, с которым работает оборудование работает напрямую, без накладных расходов.
- Компилятор находит множество ошибок ещё до запуска[2].
- Исходный код не виден конечному пользователю — он получает только двоичный код; попытка восстановить исходный текст требует использования дорогого и трудоемкого процесса "реверс-инжениринга".

Минусы:

- Компиляция может занимать существенное время (особенно для больших и сложных проектов).
- Скомпилированный файл привязан к конкретной операционной системе и архитектуре процессора. Нужно собирать программу заново под конкретную ОС (Windows, Linux) и

архитектуру (Intel/AMD, ARM, RISC-V, MIPS и т.д.).

2.1.3. Сравнительная таблица

Критерий	Компилируемые (C)	Интерпретируемые (Python)
Скорость выполнения	Высокая	Низкая (в 10–100 раз медленнее)
Подготовка к выполнению	Долгое (нужна компиляция)	Мгновенное
Переносимость	Нужна перекомпиляция под каждую ОС	Работает везде, где есть интерпретатор
Диагностика ошибок	На этапе компиляции и во время выполнения	Только во время выполнения
Распространение	Достаточно бинарного файла	Нужен исходный код + интерпретатор

2.1.4. Третьего не дано?

Хотя картина получилась черно-белой (или-или), она изрядно упрощена, и реальность не всегда настолько контрастна. На самом деле есть множество оттенков между крайностями (интерпретацией и компиляцией).

- **Байт-код и виртуальные машины.** Java и C# сначала компилируются в промежуточный **байт-код**, который затем выполняется виртуальной машиной (JVM или CLR). Это компромисс: скорость работы программы ближе к компилируемым языкам, а переносимость — к интерпретируемым. Подход не нов: еще некоторые реализации языка Pascal компилировали не в машинный код, а в так называемый P-код. То же относится и к Visual Basic от Microsoft.
- **JIT-компиляция (Just-In-Time).** Интерпретатор во время работы «на лету» компилирует часто используемые участки кода в машинный код.
- **Промежуточное представление.** Компилятор GCC или Clang сначала превращает C-код в промежуточное представление (IR), оптимизирует его, и только потом генерирует машинный код.

2.2. Языки общего назначения и предметно-ориентированные языки

Языки программирования существенно отличаются друг от друга, иих можно классифицировать по многим критериям. Одна из фундаментальных классификаций — это разделение на **языки общего назначения** (General-Purpose Languages, GPL) и **предметно-ориентированные языки** (Domain-Specific Languages, DSL) в зависимости от их назначения.



Можно привести такую аналогию: GPL — это «мультитул», которым можно сделать (практически) всё; DSL — это специфическая оснастка, специально разработанная и изготовленная для одной конкретной операции.

2.2.1. Языки общего назначения

Это языки, созданные для решения максимально широкого круга задач в самых разных областях: от веб-разработки и мобильных приложений до системного программирования и искусственного интеллекта.

Ключевые характеристики

- **Тьюринг-полнота:** Могут вычислить любую вычислимую функцию (при наличии достаточного времени и памяти).
- **Универсальность:** На них можно написать практически что угодно без использования других языков: бизнес-приложение (десктоп или веб), сервер, игру, утилиту и т. п.
- **Развитая экосистема:** Стандартные библиотеки, фреймворки, инструменты отладки, сообщества.

Примеры

`Python`, `Java`, `C++`, `C#`, `JavaScript`, `Go`, `Rust`, `Swift`.

Плюсы и минусы

Плюсы:

- **Универсальность:** Изучив один язык, можно работать в разных областях.
- **Масштабируемость:** Подходят для создания сложных программных систем.
- **Инструментарий:** Удобные IDE, профилировщики, отладчики.

Минусы:

- **Семантический разрыв:** Средства языка не соответствуют понятиям решаемой задачи. Для решения конкретной задачи приходится писать много кода, который порой непросто понять.
- **Избыточная сложность:** Язык тащит за собой возможности, которые в конкретной задаче не нужны.

2.2.2. Предметно-ориентированные языки (Domain-Specific Languages, DSL)

Это языки, созданные для решения задач в **одной конкретной предметной области**. Они жертвуют универсальностью ради максимальной выразительности, краткости и безопасности в своем узком сегменте.

Ключевые характеристики

- **Узкая специализация:** Отлично решают одну задачу и обычно бесполезны для всех остальных.
- **Часто не являются Тьюринг-полными:** В них может не быть циклов или условных операторов, что намеренно ограничивает сложность и повышает безопасность.

- **Высокая абстракция:** Операции языка напрямую соответствуют терминам предметной области.

Примеры

- `SQL` — для работы с реляционными базами данных.
- `HTML` / `CSS` — для разметки и стилизации веб-страниц.
- **Регулярные выражения (Regex)** — для поиска текстовых шаблонов.
- `Make` / `CMake` / `Gradle` — для автоматизации сборки.
- `GraphQL` — для описания запросов к API.

Разновидности DSL

1. **Внешние (External DSL):** Имеют свой собственный, уникальный синтаксис и требуют написания собственного парсера или интерпретатора.
   - **Примеры:** `SQL`, `HTML`, `Regex`, `LaTeX`.
2. **Внутренние (Internal / Embedded DSL):** Не являются самостоятельными языками. Они используют синтаксис языка общего назначения (хостового языка), но применяют специфические библиотеки или паттерны, чтобы код выглядел как предметно-ориентированный.
   - **Примеры:** Конфигурация `Gradle` (на Kotlin/Groovy), `SQLAlchemy` в Python, `LINQ` в C#, тестовые фреймворки вроде `pytest`.

Плюсы и минусы

Плюсы:

- **Краткость и выразительность:** То, что на GPL занимает 50 строк, на DSL может занять 2 строки.
- **Читаемость для непрограммистов:** На DSL может писать специалист в предметной области, не владеющий навыками профессионального программирования (аналитик или даже сам заказчик).
- **Безопасность:** Ограниченный набор операций предотвращает целые классы ошибок, специфических для GPL.

Минусы:

- **Узость применения:** Невозможно использовать для других задач.
- **Стоимость создания:** Написание хорошего парсера и инструментов поддержки для внешнего DSL — это дорого.
- **Проблемы с отладкой:** При ошибке стек вызовов может быть непонятен, а инструменты отладки отсутствуют.

2.2.3. Сравнительная таблица

Критерий	Языки общего назначения (GPL)	Предметно-ориентированные языки (DSL)
Цель	Решить любую вычислительную задачу	Решить одну конкретную задачу в узкой области
Выразительность	Средняя (нужно описывать процедуру)	Очень высокая (в своей предметной области)
Тьюринг-полнота	Да (почти всегда)	Часто нет (но бывают исключения, например, XSLT)
Кривая обучения	Высокая (много концепций)	Низкая (в рамках своей области), но бесполезна вне её
Кто пишет код	Программисты	Программисты, аналитики, DevOps, специалисты в предметной области
Пример	Написать веб-сервер на `Python`	Описать маршрут в `Nginx config` или запрос в `SQL`

2.2.4. Современный тренд: Стирание границ

В современной разработке наблюдается интересный симбиоз:

1. **DSL внутри GPL:** Разработчики всё чаще создают внутренние DSL на базе языков вроде Kotlin, Scala или Rust, чтобы получить безопасность DSL, но сохранить мощные инструменты компиляции хостового языка.
2. **GPL поглощают возможности DSL:** Например, Python имеет встроенную поддержку регулярных выражений (`re`), а многие языки добавляют синтаксический сахар для асинхронности.
3. **Infrastructure as Code (IaC):** Такие инструменты, как Terraform (HCL), создают внешние DSL, которые становятся стандартом де-факто для целых индустрий (например, DevOps).

2.2.5. Итог

Архитектура современного приложения часто выглядит так: ядро и бизнес-логика написаны на языке общего назначения (`Java`, `Go`, `Rust`), а для конфигурации, запросов, сборки и описания правил используются тщательно подобранные DSL (`SQL`, `YAML`, `Regex`). Это позволяет использовать наиболее подходящий инструмент для каждой конкретной задачи.

2.3. Парадигмы

Каждый язык программирования поддерживает как минимум одну, а то и несколько **парадигм** одновременно.

Парадигма программирования — это философия, которая определяет, как вы структурируете код, как организуете данные и как управляете потоком выполнения программы.

Глобально все парадигмы делятся на два больших типа: 1. **Императивные** (отвечают на вопрос **«Как это нужно делать?»**). Мы подробно пошгово описываем действия, необходимые для получения желаемого реультата. 2. **Декларативные** (отвечают на вопрос **«Что мы хотим получить?»**). Мы описываем желаемый результат, не вдаваясь в детали пошагового выполнения.

Рассмотрим вкратце основные парадигмы.

2.3.1. Процедурная парадигма (императивная)

Это подход, основанный на понятии «процедуры» (или функции). Программа представляет собой подробную инструкцию, состоящую из последовательности шагов, которые компьютер должен выполнить от начала до конца для полчения желаемого результата. Процедурную программу часто сравнивают с кулинарным рецептом.

Плюсы:

- Простота для понимания
- Предсказуемость

Минусы:

- В больших проектах код трудно поддерживать.

Процедурные языки: C, Pascal, BASIC, Fortran.

2.3.2. Объектно-ориентированная парадигма (императивная)

Объектно-ориентированная парадигма (ООП) решает многие проблемы процедурного подхода в больших проектах, объединяя данные и операции над ними в единую сущность — **объект**. Программа рассматривается как набор взаимодействующих друг с другом (обычно посредством обмена сообщениями) объектов.

ООП базируется на четырех «китах»:

- **Инкапсуляция:** Скрытие внутреннего состояния объекта и деталей реализации. Доступ к данным осуществляется только через публичные методы (интерфейс).
- **Наследование:** Возможность создать новый объект на основе существующего, унаследовав его данные и поведение (переиспользование кода).
- **Полиморфизм:** Разные объекты, имеющие одинаковый интерфейс, могут реагировать на одно и то же сообщение по-разному.
- **Абстракция:** Выделение главных в данном контексте характеристик объекта и игнорирование второстепенных.

Первоначально ООП разрабатывалась как средство моделирования поведения объектов реального мира, когда каждый объект знает, как реагировать на то или иное сообщение. При этом отправитель сообщения может не вдаваться в детали, как реализован конкретный получатель, если он корректно реализует свой интерфейс.

В качестве аналогии можно привести отправку посылки через почтовое отделение. При этом отправитель имеет дело только с интерфейсом (окошком для приема отправлений), и его не интересует, какими видами транспорта будет доставлена его посылка и на каких промежуточных складах она будет храниться.

Плюсы:

- Хорошо подходит для моделирования сложных систем и реального мира
- Хорошая масштабируемость
- Удобство командной работы (за счет возможности распараллеливания работ над объектами)
- Возможность легкой замены одних объектов другими с тем же интерфейсом, но другой реализацией

Минусы:

- Более сложный код
- Для некоторых задач процедура ("рецепт") подходит лучше, чем сложная модель взаимодействующих объектов.

Языки ООП: Java, C#, C++, Smalltalk, Ruby.

2.3.3. Функциональная парадигма (декларативная)

Функциональное программирование (ФП) рассматривает вычисление как применение математических функций. Здесь нет понятий «переменная» в привычном смысле (как ячейка памяти, которую можно перезаписать) и нет «состояния».

Ключевые концепции:

- **Чистые функции:** Функция всегда возвращает одинаковый результат для одинаковых аргументов и не имеет **побочных эффектов** (в частности, не меняет глобальные переменные).
- **Неизменяемость (Immutable):** Данные создаются один раз и не могут быть изменены впоследствии. В случае изменгения создаются новые данные на основе старых.
- **Функции высшего порядка:** Функции могут приниматься как аргументы других функций и возвращаться как результат, то есть не отличаются в этом плане от обычных данных.

В качестве простого примера возьмем формулу `f(x) = x + 2`. Каждый раз, когда `x = 3`, мы будем получать в результате `5` при любых условиях.

Плюсы:

- Код легче тестировать
- Легче реализуются многопоточность и параллельные вычисления, так как нет гонок изменений состояния

- Выше надежность кода

Минусы:

- Крутая кривая обучения для программистов, которые успели привыкнутьк императивным парадигмам
- Плохо подходит для некоторых задач (например, для создания графического интерфейса, работы с вводом-выводом)

Языки: Haskell, Erlang, Lisp, Clojure, Elixir.

2.3.4. Логическая парадигма (декларативная)

В логическом программировании программа представляет собой набор **фактов** и **правил** (в виде логических утверждений). Программист не пишет алгоритм решения; вместо этого он описывает предметную область и задает системе **вопрос** (запрос). Система сама находит ответ, используя механизм логического вывода.

Основные области применения:

- Экспертные системы
- Искусственный интеллект
- Парсинг естественных языков
- Доказательство теорем

Языки: Prolog, Datalog.

2.3.5. Мультипарадигменные языки

Сегодня «чистые» парадигмы встречаются редко. Большинство популярных языков сегодня являются **мультипарадигменными**. Это значит, что они позволяют программисту выбирать наиболее подходящую парадигму под конкретную задачу.

- **Python:** Хорошо подходит для процедурного стиля(поэтому часто используется как скриптовый язык), при этом поддерживает ООП и имеет элементы ФП (лямбды, `map`, `filter`).
- **JavaScript:** Изначально прототипно-ориентированный (разновидность ООП), но сегодня в нем активно используется ФП (особенно в экосистеме React и Node.js).
- **C++:** Начинался как «C с классами» (процедурный + ООП), но в современных стандартах (C++11/14/17/20) добавлена поддержка функционального стиля.
- **Rust, Scala:** Языки, которые с самого начала проектировались как гибрид ООП и ФП.

2.3.6. Итоги

Компетентный программист широкого профиля должен владеть всеми парадигмами. Это позволит ему выбрать наиболее подходящую для данной задачи. Например:

- Хотите смоделировать бизнес-сущности (Пользователь, Заказ, Товар)? → **ООП**.
- Нужно обработать большой массив данных без побочных эффектов? → **ФП**.
- Пишете драйвер или код, где важна каждая миллисекунда и контроль над памятью? → **Процедурный подход**.

Для программиста, специализирующегося на эмбеде, я рекомендую в первую очередь изучить процедурную парадигму. Во многих случаях также уместен объектный подход, но важно соблюдать чувство меры и не создавать объекты ради объектов, а применять их там, где они реально дают более понятный код.

2.4. Типизация в языках программирования

Одно из важнейших свойств любого языка программирования — то, как он работает с **типами данных**.

Типизацию можно рассматривать по двум независимым осям:

- **Когда** проверяются типы — статическая или динамическая типизация.
- **Насколько строго** проверяются типы — сильная или слабая типизация.

Эти оси **независимы** друг от друга: язык может иметь статическую и сильную типизацию, динамическую слабую и любую другую комбинацию. Рассмотрим каждую ось отдельно.

2.4.1. Когда проверяются типы

Статическая типизация

При **статической типизации** типы всех переменных и выражений известны **на этапе компиляции** (то есть фактически должны быть заданы в тексте программы к этому моменту). Компилятор анализирует программу ещё до её запуска и проверяет, что все операции применяются к данным подходящих типов. Если проверка не пройдена, программа просто не скомпилируется.

Пример на Java:

```
int x = 10;
x = "hello";  // Ошибка компиляции: нельзя присвоить строку целочисленной переменной
```

Пример на Rust:

```
let x: i32 = 10;
let y: f64 = x;  // Ошибка компиляции: нельзя неявно преобразовать i32 в f64
```

Преимущества статической типизации:

- Ошибки обнаруживаются до запуска программы, а не в процессе её работы.

- Компилятор может генерировать более эффективный машинный код, зная точные типы данных.
- Инструменты разработки (автодополнение, рефакторинг, навигация по коду) работают гораздо лучше.

Недостатки:

- Приходится писать больше кода для описания типов.
- Меньше гибкости: переменная не может «на лету» изменить свой тип.

Примеры языков со статической типизацией: Java, C, C++, Rust, Go, Haskell.

Динамическая типизация

При **динамической типизации** типы проверяются **во время выполнения программы**. Переменная сама по себе не имеет типа — тип имеет **значение**, на которое она ссылается. Одна и та же переменная может сначала хранить число, а потом — строку.

Пример на Python:

```
x = 10        # x ссылается на целое число
x = "hello"   # теперь x ссылается на строку — и это нормально
```

Пример на Ruby:

```
x = 10        # x — целое число
x = "hello"   # x — строка, никаких проблем
```

Преимущества динамической типизации:

- Код короче и гибче.
- Быстрое прототипирование: не нужно заранее описывать типы.
- Легко писать обобщённые функции, работающие с данными разных видов.

Недостатки:

- Ошибки типов проявляются только во время выполнения.
- Сложнее обеспечивать корректность больших программ.
- Инструменты анализа кода работают хуже, потому что типы часто неизвестны заранее.

Примеры языков с динамической типизацией: Python, JavaScript, Ruby, PHP, Perl, Lua.

2.4.2. Насколько строго проверяются типы



Фактически это говорит о том, насколько "охотно" язык выполняет **неявные** преобразования между типами.

Сильная (строгая) типизация

При **сильной типизации** язык **не выполняет** неявных преобразований между разными типами. Если операция требует определённого типа, а вы передали другой — будет ошибка.

Пример на Python:

```
result = "5" + 3   # TypeError: нельзя сложить строку и число
```

Пример на Ruby:

```
result = "5" + 3   # TypeError: нет неявного преобразования Integer в String
```

Чтобы выполнить операцию, программист должен **явно** преобразовать типы (Пример на Python:):

```
result = int("5") + 3   # 8 — преобразование явное и видно в коде
```

Преимущества:

- Меньше скрытых багов: язык не пытается угадывать ваши намерения.
- Код предсказуем: явно видно, где и какие преобразования происходят.

Недостатки:

- Приходится писать больше явных преобразований.

Примеры языков с сильной типизацией: Python, Ruby, Haskell, Rust, Go, Erlang, Java (с оговорками).

Слабая (нестрогая) типизация

При **слабой типизации** язык **сам неявно преобразует** значения между типами, чтобы выполнить операцию. Часто это удобно, но иногда приводит к неожиданным результатам.

Пример на Javascript:

```
let result = "5" + 3;   // "53" — число неявно превратилось в строку
let another = "5" - 3;  // 2 — строка неявно превратилась в число
```

Пример на PHP:

```
$result = "5" + 3;      // 8 — строка неявно превратилась в число
```

Пример на C:

```
int x = 10;
double y = x;           // OK: неявное расширение int до double
```

Преимущества:

- Код короче: не нужно писать явные преобразования.
- Удобно в скриптовых задачах.

Недостатки:

- Скрытые баги: язык может «молча» сделать не то, что вы ожидали.
- Сложнее отлаживать: не всегда очевидно, где и какие преобразования произошли.

Примеры языков со слабой типизацией: C, C++, JavaScript, PHP, Perl, Visual Basic.

2.4.3. Четыре категории языков

Объединив две оси, получаем четыре категории языков:

	Сильная	**Слабая**
Статическая	Java, Rust, Go, Haskell	C, C++, Kotlin (с оговорками)
Динамическая	Python, Ruby, Erlang	JavaScript, PHP, Perl

Рассмотрим каждую категорию подробнее.

Статическая сильная типизация

Типы известны при компиляции, неявные преобразования запрещены или крайне ограничены.

Пример на Rust:

```
let x: i32 = 5;
let y: f64 = x;          // Ошибка: нельзя неявно преобразовать i32 в f64
let y: f64 = x as f64;   // OK: явное преобразование
```

Пример на Go:

```
var x int = 5
var y float64 = float64(x) // OK: явное преобразование
var z float64 = x          // Ошибка компиляции
```

Языки этой категории требуют от программиста максимальной дисциплины, но зато дают сильные гарантии корректности. Ошибки, связанные с некорректным использованием типов, практически исключены.

Статическая слабая типизация

Типы известны при компиляции, но компилятор охотно выполняет неявные преобразования.

Пример на C:

```
int x = 10;
double y = x;           // OK: неявное расширение int до double
void *p = &x;
int *q = p;             // OK: void* неявно преобразуется в любой указатель
```

Пример на C++:

```
void f(std::string s);
f("hello");  // OK: строковый литерал неявно преобразуется в std::string
```

Такие языки дают гибкость, но требуют внимательности: компилятор не всегда предупреждает о преобразованиях, которые могут привести к потере данных или неожиданному поведению.

Динамическая сильная типизация

Типы проверяются в рантайме, неявные преобразования запрещены.

Пример на Python:

```
x = "5"
y = 3
z = x + y       # TypeError: нельзя сложить str и int
```

Пример на Ruby:

```
x = "5"
y = 3
z = x + y       # TypeError: нет неявного преобразования Integer в String
```

Такие языки сочетают гибкость динамической типизации с предсказуемостью сильной: если что-то пошло не так, вы сразу получите понятную ошибку.

Динамическая слабая типизация

Типы проверяются во время выполнения, и язык сам неявно преобразует значения.

Пример на Javascript:

```
let x = "5";
let y = 3;
let z = x + y;   // "53" — число превратилось в строку
let w = x - y;   // 2 — строка превратилась в число
```

Пример на PHP:

```
$x = "5 apples";
$y = 3;
$z = $x + $y;   // 8 — строка неявно превратилась в число
```

Это самый «либеральный» режим: язык максимально старается выполнить операцию, даже если программист имел в виду что-то другое.

2.4.4. Практические следствия

Выбор языка с той или иной типизацией — это всегда компромисс. Вот несколько общих соображений:

- **Статическая типизация** предпочтительна для больших проектов, где важны надёжность и сопровождаемость. Она помогает находить ошибки на ранних этапах.
- **Динамическая типизация** удобна для прототипов, скриптов и небольших программ, где скорость разработки важнее гарантий.
- **Сильная типизация** снижает количество скрытых багов, потому что язык не «угадывает» ваши намерения.
- **Слабая типизация** сокращает объём явного кода, но требует внимательности: легко получить неожиданный результат.

2.4.5. Итоги

- Типизацию удобно рассматривать по двум независимым осям: **когда** проверяются типы (статическая / динамическая) и **насколько строго** (сильная / слабая).
- **Статическая типизация** проверяет типы при компиляции, **динамическая** — во время выполнения.
- **Сильная типизация** запрещает неявные преобразования между несовместимыми типами, **слабая** — выполняет их автоматически.
- Эти оси независимы: язык может быть любой из четырёх комбинаций.
- Граница между сильной и слабой типизацией неформальна и определяется тем, насколько охотно язык выполняет неявные преобразования.
- Выбор типа типизации — это всегда компромисс между гибкостью, безопасностью и объёмом кода.

Знание этих различий помогает лучше понимать философию языка, который вы изучаете.

2.5. Резюме

Пришла пора подвести итог и вернуться к языку C. Какое место он занимает в этом калейдоскопе?

Язык C является языком **общего назначения**, поддерживающим **процедурную** парадигму, со **слабой статической** типизацией. Обязательно запомните это, поскольку отсюда вытекают многие его свойства.

[1] Возможно, некоторые модификации все же придется сделать с учетом особенностей платформы

[2] Виды ошибок, которые можно обнаружить во время компиляции, сильно зависят от языка программирования

Chapter 3. Первая программа

При изучении языка программирования можно пойти двумя путями.

Первый путь, пожалуй, более правилен с академической точки зрения. Сначала мы заучиваем ключевые слова языка и основные концепции, затем учимся строить из них синтаксически корректные конструкции, и в конце долгого пути наконец получаем завершенную программу, готовую к выполнению. Но этот путь вряд ли будет быстрым и увлекательным. Дело в том, что мы сможем увидеть хоть какой-то результат лишь в самом конце изучения языка, а до этого момента придется обходиться без обратной связи.

Есть и второй путь. Он не столь совершенен с теоретической точки зрения, но гораздо увлекательнее первого. Мы начнем с самой простой из возможных программ, которая делает самое элементарное действие, которое мы можем обнаружить: печатает приветствие на экране. Несмотря на простоту, это будет полностью завершенная программа, которая содержит все те же элементы, что и гораздо более сложные программы. Возможно, не все из них будут очевидны с самого начала (хотя мы постараемся сделать их интуитивно понятными).

Составив программу, мы разберемся, какие действия нужно сделать, чтобы заставить компьютер выполнить ее. Когда все заработает, у нас будет рабочий каркас, который можно использовать для проверки действия только что изученных элементов языка. Мы просто будем добавлять их в работающую программу и смотреть полученный результат. Это даст нам возможность постепенно изучать язык и при этом получать быструю обратную связь. Этот подход хорошо зарекомендовал себя на практике в течение десятилетий, и мы им воспользуемся.

Приступим. Что делает воспитанный человек, входя в общество? Разумеется, здоровается. Давайте и мы поприветствуем мир языка C, в который мы вступаем.

3.1. Программа «Hello World!»

Пожалуй, это одна из самых простых программ, которая делает что-то осмысленное, а именно: выводит приветствие на консоль. Вот ее полный текст:

```
/* ①
Это многострочный комментарий.
Обычно подобный комментарий приводится в начале программы.
Он может описывать ее назначение, примененные алгоритмы,
имя автора, его контактные данные, лицензию и что угодно еще,
что может оказаться полезным читателю кода.

Компилятор полностью игнорирует комментарии.
*/

#include <stdio.h> ②
#define GREETING "Hello World!\n" ③
```

```
int main(void) ④
{ ⑤
    // Вывод приветствия на экран консоли ⑥
    printf(GREETING); ⑦⑧
    return 0; ⑨
} ⑩
```

① — многострочный комментарий.

② — включение заголовочного файла.

③ — определение текстового литерала через макрос.

④ — главная функция программы.

⑤ — скобка, открывающая функцию.

⑥ — однострочный комментарий.

⑦ — вывод приветствия на консоль.

⑧ — разделитель.

⑨ — возврат в операционную систему.

⑩ — скобка, закрывающая функцию.

Рассмотрим каждый из этих элементов немного подробнее, отложив детальное изучение до следующих глав.

3.2. Комментарии

В основном комментарии имеют смысл только для читающего текст программы человека. Копилятор игнорирует их содержимое. На самом деле, как мы увидим вскоре, они просто удаляются на одной из фаз компиляции.

В языке C используются два вида комментариев: онострочный и многострочный.

3.2.1. Многострочный комментарий

```
/*
Это многострочный комментарий.
Обычно подобный комментарий приводится в начале программы.
Он может описывать ее назначение, примененные алгоритмы,
имя автора, его контактные данные, лицензию и что угодно еще,
что может оказаться полезным читателю кода.

Компилятор полностью игнорирует комментарии.
*/
```

Многострочный комментарий начинается последовательностью литер `/*` и заканчивается последовательностью `*/`.

3.2.2. Однострочный комментарий

```
    // Вывод приветствия на экран консоли
```

Однострочный комментарий компактнее, но при этом его возможности более ограниченны. Он начинается последовательностью литер `//` и продолжается до конца строки.

3.3. Включение заголовочного файла.

```
#include <stdio.h>
```

Язык C чрезвычайно минималистичен. Подобно Микеланджело, его создатели также отсекали от глыбы мрамора все лишнее, оставив лишь самое необходимое.

Те, для кого C — не первый язык программирования, порой бывают удивлены, что в составе языка нет даже средства вывода сообщения на консоль. Хотя по сути удивляться годесь нечему: во встраиваемой системе может попросту не оказаться дисплея, пригодной для вывода строки текста, и его заменяют несколько светодиодных индикаторов. В таком случае не имеет смысла занимать скромные ресурсы микроконтроллера кодом поддержки работы с консолью, который может никогда не потребоваться.

Разработчики C пошли по другому пути. Они оставили минимальное ядро языка, к которому при необходимости подключаются библиотечные модули. Это позволяет не только избегать напрасной растраты ресурсов, но и выполнять тонкую настройку среды под конкретную реализацию аппаратуры.

Впрочем, при этом возникает одна проблема. Для того, чтобы вызвать библиотечную функцию, компилятор должен знать о ней необходимую информацию. Другими словами, нужно предварительно *объявить* функцию. К счастью, нет необходимости делать это вручную. Вместе с библиотеками в состав среды разработки входит набор файлов с готовыми объявлениями, так называемые *заголовки* (headers). Чтобы использовать функции из некоторой библиотеки, достаточно подключить к программе соответствующий файл заголовка. Именно это делает директива `#include <file.h>`.

В нашем случае к программе подключается заголовок `<stdio.h>` (standard input/output), в котором описаны средства стандартной библиотеки ввода/вывода. Это необходимо для того, чтобы впоследствии вызвать функцию `printf`.

В принципе файл заголовка можно подключить в любом месте до использования объявленной в нем функции, но по традиции все заголовки подключаются в начале программы. Мы сделает так же.

3.4. Определение текстовой строки через макрос

```
#define GREETING "Hello World!\n"
```

Эта конструкция связывает текст приветствия, выводимого на экран, с символическим именем `GREETING` (я пока намеренно не использую слово *идентификатор*, с которым мы познакомимся позже).

Текстовая строка заключается в двойные кавычки. Помимо литер, которые будут выведены на экран, в конце есть также последовательность `\n`, которая, как мы увидим вскоре, на экране отсутствует. Это так называемая *escape-последовательность*, которая заставляет консоль перейти на новую строку. На данный момент нам достаточно этих сведений.

Конструкция `#define` является *макроопределением* (или коротко *макросом*). В данном случае она связывает имя `GREETING` со значением "Hello World!\n", а именно: каждый раз, когда компилятор встречает в тексте программы слово `GREETING`, он подставляет вместо него "Hello World!\n". Сейчас не будем вдаваться в детали, *зачем* мы это делаем; язык вполне позволяет обойтись без этого и использовать непосредственно значения в нужных местах программы. Вернемся к этому вопросу позже в разделе, посвященном макросам и их использованию.

3.5. Главная функция программы

```
int main(void)
```

Язык C следует так называемой *парадигме структурного программирования*[1]. Ее суть заключается в том, что программа представляется в виде иерархической структуры *блоков*[2]. Одним из видов строительных блоков в парадигме структурного программирования является *подпрограмма*[3]; в языке C их называют *функциями*.

Про функции мы поговорим отдельно в соответствующем разделе. Сейчас же нам достаточно знать, что среди функций есть особенная с именем `main`. Именно с нее по умолчанию начинается выполнение программы.

Допустимо несколько вариантов объявления функции `main`. Для нашей цели вполне подходит простейший, в котором функция не принимает *параметров* (об этом говорит слово `void` в круглых скобках сразу после имени функции) и по завершении работы возвращает целочисленный результат (на это указывает слово `int` перед именем функции).

Куда передается результат работы программы?

Когда разработанная нами программа запускается под управлением ОС, у нее есть возможность вернуть ОС одно целочисленное значение (*код завершения*). Мы сами определяем семантику этого значения. Однако общепринято возвращать значение `0`, если программа отработала и завершилась нормально, и ненулевой код ошибки, если что-то пошло не так. Коды ошибки определяет сам автор программы, и правила хорошего тона диктуют, что он должен описать их в документации к своей программе.

Если же программа выполняется на "голом железе" (bare-metal), она никогда не

заканчивается, и использовать механизм кода завершения не имеет смысла.

3.6. Открывающая и закрывающая функцию скобки

Сразу же после объявления функции `main` находится открывающая фигурная скобка `{`. С этой точки начинается код функции и продолжается до парной ей закрывающей скобки `}`.

Именно фигурные скобки определяют структуру программы на C. Во времена Кернигана и Ритчи в качестве консоли обычно использовалась пишущая машинка, поэтому для экономии бумаги открывающую скобку размещали в той же строке,что и объявление функции. На видеомониторе экономить бумагу нет необходимости, поэтому встречается другой вариант: открывающая скобка располагается на отдельной строке, над парной к ней закрывающей. какой вариант использовать - исключительно дело вкуса, поскольку это оказывает влияние только на читаемость кода. Компилятор C (в отличие, например, от интерпретатора Python) не обращает никакого внимания на форматирование исходного текста, поэтому вы можете выбрать тот вариант, который вам нравится, или даже тзобрести свой собственный.

3.7. Вывод приветствия на консоль

```
    printf(GREETING);
```

Одним из способов вывода сообщения на консоль является вызов библиотечной функции `printf`. В отличие от `main` мы не пишем ее исходный код, поскольку он уже присутствует в одной из стандартных библиотек, поставляемых вместе с компилятором[4].

Эта функция обладает очень большими возможностями, и ее полное описание займет большую главу. В простейшем случае (нашем) функция имеет единственный строковый арогумент и просто выводит его на консоль (не забываем про escape-последовательности!).

После завершения работы этой функции на консоли в текущей позиции появится сообщение:

Hello World!

После этого произойдет переход на новую строку.

3.8. Разделитель

Обратите внимание на разделитель (`;`) после вызова функции `printf`.

В языке C этот разделитель используется для отделения инструкций друг от друга. Его отсутствие приведет к ошибке компиляции.

3.9. Возврат в операционную систему

```
    return 0;
```

При объявлении функции `main` мы указали, что в конце работы функция вернет целочисленное значение, которое будет интерпретировано операционной системой как код возврата программы. Инструкция `return` делает именно это - завершает выполнение `main` и возвращает следующее за ней значение ОС.

В конце инструкции мы также видим обязательный разделитель `;`.

Итак, мы составили первую небольшую, но завершенную программу на языке С. Следующим логичным шагом будет ее выполнить и увидеть результат. Этим мы займемся в следующей главе.

[1] https://tinyurl.com/dcjhd75m

[2] https://tinyurl.com/2rv5xzc3

[3] https://tinyurl.com/4d8463b3ъ

[4] Функция `printf` не всегда доступна, поскольку в минимальной конфигурации в системе может просто не быть консоли.

Chapter 4. Компиляция и выполнение кода C

Итак, у нас есть наша первая программа на языке C. Самое время попытаться выполнить ее и увидеть результат.

Язык C является **компилируемым** языком. Это означает, что написанный нами исходный текст на языке C не выполняется непосредственно. Сначала он должен быть преобразован в машинный код — набор инструкций, который может выполнить процессор.

В этой главе мы подробно разберем путь, который проходит программа от исходного текста до исполняемого машинного кода. В качестве примера возьмем классический компилятор C из набора GCC (GNU Compiler Collection).

4.1. Выбор компилятора

Из множества компиляторов C в качестве примера для обучения я выбрал GCC. Этот выбор не случаен. Он обусловлен несколькими причинами:

4.1.1. Свободное распространение и доступность

GCC (GNU Compiler Collection) — это свободное программное обеспечение с лицензией GPL. Это означает:

- **Бесплатность**: Любой студент, преподаватель или разработчик может установить GCC без каких-либо лицензионных ограничений
- **Кроссплатформенность**: GCC работает на Linux, Windows (через MinGW/MSYS2), macOS, BSD и многих других системах
- **Открытый исходный код**: При желании можно изучать, как работает сам компилятор

4.1.2. Стандарт де-факто в мире Unix/Linux

В экосистеме Linux и Unix-подобных систем GCC является основным компилятором:

- Ядро Linux компилируется с помощью GCC
- Большинство дистрибутивов Linux используют GCC по умолчанию
- Значительная часть инфраструктуры open-source проектов построена вокруг GCC

Изучая GCC, вы изучаете инструмент, который используется в реальной индустрии.

4.1.3. Образовательная ценность

GCC имеет богатую историю (первая версия выпущена в 1987 году) и хорошо документирован:

- **Подробная документация**: Официальные мануалы, туториалы, книги

- **Прозрачность работы**: Можно увидеть все этапы компиляции (препроцессинг, компиляция, ассемблирование, компоновка)
- **Богатый набор флагов**: GCC предоставляет управляемый флагами доступ ко всем фазам компиляции, что идеально для обучения

4.1.4. Совместимость со стандартами

GCC строго следует стандартам языка C[1]:

- Поддерживает C89/C90, C99, C11, C17, C23[2]
- Имеет флаги для строгого соответствия стандартам (`-std=c11`, `-pedantic`)
- Предоставляет предупреждения о нестандартном коде (`-Wall`, `-Wextra`)

4.1.5. Экосистема инструментов

GCC — это не просто компилятор, а часть большой экосистемы GNU:

- **Binutils**: `as` (ассемблер), `ld` (линкер), `objdump`, `objcopy`, `nm`, `readelf`
- **GDB**: Отладчик, тесно интегрированный с GCC
- **Make**: Система сборки, которая работает с GCC "из коробки"

4.1.6. Применение в embedded

GCC реализован в множестве разных платформ, в том числе используемых во встраиваемых системах:

- **ARM GCC** (`arm-none-eabi-gcc`) — для микроконтроллеров ARM
- **AVR GCC** — для микроконтроллеров AVR
- **RISC-V GCC** — для архитектуры RISC-V
- **MIPS GCC**, **SPARC GCC** и другие

4.1.7. Возможные альтернативы

помимо GCC, существуют и другие компиляторы C.

Clang/LLVM

- **Преимущества**: Быстрее компилирует, лучше сообщения об ошибках, модульная архитектура
- **Недостатки**: Меньше флагов оптимизации для специфичных архитектур
- **Использование**: macOS (по умолчанию), FreeBSD, некоторые embedded проекты
- **Совместимость**: В большинстве случаев полная замена GCC (те же флаги)

MSVC (Microsoft Visual C++)

- **Преимущества**: Лучшая интеграция с Windows, отладчик Visual Studio
- **Недостатки**: Только под Windows, проприетарный, отличается синтаксис некоторых расширений
- **Использование**: Разработка под Windows
- **Особенности**: Другой синтаксис для некоторых pragma, другой формат объектных файлов (PE/COFF)

Intel C++ Compiler

- **Преимущества**: Оптимизация для процессоров Intel
- **Недостатки**: Платный, одноплатформенный (Intel), специфичные оптимизации
- **Использование**: Высокопроизводительные вычисления (HPC)

Коммерческие компиляторы для embedded

- **IAR Embedded Workbench**, **Keil MDK**, **Green Hills MULTI**
- **Преимущества**: Специализированные оптимизации, поддержка конкретных микроконтроллеров
- **Недостатки**: Очень дорогие, проприетарные, отсутствуют версии для PC[3]

Заключение

Мы используем GCC, потому что это:

- Бесплатный и доступный инструмент
- Стандарт индустрии для Linux и поддержка многих платформ embedded
- Образовательный инструмент с прозрачной работой
- Часть экосистемы GNU

Однако принципы, которые мы будем рассматривать дальше, **универсальны** и применимы практически к любому компилятору C. Различаются только конкретные флаги командной строки и названия некоторых артефактов.

4.2. Фазы компиляции

Процесс преобразования исходного кода в исполняемый файл в GCC традиционно делится на четыре основные фазы. Рассмотрим их на примере нашей простой программы `hello_world.c`, состоящей из единственного файла. Для наглядности выполним это вручную, из командной строки, чтобыподробно видеть весь процесс.

4.2.1. Препроцессор

Первым в игру вступает **препроцессор**. Он обрабатывает макродирективы (макросы), начинающиеся с литеры `#` (например, `#include`, `#define`, `#ifdef`):

- Раскрывает макросы (заменяет обращение к макросу на его содержимое, указанное в определении макроса).
- Подставляет содержимое заголовочных файлов вместо директив `#include`, как если бы сам разработчик скопипастил этот текст в свой код.
- Удаляет комментарии. Компилятор их даже не увидит.
- Обрабатывает условную компиляцию (удаляет код, для которого не выполняются определенные истории).

Язык макросов слишком сложен, чтобы описать его в этом разделе. Поэтому позднее мы рассмотрим его более подробно, а сейчас ограничимся минимумом для понимания общей картины.

Обычно результат работы препроцессора не виден программисту, он сразу же передается по цепочке дальше. Но при желании можно завершить компиляцию на этом этапе и рассмотреть, как выглядит обработанный препроцессором код. Иногда это нужно, когда разработчик перемудрил со слишком сложными макросами и хочет разобраться, что же происходит на самом деле. Также полезно посмотреть на обработанный код, чтобы лучше понять все происходящее при компиляции.

Чтобы остановить компиляцию на этапе препроцессора и получить результат его работы, используется флаг `-E`:

```
gcc -E hello_world.c -o hello_world.i
```

Файл `hello_world.i` будет содержать уже обработанный препроцессором код на языке C, готовый к компиляции.

4.2.2. Компиляция

На этом этапе компилятор анализирует синтаксис обработанного препроцессором кода, строит абстрактное синтаксическое дерево (AST), проводит оптимизации и переводит код с языке C в **ассемблерный код** для целевой архитектуры.

Мы можем остановить компиляцию на этом шаге и посмотреть результат. Для этого воспользуемся флагом `-S`:

```
gcc -S hello_world.i -o hello_world.s
```

Файл `hello_world.s` содержит текстовое представление инструкций процессора, на котором будет выполняться программа (например, x86_64 или ARM).

4.2.3. Ассемблирование

Ассемблер переводит текстовый ассемблерный код, полученный в результате компиляции, в **объектный файл**.

Объектный файл содержащий машинный код, но он еще непригоден для выполнения. Во-первых, все адреса в нем заданы относительно некоторой начальной точки. Во-вторых, в нем отсутствует код из библиотек, которые мы включили посредством `#include`. Вместо этого в нем присутствует таблица символов, посредством которой мы можем использовать код из внешних модулей и код из внешних модулей может использовать наш, и некоторые метаданные для компоновки.

Флаг `-c` говорит GCC ассемблировать код, но не выполнять компоновку:

```
gcc -c hello_world.s -o hello_world.o
```

4.2.4. Компоновка

Компоновщик (линкер) берет один или несколько объектных файлов, составляющих программу, а также использованные в ней библиотеки и объединяет их в единый **исполняемый файл**, разрешает внешние ссылки (связывает вызовы функций из других модулей с их реальными адресами в памяти) и формирует финальную структуру файла.

```
gcc hello_world.o -o hello_world
```

Мы получили исполняемый файл `hello_world`.

4.2.5. Выполнение программы

Мы можем запустить нашу программу на выполнение:

```
$./hello_world
Hello World!
```

Мы полностью прошли путь от исходного кода на C до исполняемого файла и получили вывод приветствия на консоль.

4.2.6. То же самое, но быстрее

Мы намеренно компилировали нашу первую программу неторопливо, по шагам, чтобы понять суть происходящих процессов и увидеть результаты кажждого из них. Тот же результат можно было получить быстрее, в одно действие:

```
gcc hello_world.c -o hello_world
```

GCC последовательно выполнит все четыре фазы.

4.3. Независимая компиляция модулей

Одним из главных преимуществ языка C является возможность **независимой компиляции** (раздельной трансляции). Эта крайне полезная возможность предусмотрена не во всех языках. Например, классический Pascal требует, чтобы весь код программы находился в одном файле.

Программа на C может быть разделена между несколькими файлами. Это можно использовать для того, чтобы разбить сложную логику на несколько слабо зависящих друг от друга модулей меньшего размера, чтобы было проще их читать и редактировать.

Каждый файл `.c` вместе со всеми включенными в него заголовочными файлами образует **единицу трансляции** (translation unit). Компилятор обрабатывает каждую единицу трансляции независимо, не зная о существовании других файлов `.c` в проекте. Для связи между единицами трансляции нужны лишь заголовочные файлы `.h`? содержащие необходимые для компиляции объявления и определения (наподобие `stdio.h`).

```
# Компилируем модули независимо друг от друга
gcc -c main.c -o main.o
gcc -c math_ops.c -o math_ops.o
gcc -c utils.c -o utils.o
```

Преимущества независимой компиляции:

- **Скорость сборки:** При изменении кода в `math_ops.c` нет необходимости перекомпилировать `main.c` и `utils.c`. Перекомпилируется только измененный модуль, после чего объектные файлы быстро перекомпоновываются. В больших проектах (миллионы строк кода) это экономит часы времени.
- **Модульность и инкапсуляция:** Функции, объявленные в `.c` файле без ключевого слова `extern` (или с `static`), не видны за пределами своей единицы трансляции (подробнее разберем далее). Это позволяет скрывать внутреннюю реализацию и при необходимости менять ее, не затрагивая структуру остальных модулей.
- **Повторное использование:** Скомпилированные объектные файлы или библиотеки можно использовать в других проектах без их исходного кода.

4.4. Использование библиотек

Когда независимых модулей становится много, их можно объединить в **библиотеки** для удобства. Библиотеки бывают двух основных типов: **статические** и **динамические**.

4.4.1. Статические библиотеки

Статическая библиотека (в Linux имеет расширение `.a`, в Windows `.lib`) — это просто набор объектных файлов (`.o`), собранных для удобства в один архив. Она создается с помощью утилиты `ar` (archiver):

```
ar rcs libmath_ops.a math_ops.o utils.o
```

При компоновке исполняемой программы компоновщик берет из статической библиотеки **только те объектные модули**, которые необходимы для разрешения текущих внешних ссылок, и вставляет их машинный код напрямую в исполняемый файл.

Плюсы статических библиотек:

- Получаем монолитный исполняемый файл, который является самодостаточным.

Минусы:

- Если мы используем одни и те же библиотеки во многих программах, в каждую из них копируется один и тот же код. Это увеличивает необходимый для их хранения объем памяти.
- При внесении изменений в библиотеки необходимо заново собрать все программы, в которых они используются. Это особенно важно в случае, если изменения вносятся с целью исправления обнаруженных в ходе эксплуатации программы ошибок.

4.4.2. Динамические (разделяемые) библиотеки

Динамическая библиотека (в Linux `.so`, в Windows `.dll`) компилируется в отдельный бинарный файл. Машинный код библиотеки **не копируется** в исполняемый файл программы. Вместо этого при запуске программы загрузчик находит библиотеку в памяти или на диске и подгружает её в память, настраивая все ссылки на нее в соответствии с адресом загрузки.

Создание динамической библиотеки:

```
gcc -shared -fPIC math_ops.o utils.o -o libmath_ops.so
```

Плюсы:

- Библиотечный код находится в отдельном от основной программы файле с исполняемым кодом, не увеличивая размер исполняемой программы.
- Можно изменять код динамической библиотеки (например, с целью оптимизации или исправления ошибок), не затрагивая исполняемый код основной программы.
- Некоторые готовые динамические библиотеки входят в состав ОС или доступны к загрузке от сторонних поставщиков. Их можно использовать как "черный ящик", не тратя время на разработку.
- Можно загружать динамические библиотеки в память при необходимости. Если в текущем сеансе работы какая-то функция не используется, она не будет загружена, тем самым не потребляя необходимые для ее работы ресурсы.

Минусы:

- Усложняется распространение двоичного кода программы. Помимо исполняемой программы, необходимо обеспечить наличие всех используемых разделяемых библиотек. Это может быть непростой задачей, особенно если разные программы используют разные несовместимые версии разделяемых библиотек.
- Нельзя использовать разделяемые библиотеки в прошивках для микроконтроллеров, поскольку весь исполняемый код должен быть загружен в постоянную память в процессе заливки программы.

> Флаг `-fPIC` означает Position Independent Code — позиционно-независимый код, необходимый для динамических библиотек.

4.4.3. Некоторые флаги GCC для работы с библиотеками

Чтобы скомпоновать программу, использующую библиотеки, GCC нужно указать, где искать заголовочные файлы и сами библиотеки:

`-I<путь>` **(Include)**

— указывает директорию для поиска заголовочных файлов (`#include`).

`-L<путь>` **(Library)**

— указывает директорию для поиска библиотек при компоновке.

`-l<имя>` **(library)**

— указывает имя библиотеки для компоновки. Имя пишется без префикса `lib` и расширения. Например, для `libmath_ops.a` флаг будет `-lmath_ops`.

Пример:

```
# Компиляция с использованием заголовков из ./include и библиотек из ./lib
gcc main.c -I./include -L./lib -lmath_ops -o my_app
```

4.5. Полезные артефакты компиляции

В процессе компиляции генерируется множество промежуточных файлов. Каждый из них содержит информацию, которая может оказаться полезной в определенных условиях. Понимание этих артефактов помогает в отладке, оптимизации и анализе программы.

Далее вкратце описаны наиболее часто используемые артефакты компиляции, способы их получения и использования.

4.5.1. Объектный код

Объектный файл (обычно с расширением `.o` в Unix/Linux или `.obj` в Windows) — это бинарный файл, содержащий машинный код, сгенерированный компилятором из одной единицы трансляции (одного `.c` файла).

Объектный файл содержит:

- **Машинный код** (инструкции процессора);
- **Таблицу символов** (имена функций и глобальных переменных);
- **Таблицу перемещений** (информация о ссылках на внешние символы, которые должны быть разрешены линкером);
- **Секции** (код (`.text`), инициализированные данные (`.data`), неинициализированные данные (`.bss`) и другие);
- **Отладочную информацию** (если компиляция проводилась с ключом с `-g`).

Получение объектного файла:

```
# Компиляция в объектный файл
gcc -c module.c -o module.o

# Просмотр содержимого объектного файла
objdump -d module.o         # Дизассемблирование кода
nm module.o                 # Таблица символов
nm -u module.o              # Проверка неопределенных символов (внешних ссылок)
nm -u module.o              # Проверка неопределенных символов (внешних ссылок)
nm --defined-only module.o  # Проверка экспортируемых символов
readelf -a module.o         # Полная информация об ELF файле
objdump -h module.o         # Информация о секциях
```

4.5.2. Загрузочный модуль

Загрузочный модуль — это готовый к выполнению исполняемый файл, полученный после компоновки. В отличие от объектного файла, все внешние ссылки разрешены, и код готов к загрузке в память процессором.

Форматы загрузочных модулей:

ELF (Executable and Linkable Format)

— стандарт для Linux и многих Unix-систем

PE/PE32+ (Portable Executable)

— формат для Windows

Mach-O

— формат для macOS и iOS

COFF (Common Object File Format)

— устаревший формат, предшественник PE

Получение загрузочного модуля:

```
# Компоновка в исполняемый файл
```

```
gcc main.o module.o -o program

# Проверка типа файла
file program
```

Анализ загрузочного модуля:

```
# Динамические зависимости (для ELF)
ldd program

# Информация о сегментах памяти
readelf -l program

# Информация о секциях
readelf -S program

# Размер секций
size program
```

4.5.3. Листинг компиляции

Листинг — это текстовый файл, содержащий исходный код программы вместе с сгенерированным машинным кодом. Это мощный инструмент для анализа того, как компилятор преобразует ваш код.

Получение листинга:

```
# Генерация ассемблерного файла
gcc -S -O2 module.c -o module.s

# Просмотр содержимого
cat module.s
```

Для более наглядного анализа можно получить листинг, где исходный код на C чередуется с ассемблером:

```
# Компиляция с генерацией смешанного листинга
gcc -c -g -Wa,-a,-ad module.c -o module.o > module.lst

# Или использование objdump
objdump -d -S -l module.o > module_mixed.lst
```

4.5.4. Карта памяти

Карта памяти (map file) — это текстовый файл, генерируемый линкером, который показывает, как различные модули и символы размещены в памяти итоговой программы.

Получение карты памяти:

```
# Генерация map файла при компоновке
gcc main.o module.o -o program -Wl,-Map=program.map
```

Файл карты памяти содержит несколько важных секций:

1. Распределение памяти
2. Размещение секций
3. Таблица символов
4. Статистика использования памяти

Файл карты памяти помогает:

- Найти, где размещены конкретные функции и переменные
- Определить размер каждой секции
- Выявить дублирование кода
- Оптимизировать использование памяти
- Найти "тяжелые" модули, занимающие много места

4.5.5. Файлы зависимостей

При использовании `make` или других систем сборки важно отслеживать, какие заголовочные файлы включены в каждый `.c` файл. Это позволяет перекомпилировать только те модули, которые действительно изменились.

Генерация файлов зависимостей:

```
# Компиляция с генерацией файла зависимостей
gcc -MMD -MP -c module.c -o module.o
```

Файл `.d` содержит список всех файлов, от которых зависит объектный файл. При изменении любого из этих файлов необходимо перекомпилировать модуль.

4.5.6. Отладочная информация

При компиляции с флагом `-g` компилятор добавляет в объектный и исполняемый файлы отладочную информацию, позволяющую отладчику связывать машинный код с исходным кодом.

Форматы отладочной информации:

DWARF (Debugging With Attributed Record Formats)

— стандарт для Unix/Linux

PDB (Program Database)

— формат для Windows (MSVC)

STABS

— устаревший формат

Получение отладочной информации^

```
# Компиляция с отладочной информацией
gcc -g -O0 module.c -o module.o

# Извлечение отладочной информации
objdump --dwarf=info module.o > debug_info.txt
readelf --debug-dump=info module.o > debug_info.txt
```

Отладочная информация может зхначительно увеличить размер исполняемого файла. Для уменьшения его размера отладочную информацию можно вынести в отдельный файл:

```
# Извлечение отладочной информации
objcopy --only-keep-debug program program.debug

# Удаление отладочной информации из исполняемого файла
strip --strip-debug --strip-unneeded program

# Добавление ссылки на файл отладки (для GDB)
objcopy --add-gnu-debuglink=program.debug program
```

4.5.7. Файлы покрытия кода

При тестировании программы можно собирать статистику покрытия кода — какие строки и ветки кода выполнялись во время работы программы.

Генерация файлов покрытия:

```
# Компиляция с поддержкой покрытия кода
gcc --coverage -fprofile-arcs -ftest-coverage module.c -o module

# Это создаст файлы:
# module.gcno - содержит информацию о структуре программы (генерируется при
компиляции)

# Запуск программы
./program

# После выполнения создаются файлы:
# module.gcda - содержит информацию о выполнении (генерируется при запуске)
```

```
# Генерация отчета о покрытии
gcov module.c
```

Файл `.gcov` содержит исходный код с пометками, какие строки выполнялись.

4.5.8. Препроцессированный код

Иногда полезно увидеть результат работы препроцессора — полностью раскрытый код со всеми макросами и включенными заголовочными файлами.

Получение препроцессированного кода:

```
# Генерация препроцессированного файла
gcc -E module.c -o module.i

# Или с сохранением комментариев
gcc -E -C module.c -o module_with_comments.i
```

Препроцессированный файл может быть очень большим (десятки тысяч строк). Его имеет смысл создавать, если вы собираетесь:

- Отлаживать сложные макросы
- Разобраться, какие заголовочные файлы включены
- Проанализировать результаты условной компиляции

4.5.9. Файлы промежуточного представления

Промежуточное представление (Intermediate Representation) используется для оптимизации кода. Можно получить доступ к IR на разных этапах:

```
# Генерация RTL (Register Transfer Language) для GCC
gcc -fdump-rtl-all module.c

# Генерация GIMPLE (высокоуровневое IR)
gcc -fdump-tree-gimple module.c
```

4.5.10. Файлы профилирования

Для оптимизации производительности можно собирать профилировочную информацию:

```
# Компиляция с профилированием
gcc -pg module.c -o module

# Запуск программы (создает файл gmon.out)
./program
```

```
# Анализ профилирования
gprof module gmon.out > profile.txt
```

4.5.11. Сводная таблица артефактов

Артефакт	Расширение	Назначение
Исходный код	`.c`, `.h`	Исходный текст программы
Препроцессированный код	`.i`	Код после работы препроцессора
Ассемблерный код	`.s`	Ассемблер для целевой архитектуры
Объектный файл	`.o`, `.obj`	Машинный код одной единицы трансляции
Загрузочный модуль	ELF, PE, Mach-O	Готовый к выполнению исполняемый файл
Карта памяти	`.map`	Информация о размещении в памяти
Файл зависимостей	`.d`	Список зависимостей для make
Отладочная информация	`.debug`, PDB	Информация для отладчика
Файлы покрытия	`.gcno`, `.gcda`	Статистика покрытия кода
Бинарный образ	`.bin`, `.hex`	Образ для прошивки в embedded системах

4.5.12. Резюме

Артефакты компиляции — это не просто промежуточные файлы, а мощные инструменты для анализа, отладки и оптимизации программы:

- Объектные файлы содержат машинный код и метаданные для компоновки.
- Загрузочные модули готовы к выполнению.
- Листинги позволяют увидеть, как компилятор преобразует ваш код.
- Карты памяти показывают структуру итоговой программы.
- Файлы зависимостей позволяют упорядочить и ускорить сборку.
- Отладочная информация связывает машинный код с исходным.

Понимание и умение использовать эти артефакты — важный навык для любого разработчика на C. Рекомендую попрактиковаться в их получении и интерпретации их содержимого.

4.6. Выполнение программы на C

Теперь, когда мы, во-первых, изучили устройство простой программы на языке C и, во-вторых, теоретически разобрались, как происходит компиляция, самое время попробовать применить эти знания на практике. Выполним наш код `hello_world.c`.

Это можно сделать несколькими различными способами. Каждый из них обладает своими достоинствами и недостатками. Мы попробуем несколько способов, чтобы лучше понять особенности каждого и условия, при которых он дает наилучший результат по сравнению с остальными.

4.6.1. Использование онлайн компиляторов

Онлайн компиляторы — это веб-сервисы, которые позволяют писать, компилировать и запускать код прямо в браузере без установки какого-либо программного обеспечения. Все, что вам нужно, - это наличие терминала с поддержкой web (подойдет даже планшет с достаточной диагональю экрана) и стабильное соединение с интернетом. Поскольку основную работу будет выполнять удаленный сервер, вычислительная мощность вашего устройства не имеет особого значения.

Некоторые онлайн компиляторы:

OneCompiler (https://onecompiler.com)

— пожалуй, один из простейших вариантов с минимумом опций

OnlineGDB (https://onlinegdb.com)

— поддержка C/C++, отладчик

GodBolt Compiler Explorer (https://godbolt.org)

— позволяет исследовать ассемблерный код, порождаемый разными компиляторами

Этим списком отнюдь не исчерпывается набор доступных онлайн компиляторов. Некоторые из них настолько просты, что вряд ли способны на что-то большее, чем `Hello World`. Другие позволяют выбирать компилятор, версию стандарта языка и даже предоставляют сервис полноценной IDE. Выможете подобрать для себя наиболее соответствующий вашим потребностям.

Пример работы с **OneCompiler**:

1. Заходим на сайт https://onecompiler.com:

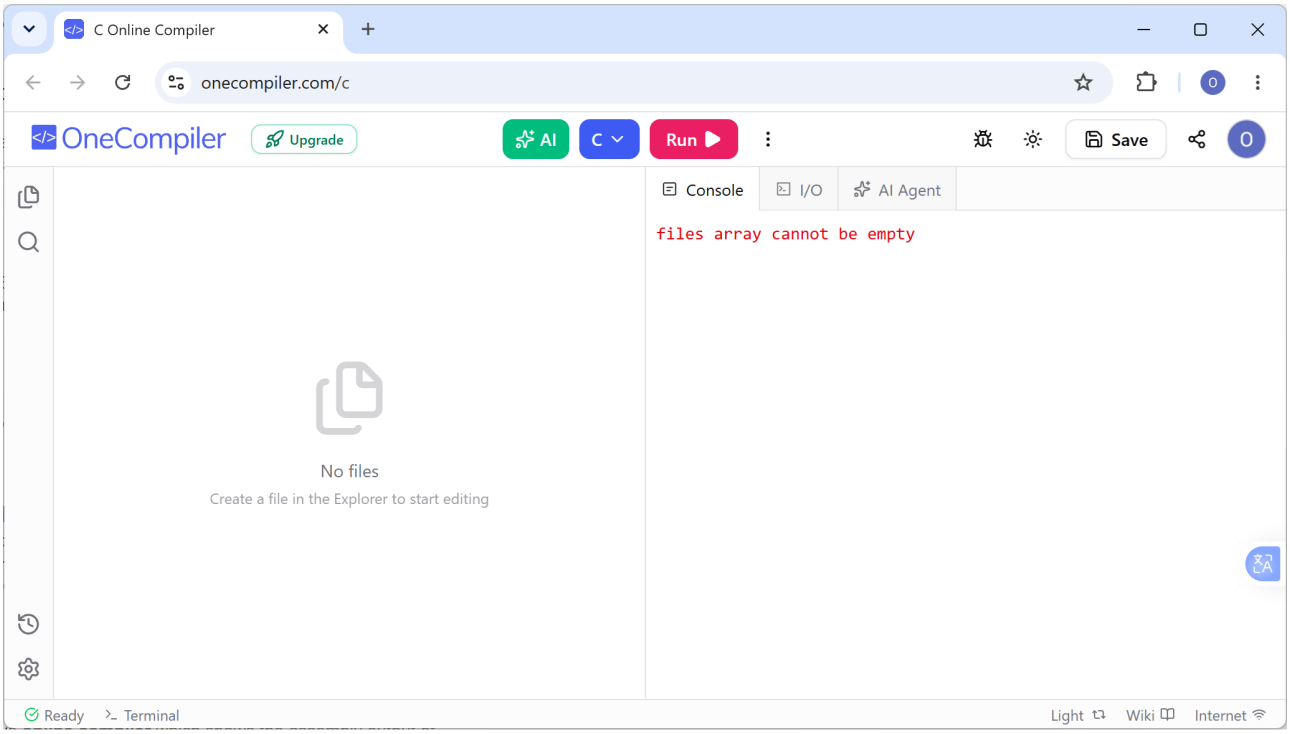


Рисунок 1. Домашняя страница онлайн компилятора

2. Убеждаемся, что выбран язык C. При необходимости выбираем его из выпадающего списка:

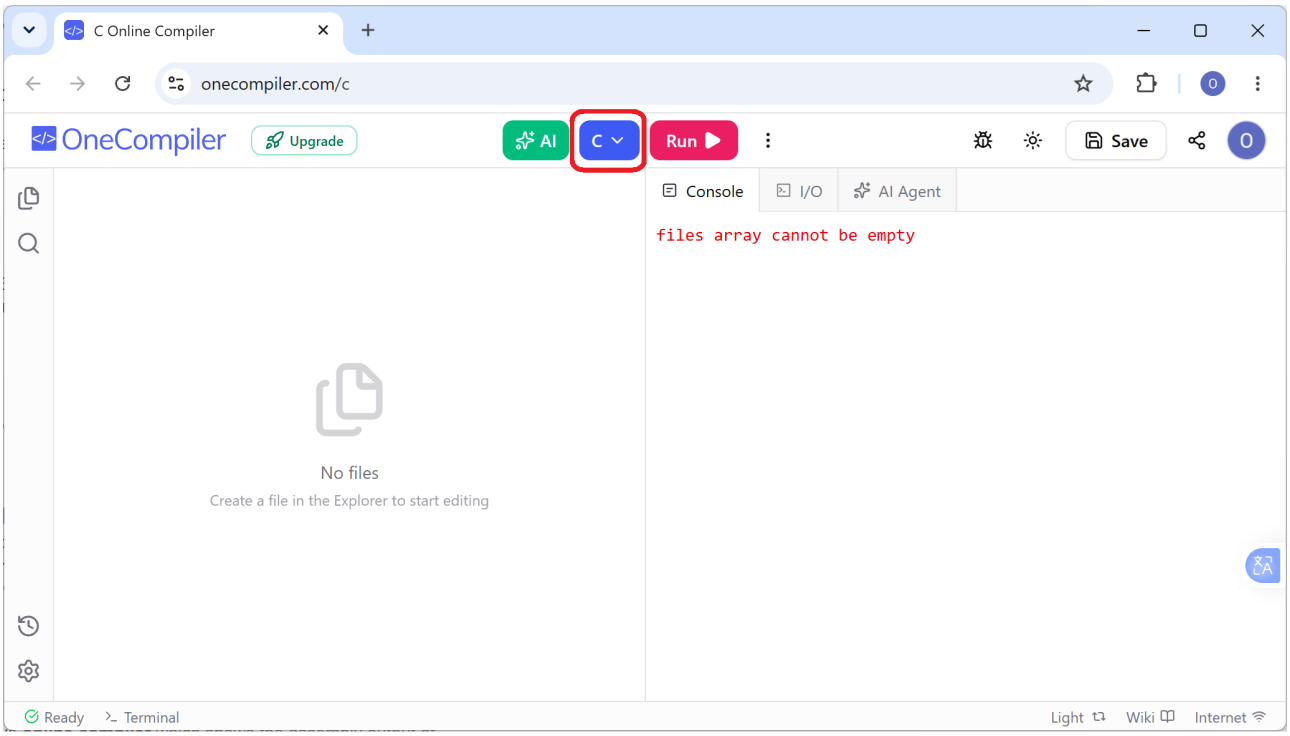


Рисунок 2. Выбор языка

3. Вводим текст нашей программы в окно исходного файла, предварительно создав его соответствующей кнопкой:

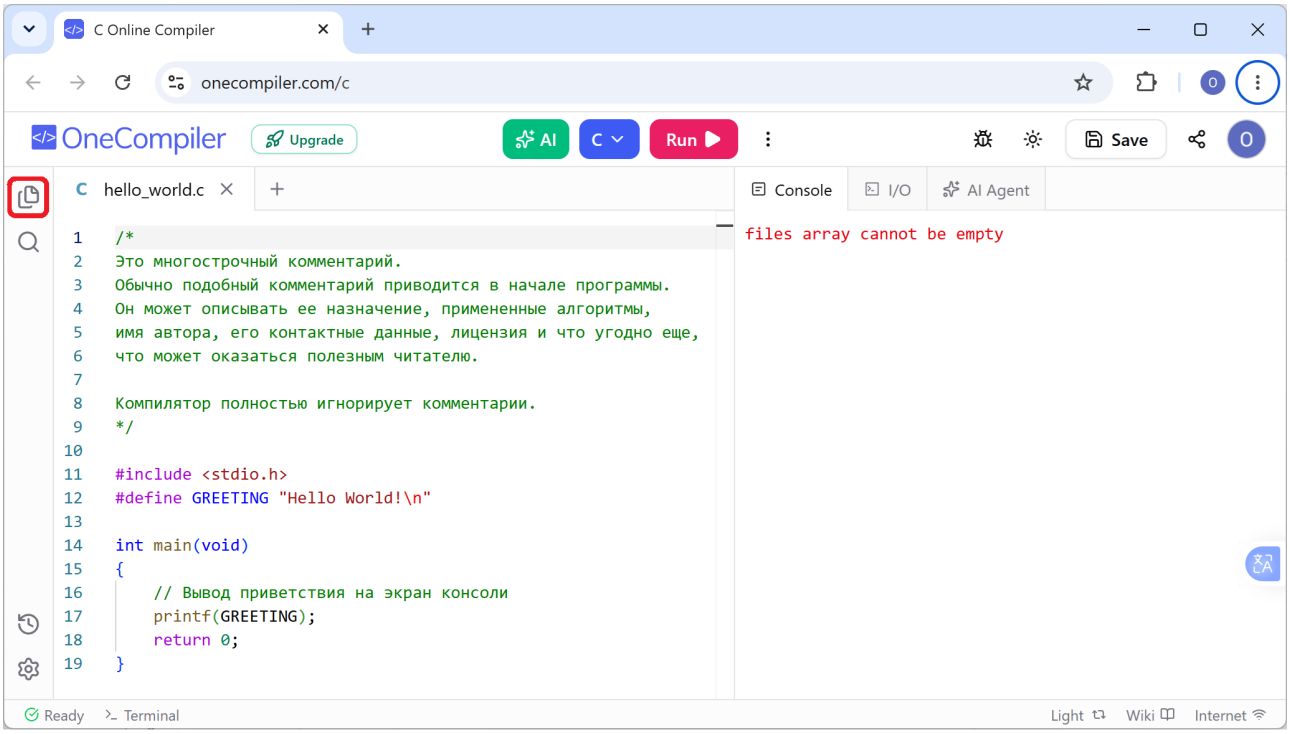


Рисунок 3. Ввод программы

4. Запускаем компиляцию и выполнение программы нажатием кнопки **Run**:

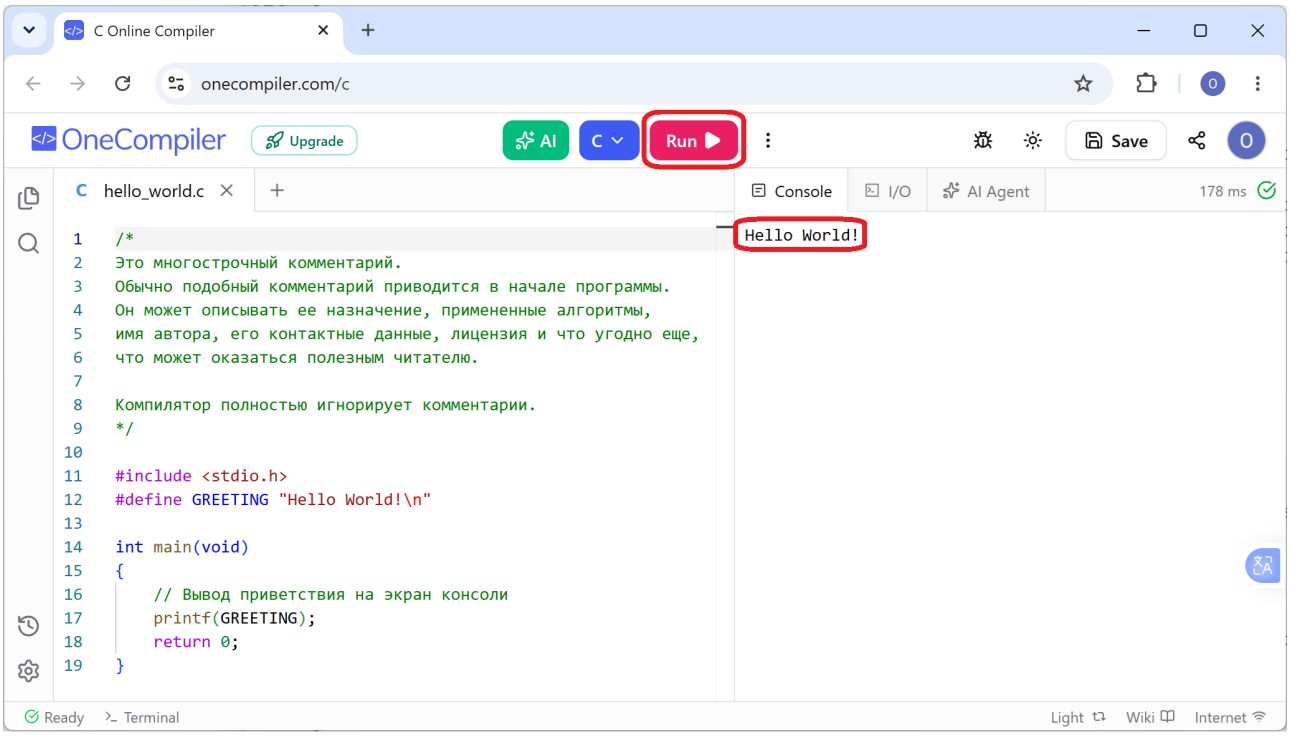


Рисунок 4. Компиляция и выполнение программы

В правой части окна на закладке `Console` видим выведенный программой текст.

В случае, если текст программы был введен с ошибкой, вместо вывода получим сообщение об ошибке. **OneCompiler** дает довольно невразумительную диагностику; однако если воспользоваться услугам встроенного AI-агента, мы видим гораздо более осмысленное сообщение:

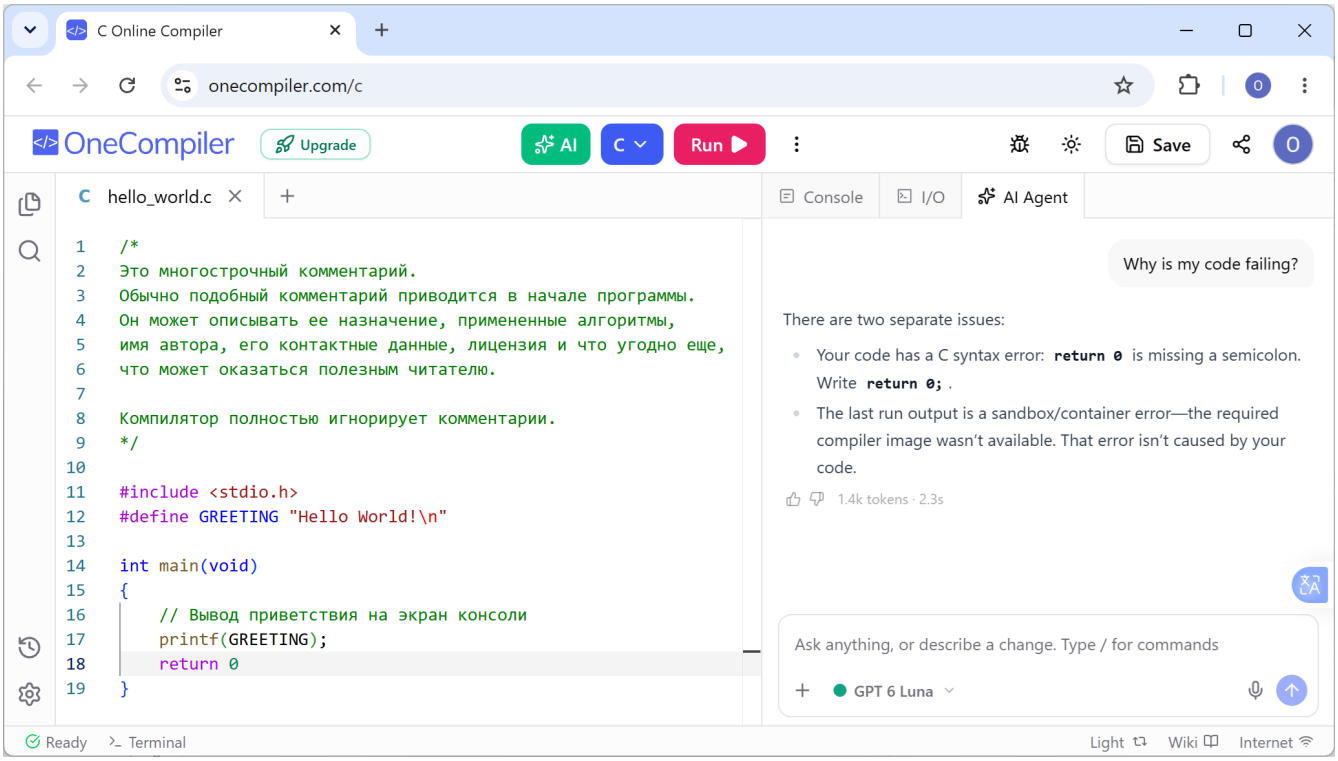


Рисунок 5. Сообщение об ошибке

Преимущества онлайн компиляторов

- **Не требуют установки** — достаточно воспользоваться браузером
- **Кроссплатформенность** — доступен на Windows, macOS, Linux, мобильных устройствах
- **Быстрый старт** — можно сразу писать и запускать код
- **Доступ к разным компиляторам** — GCC, Clang, MSVC без установки

Недостатки онлайн компиляторов

- **Ограничения по времени выполнения** — если программа не завершается за несколько секунд, ее выполнение прекращается
- **Ограничения по памяти** — обычно не более 256-512 МБ
- **Нет доступа к файловой системе** — нельзя работать с файлами на диске
- **Ограниченная поддержка библиотек** — не все библиотеки доступны
- **Зависимость от интернета** — требуется стабильное подключение
- **Проблемы с конфиденциальностью** — код загружается на сторонний сервер и может быть перехвачен
- **Не подходит для больших проектов** — только для простых программ
- **Ограничения доступа** — в условиях нарастающих санкций все больше облачных ресурсов становятся недоступными для пользователей из России; возможно, что для доступа к выбранному вами онлайн компилятору вскоре потребуется установка соединения VPN.

Когда имеет смысл использовать онлайн компиляторы

- Быстрая проверка синтаксиса или алгоритма
- Обучение основам языка C
- Демонстрация кода коллегам или на форуме
- Работа на компьютере без возможности установки ПО
- Сравнение вывода разных компиляторов

4.6.2. Использование командной строки

Мы уже выполнили компиляцию нашей программы из командной строки в разделе Фазы компиляции, поэтому здесь я не стану повторяться. Лишь напомню вратце основные моменты:

```
# Компиляция
gcc hello_world.c -o hello_world

# Запуск
./hello_world

# Компиляция с предупреждениями и оптимизацией
gcc -Wall -Wextra -O2 hello.c -o hello

# Компиляция с отладочной информацией
gcc -g hello.c -o hello_debug

# Компиляция с использованием стандарта C11
gcc -std=c11 -pedantic hello.c -o hello
```

Преимущества командной строки

- **Полный контроль** — доступ ко всем флагам и опциям компилятора
- **Автоматизация** — легко интегрируется в скрипты и системы сборки
- **Минимальные ресурсы** — не требует графического интерфейса
- **Быстрая компиляция** — нет накладных расходов IDE
- **Работа на серверах** — идеально для headless систем
- **Наглядность процесса** — вы видите все этапы компиляции

Недостатки командной строки

- **Требует знания команд** — нужно помнить флаги и синтаксис
- **Более сложная отладка** — требуется отдельный отладчик (gdb)
- **Неудобно для больших проектов** — неоходимо ручное управление зависимостями, чтобы не перекомпилировать каждый раз весь проект

Когда целесообразно использовать командную строку

- Работа на серверах и встраиваемых системах
- Автоматизация сборки через простые скрипты
- Небольшие проекты и утилиты
- Когда нужен полный контроль над процессом компиляции
- Изучение деталей процесса компиляции

4.6.3. Использование IDE

IDE (Integrated Development Environment) — это интегрированная среда разработки, которая объединяет редактор кода, компилятор, отладчик и другие инструменты в одном приложении. Разработчик может выполнять практически все необходимые для ведения проекта действия. не покидая IDE.

Некоторые популярные IDE для C

Code::Blocks

— свободная, легкая, кроссплатформенная IDE с поддержкой GCC

CLion (JetBrains)

мощная IDE с продвинутым анализом кода

Eclipse CDT

— открытая IDE с поддержкой C/C++

Visual Studio (MS Windows)

— мощная IDE для разработки в среде MS Windows

Xcode (macOS)

— официальная IDE для платформ Apple

Dev-C++

— легкая IDE для начинающих (на базе MinGW)

Пример работы с Code::Blocks

Code::Blocks — это свободная IDE с открытым исходным кодом, которая хорошо подойдет для изучения языка C. Она работает на Windows, Linux и macOS, имеет встроенную поддержку GCC и простой, интуитивно понятный интерфейс.

Я выполнил этот пример в среде MS Windows. Если вы работаете в Linux, возможны несущественные отличия того, что вы eblbnt на своем экране, от следующих иллюстраций.

Установка и настройка

1. Скачайте установщик с официального сайта https://www.codeblocks.org
2. **Важно**: для Windows выбирайте версию `codeblocks-XX.XXmingw-setup.exe` — она включает

компилятор MinGW GCC, что избавляет от необходимости устанавливать его отдельно

3. Запустите установщик и следуйте инструкциям
4. При первом запуске Code::Blocks предложит выбрать компилятор по умолчанию — оставьте GNU GCC Compiler

Создание проекта

1. Запустите Code::Blocks

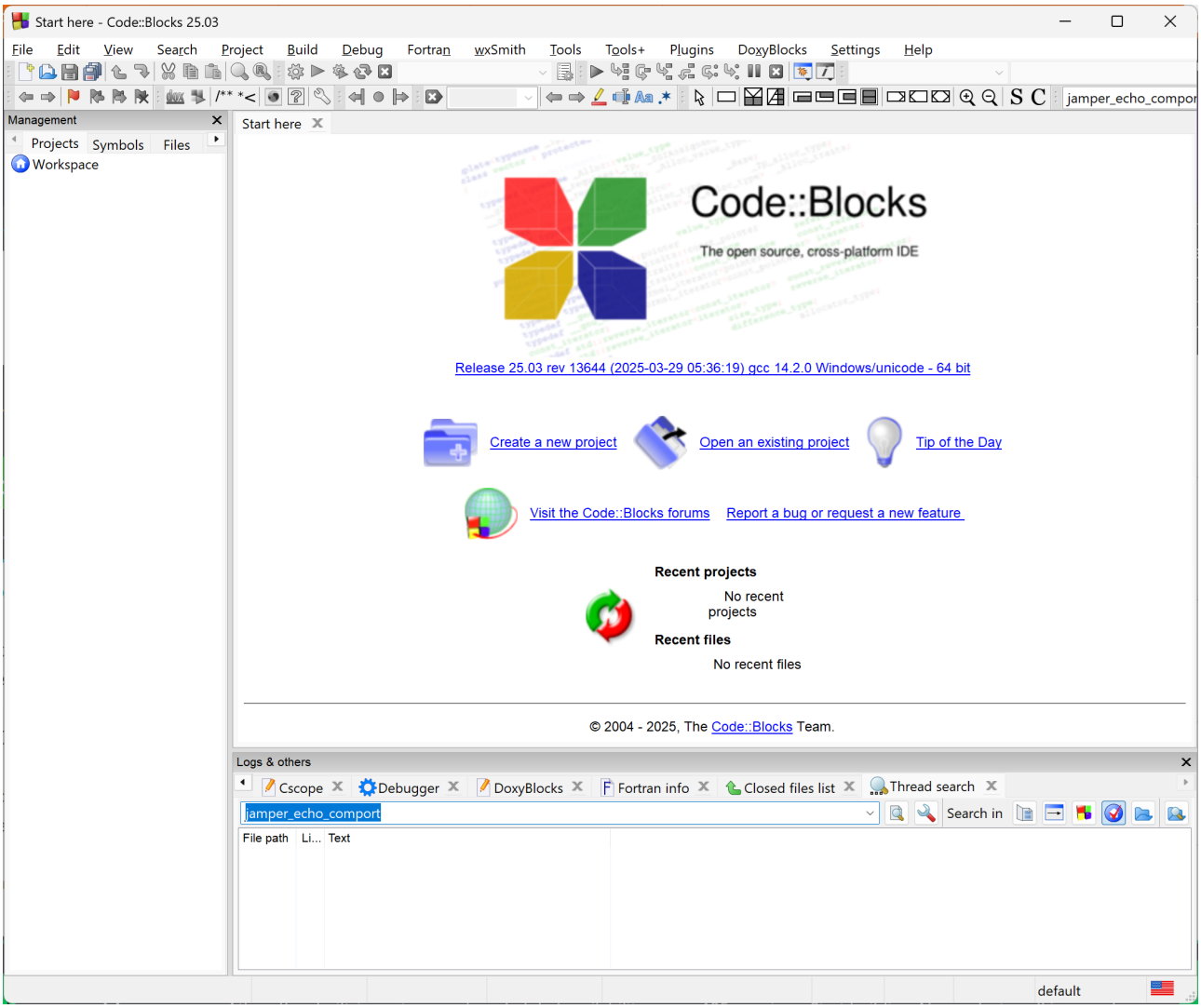


Рисунок 6. IDE Code::Blocks

2. Выберите `File → New → Project…`
3. В открывшемся окне выберите `Console application` и нажмите `Go`

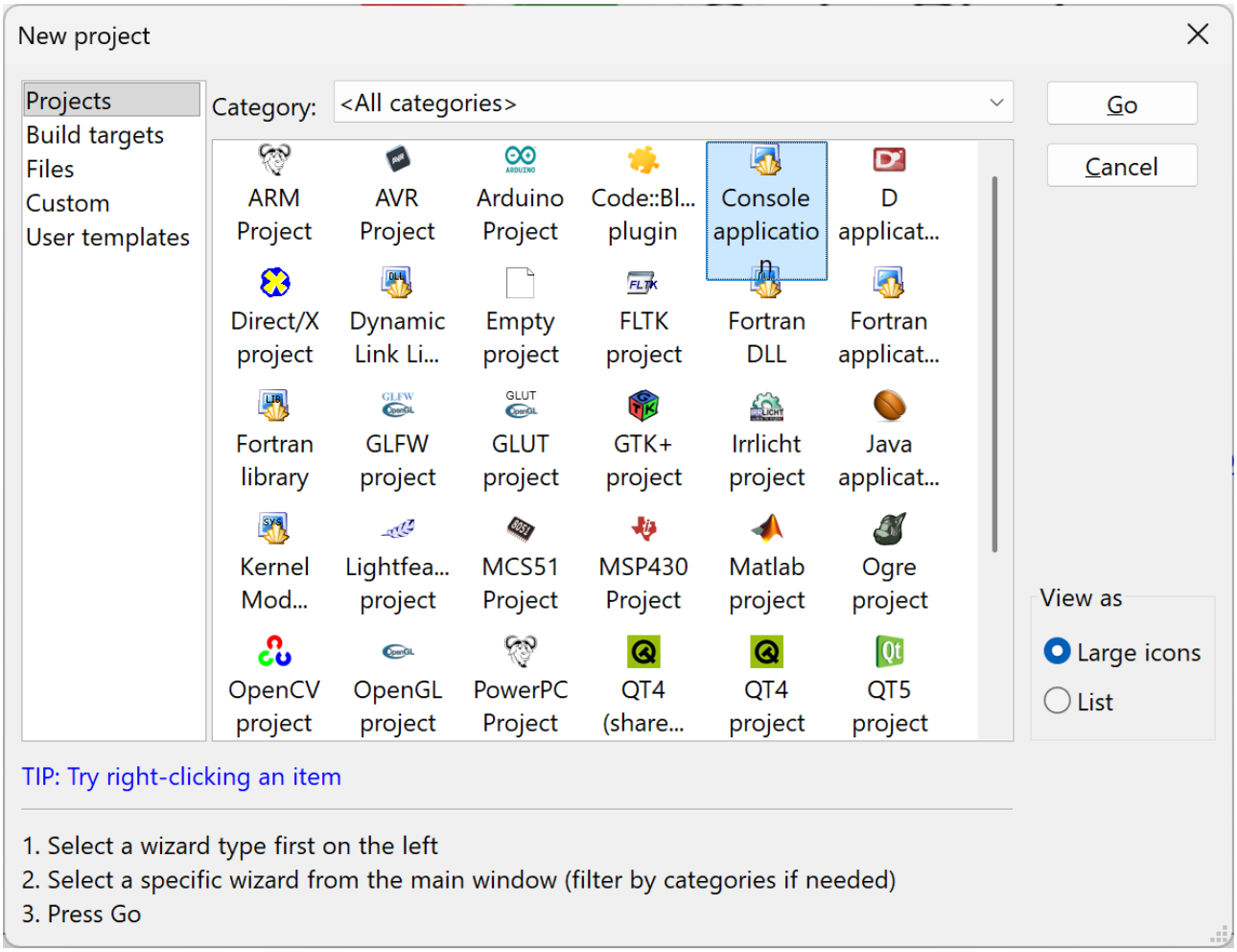


Рисунок 7. Выбор типа проекта

4. Выберите язык `C`, нажмите `Next`

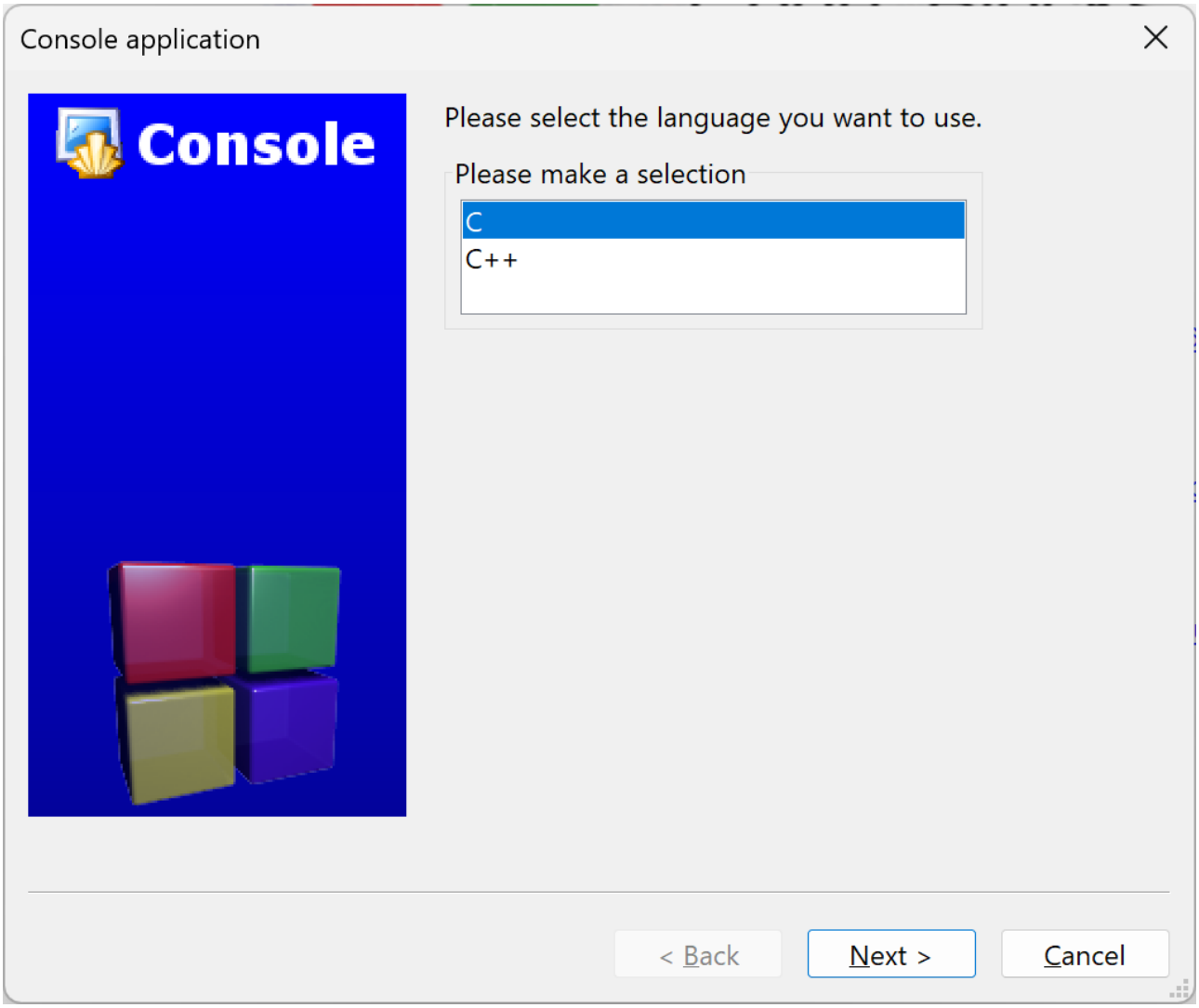


Рисунок 8. Выбор языка проекта

5. Укажите название проекта (например, `hello_world`) и директорию для сохранения, , нажмите `Next`

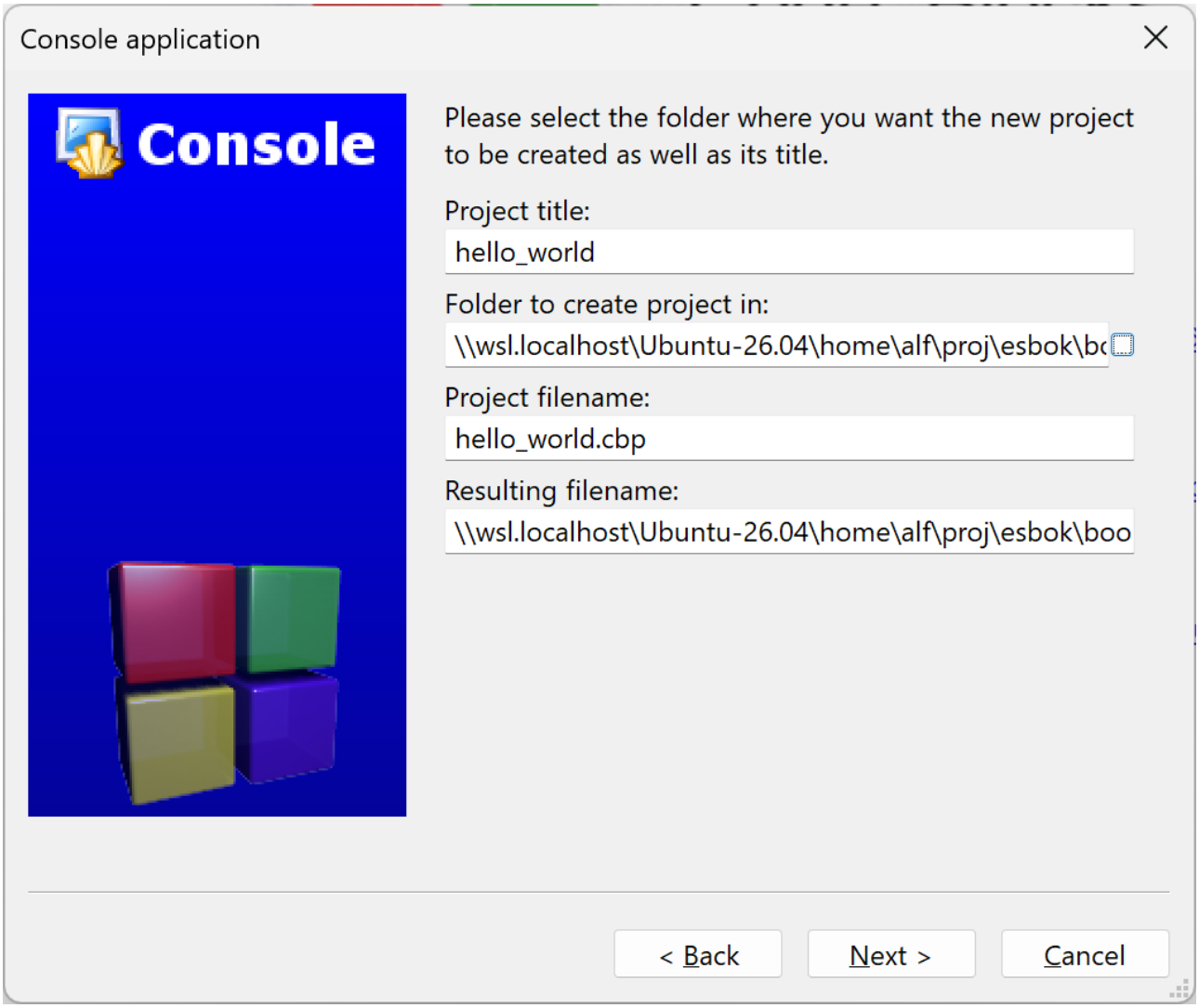


Рисунок 9. Название и расположение проекта

6. Выберите компилятор `Cygwin GCC` и нажмите `Finish`

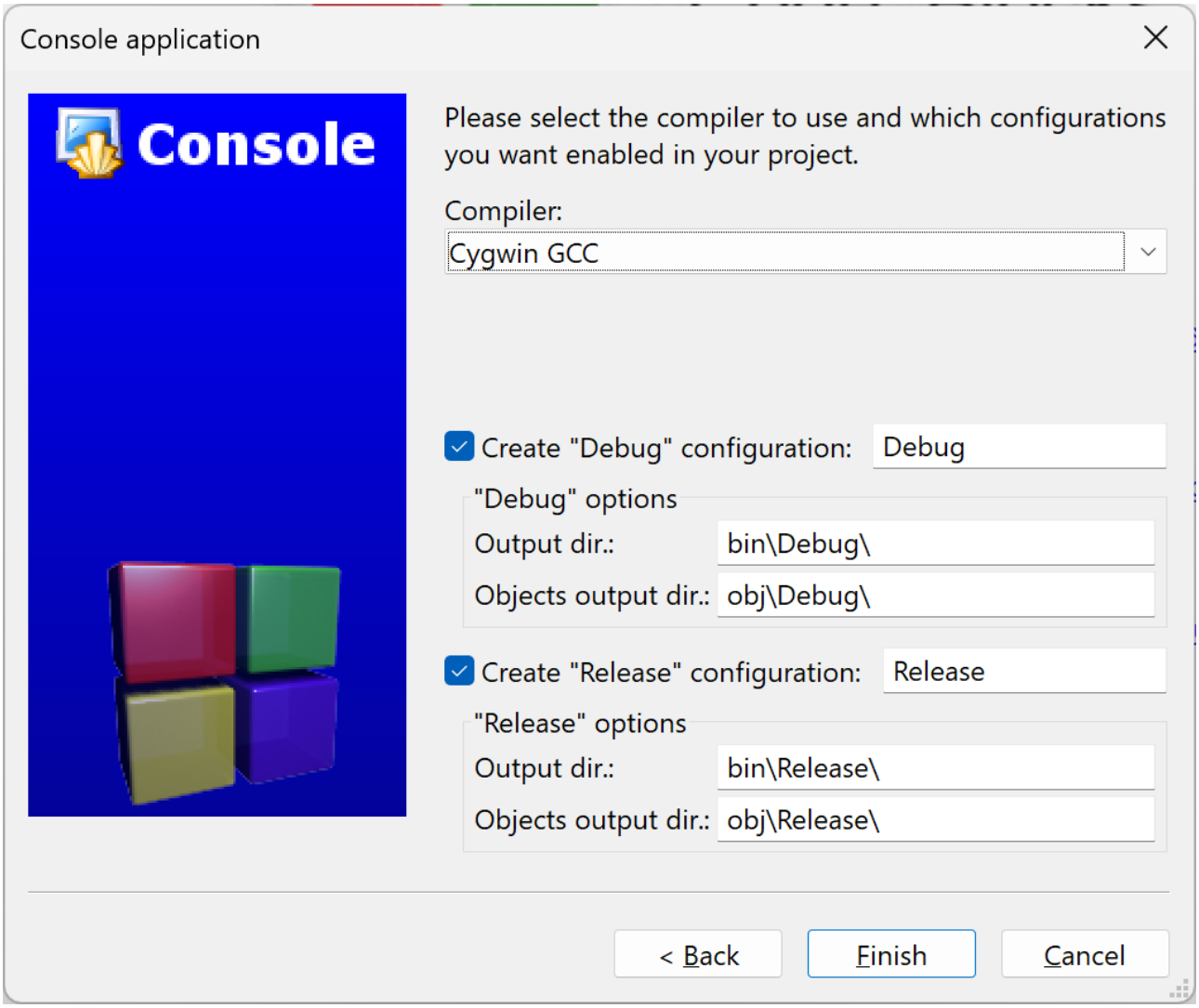


Рисунок 10. Выбор компилятора

Code::Blocks автоматически создаст структуру проекта с файлом `main.c`, содержащим шаблон программы "Hello, World!".

Вы можете заменить текст программы своим или оставить как есть.

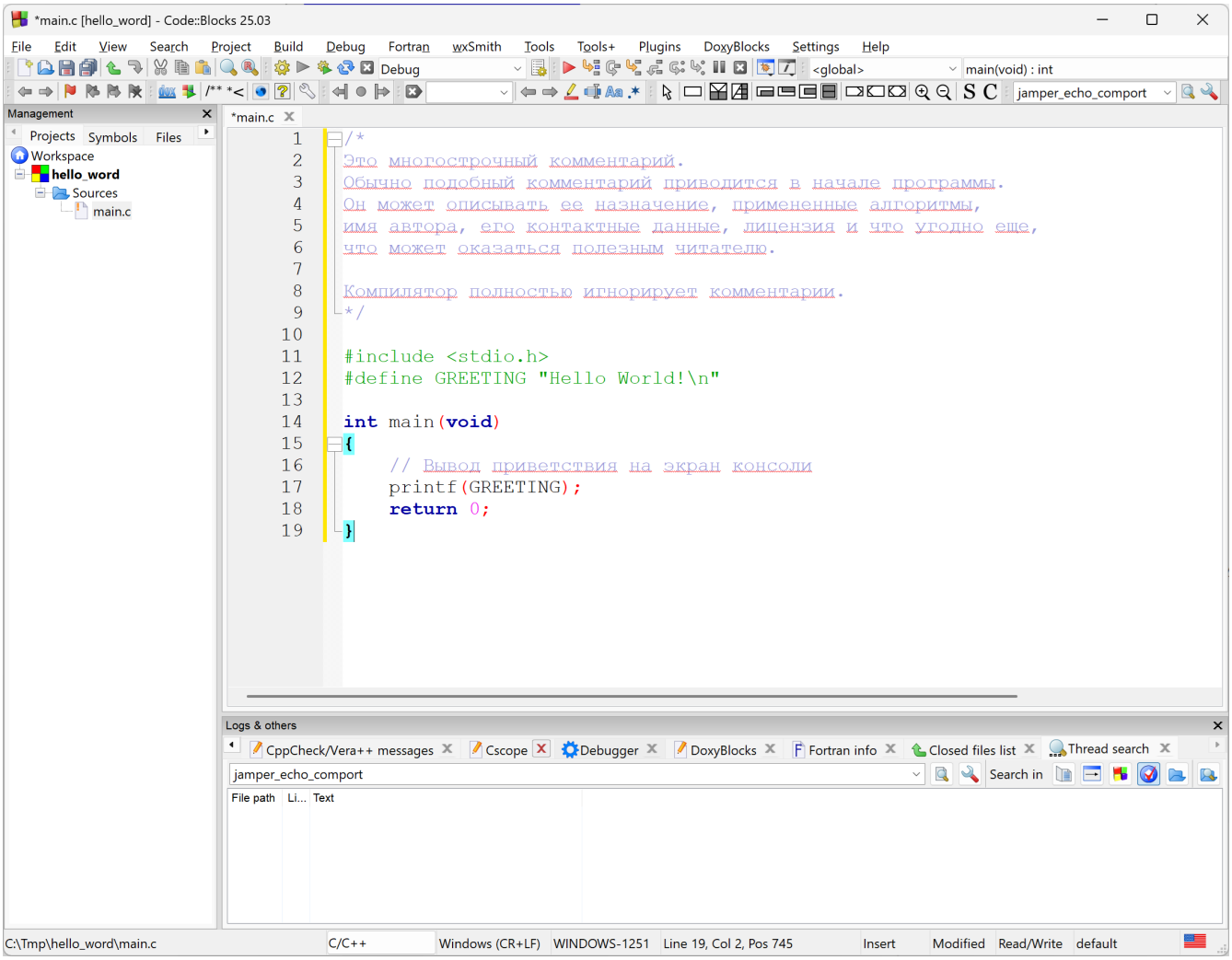


Рисунок 11. IDE с исходной программой

Компиляция и запуск

Code::Blocks предоставляет несколько способов компиляции и запуска:

- **Build** (`Ctrl+F9` или `Build → Build`) — только компиляция без запуска
- **Run** (`Ctrl+F10` или `Build → Run`) — запуск ранее скомпилированной программы
- **Build and Run** (`F9` или `Build → Build and Run`) — компиляция и запуск (самый удобный вариант)

После нажатия `F9` откроется консольное окно с результатом:

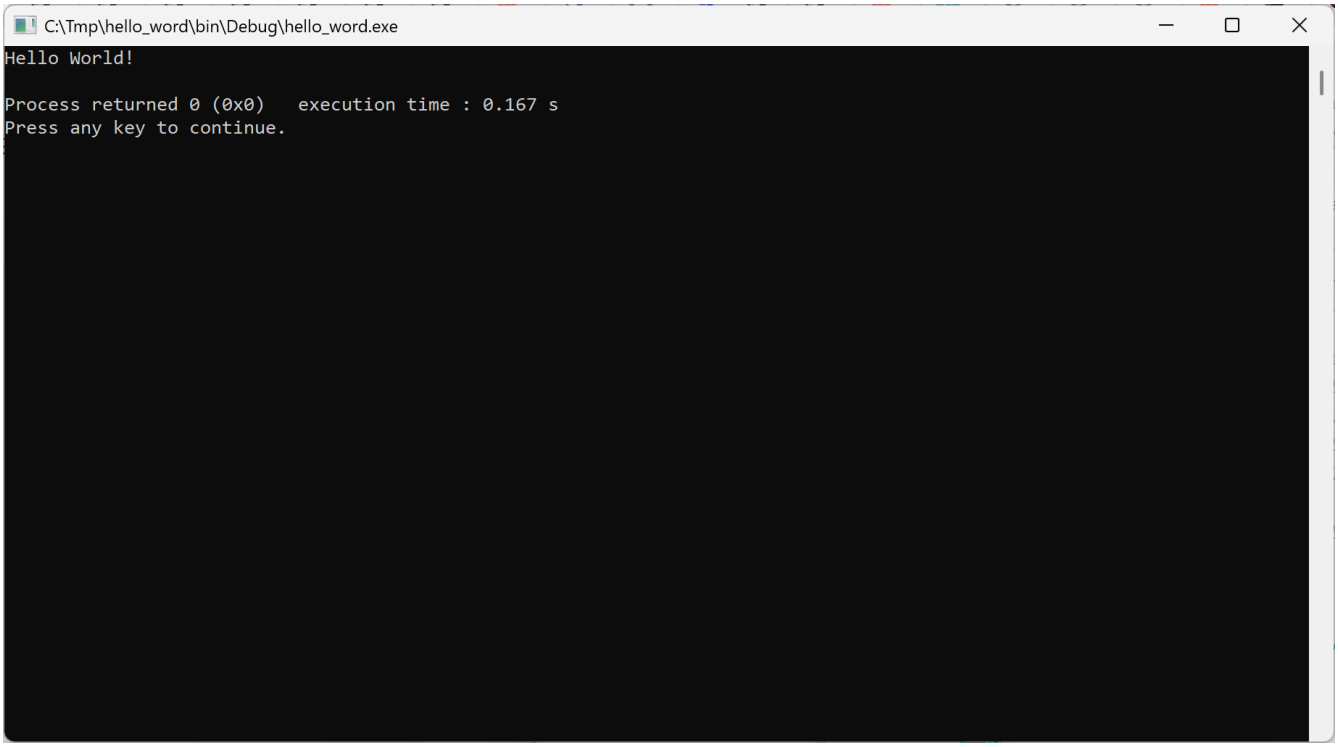


Рисунок 12. Вывод программы

Сравнение преимуществ и недостатков использования IDE для разработки на C

Интегрированная среда разработки (IDE) — это мощный инструмент, который значительно упрощает процесс разработки программ на C. Однако, как и любой инструмент, IDE имеет свои преимущества и недостатки. Рассмотрим их подробно и сравним с альтернативными подходами.

Преимущества использования IDE

- IDE интегрирует все необходимые инструменты в едином приложении: многооконный редактор кода с развитой навигацией по проекту, компилятор, отладчик с визуальным интерфейсом, систему сборки, управление проектами, управление репозиторием, и т. п., поэтому разработчику не нужно постоянно переключаться между разными окнами и утилитами.
- Многие IDE предлагают средства рефакторинга кода.

Недостатки использования IDE

- Высокие требования к ресурсам компьютера разработки.
- Сложность настройки: зачастую IDE сразу "из коробки" выполняет ожидаемые пользователем действия; однако в случаях, когда поведение по умолчанию требуется изменить, это не всегда удается быстро сделать, поскольку многие настройки глубоко упрятаны разработчиками IDE.
- "Магия" работы: сборка сложного проекта производится одним нажатием кнопки, детали скрыты от пользователя; поэтому, когда что-то происходит не по плану, трудно бывает понять, что именно и как это изменить.

Какую IDE выбрать для эмбеддинга

Рабочая среда для эмбеддинга имеет свою специфику по сравнению с разработкой ПО общего назначения. Например, многие IDE имеют встроенные средства для прошивки исполнимого кода в память программ МК или для интерактивной отладки. Полезной может оказаться функция конфигурирования и инициализации аппаратных ресурсов.

Имеет смысл потратить некоторое время на выбор наиболее подходящего для вас инструмента. Например, для работы с МК семейства STM32 я использую бесплатный инструмент от производителя под названием **STM32CubeIDE**[4], построенный на базе проекта с открытым кодом **Eclipse** (в настоящий момент этот продукт, как и многие другие, недоступен для прямого скачивания из России; чтобы установить его, потребуется совершить несложные действия, хорошо знакомые пользователям Internet с хотя бы небольшим опытом). У этого инструмента есть как сторонники, так и противники, поэтому рекомендую не полагаться слепо на чужое мнение, а составить собственное на основе личного опыта.

4.6.4. Инструменты сборки проектов

Когда реальный проект перерастает масштабы учебной задачи и начинает разрастаться как по объему кода, так и по сложности, со временем выясняется, что его сборка начинает оттягивать на себя до неприличия большую долю от общей трудоемкости. Поскольку это хоть и не слишком простой, но все же рутинный процесс по сравнению с разработкой кода, появляется желание разгрузить команду разработчиков от этой обузы. К счастью, для этого есть специальные инструменты, которые мы вкратце рассмотрим в этом разделе.

При разработке программ на C, состоящих из множества файлов, ручная компиляция каждой единицы трансляции становится непрактичной. **Системы сборки** — это инструменты, которые автоматизируют процесс преобразования исходного кода в исполняемый файл, управляя зависимостями, флагами компиляции и порядком выполнения команд.

Системы сборки

Предположим, что у нас есть проект из нескольких файлов. Для его сборки нужно предпринять следующий набор действий:

```
gcc -Wall -g -c main.c -o main.o
gcc -Wall -g -c math_ops.c -o math_ops.o
gcc -Wall -g -c utils.c -o utils.o
gcc -Wall -g -c io.c -o io.o
gcc -Wall -g main.o math_ops.o utils.o io.o -lm -o program
```

Повторять эту последовательность вручную каждый раз, когда возникает необходимость пересборки проекта, довольно утомительно и к тому же чувствительно к опечаткам. Можно объединить эти команды в один скрипт (к примеру, для запуска в `bash` для Linux или `PowerShell` для MS Windows). Однако это решает проблему лишь частично.

Предположим, мы изменили один из файлов (допустим, это будет `utils.c`. чтобы собрать

проект с учетом этих изменений, нам придется запустить скрипт сборки. Однако есть ли реальная необходимость выполнять **все** команды скрипта? Ведь остальные файлы не менялись; если в рабочей директории проекта остались объектные файлы после предыдущей их компиляции, они все еще актуальны, и ими можно воспользоваться при линковке. Фактически только измененный(е) исходный(е) файл(ы) действительно нуждаются в перекомпиляции.

> Почему мы так стремимся избегать лишней работы? Ведь компиляция происходит почти мгновенно.
>
> Вы наглядно увидите это, когда от "Hello World" перейдете к большим проектам в десятки и сотни тысяч строк. Там полная сборка может занять весьма ощутимое время даже на очень мощном компьютере.
>
> 
>
> В реальности ситуация немного сложнее. Если мы изменили не *исходный* файл (*.c), а *заголовочный* (*.h) (не будем сейчас устраивать дискуссию на тему, что заголовочный файл можно тоже отнести к исходным), то следует перекомпилировать **все** исходные файлы, которые включают этот заголовочный файл посредством директивы `#include`. Впрочем, это не меняет принципиально суть вышесказанного: при изменениях достаточно перекомпилировать только те исходные файлы, которые зависят от изменений.

Простой скрипт не может решить эту проблему. На самом деле здесь есть еще одна проблема: объектные файлы, как мы уже знаем, являются промежуточными артефактами компиляции и могут быть получены в любой момент, поэтому,в отличие от исходного кода, их не стараются бережно хранить. вполне может оказаться, что во время очередной уборки в рабочем пространстве все или часть объектных файлов были удалены. Это значит, что, хотя их исходный код не менялся, их тоже придется перекомпилировать.

Если делать это вручную, придется завести общий скрипт сборки и комментировать строки, которые в данной сборке не участвуют. Это трудоемко и к тому же чревато ошибками: если какая-то нужная строка окажется не выполнена, мы проведем компоновку с устаревшим объектным кодом и будем долго искать ошибку (или, что еще хуже, ее найдет пользователь нашей программы).

Вывод: нужен инструмент, который сам автоматически проведет сборку. Разумеется, он должен быть достаточно умным, чтобы не делать лишнюю работу и в то же время не пропустить нужную. К счастью, такие инструменты есть, и они называются **системами сборки**.

Системы сборки можно разделить на **утилиты сборки** (`Make`, `Ninja`), которые выполняют собственно сборку в соответствии с заданными правилами и зависимостями, и **генераторы скриптов сборки** (`CMake`, `Meson`, `Autotools`), которые сами проект не собирают, а создают управляющие скрипты для утилит сборки. Есть более сложные инструменты для очень больших проектов, но их мы рассматривать в этом учебном пособии не будем.

Make — классическая утилита сборки

Make — это старейшая и наиболее распространенная утилита сборки. Она появилась в 1976 году в Bell Labs и до сих пор остается стандартом де-факто.

Принцип работы

Утилита `Make` непроста, и ее описанию посвящены целые тома, поэтому небольшой раздел учебника по языку C — неподходящий источник, после прочтения которого можно стать экспертом по этой утилите. Здесь я привожу лишь самый краткий обзор, основная цель которого — дать читателю понять, нужно ли ему копать глубже в этом направлении.

Make читает файл `Makefile`, который содержит **правила** сборки в формате:

```
цель: зависимости
    команды для сборки цели
```

Make сравнивает временные метки *целей* и *зависимостей*. Если зависимость новее цели или цели не существует, выполняются *команды*, в результате выполнения которых должна быть построена цель.

Пример Makefile:

```
# Переменные
CC = gcc
CFLAGS = -Wall -Wextra -g -std=c11
LDFLAGS = -lm
TARGET = program

# Списки файлов
SRCS = main.c math_ops.c utils.c io.c
OBJS = $(SRCS:.c=.o)
DEPS = $(SRCS:.c=.d)

# Главная цель (по умолчанию)
all: $(TARGET)

# Компоновка
$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^ $(LDFLAGS)

# Компиляция с автоматическим созданием зависимостей
%.o: %.c
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@

# Подключение файлов зависимостей
-include $(DEPS)

# Очистка
clean:
```

```
    rm -f $(OBJS) $(DEPS) $(TARGET)

# Перекомпиляция
rebuild: clean all

# Установка
install: $(TARGET)
    install -m 755 $(TARGET) /usr/local/bin/

# Объявление нефайловых целей
.PHONY: all clean rebuild install
```

Особенности синтаксиса:

Табуляция обязательна

— команды должны начинаться с символа табуляции (`\t`) **(не пробелов!!!)***

Переменные

— `$(VAR)` или `${VAR}`

Автоматические переменные

- `$@` — имя цели
- `$<` — первая зависимость
- `$^` — все зависимости
- `$?` — только более новые зависимости
- **Функции** — `$(wildcard *.c)`, `$(patsubst %.c,%.o,$(SRCS))`
- **Условные конструкции** — `ifeq`, `ifdef`, `ifneq`

Варианты Make

Существует несколько реализаций Make:

- **GNU Make** — самая распространенная, стандарт для Linux
- **BSD Make** — используется в FreeBSD, OpenBSD
- **NMake** — версия для Windows от Microsoft
- **Jom** — многопоточная версия NMake от Qt

Преимущества Make

- **Универсальность** — может применяться фактически для любых проектов (не только для сборки программ C и вообще не только в области программирования)
- **Простота для базовых задач** — короткие Makefile легко читаются
- **Гибкость** — можно описать любые правила сборки
- **Богатая экосистема** — тысячи примеров и туториалов
- **Интеграция** — работает с любыми компиляторами и инструментами

Недостатки Make

- **Устаревший излишне сложный синтаксис** — много архаичных конструкций
- **Проблема с табуляцией** — частая ошибка новичков
- **Кроссплатформенность** — разные версии Make несовместимы
- **Сложность для больших проектов** — Makefile становятся нечитаемыми
- **Нет встроенной поддержки** — зависимости, тесты, пакеты нужно описывать вручную

Когда целесообразно использовать Make

- Небольшие проекты
- Проекты, которые должны собираться "из коробки" на любой Unix-системе
- Автоматизация простых задач
- Legacy-проекты

CMake — кроссплатформенный генератор

CMake (Cross Platform Make) — это не система сборки, а **генератор** скриптов для систем сборки. CMake читает файл `CMakeLists.txt` и генерирует Makefile (для `Make`), проекты Visual Studio (для Windows), проекты Xcode (для macOS)? файлы для Ninja и т. п.

Принцип работы

```
CMakeLists.txt → cmake → Makefile → make → программа
```

Пример CMakeLists.txt

```
# Минимальная версия CMake
cmake_minimum_required(VERSION 3.16)

# Объявление проекта
project(HelloWorld
    VERSION 1.0.0
    DESCRIPTION "Hello World example"
    LANGUAGES C
)

# Стандарт C
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)

# Поиск зависимостей
find_package(M REQUIRED)

# Сбор исходных файлов
file(GLOB SOURCES "src/*.c")
```

```
file(GLOB HEADERS "include/*.h")

# Создание исполняемого файла
add_executable(hello ${SOURCES} ${HEADERS})

# Настройка целевого файла
target_include_directories(hello PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_link_libraries(hello PRIVATE m)

target_compile_options(hello PRIVATE
    -Wall -Wextra -Wpedantic
)

# Установка
include(GNUInstallDirs)
install(TARGETS hello
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)

# Тестирование
enable_testing()
add_test(NAME hello_test COMMAND hello)
```

Возможности CMake

Начиная с версии 3.0 (и особенно 3.15+), CMake предлагает современный подход:

Целевой подход (target-based)
: — все настройки привязаны к целям

Модульная структура
: — `add_subdirectory()` для подпроектов

FetchContent
: — загрузка зависимостей во время конфигурации

Package management
: — интеграция с Conan и vcpkg

Преимущества CMake

Кроссплатформенность
: — один `CMakeLists.txt` для всех платформ

Наглядный синтаксис
: — понятнее, чем Makefile

Большое сообщество

— стандарт для open-source проектов

Интеграция с IDE

— CLion, VS Code, Qt Creator используют CMake "из коробки"

Управление зависимостями

— `find_package()` для системных библиотек

Тестирование

— встроенный CTest

Упаковка

— встроенный CPack для создания установщиков

Недостатки CMake

Двухэтапный процесс

— сначала конфигурация, потом сборка

Все еще сложный синтаксис

— много устаревших команд, путаница между старым и новым стилем

Медленная конфигурация

— большие проекты конфигурируются долго

Крутая кривая обучения

— перед использованием нужно изучить много команд и концепций

Проблемы с версиями

— плохая совместимость старых версий с современными (например, код для CMake 3.10 не работает на 3.5)

Избыточность

— для маленьких проектов слишком сложен

Когда целесообразно использовать CMake

- Кроссплатформенные проекты
- Проекты с множеством зависимостей
- Библиотеки для распространения
- Современные open-source проекты
- Когда нужна поддержка разных IDE

Другие инструменты сборки проектов

Рассмотренными инструментами не исчерпывается список доступных инструментов

сборки проектов. вам могут встретиться также **Autotools** (набор инструментов, который долгое время был стандартом для Unix-проектов), **Bazel**, **Conan**, **vcpkg** и многие другие.



За многие годы работы над программными продуктами я обнаружил неутешительную истину: если для решения некоторой задачи используется множество инструментов, это значит, что среди них нет ни одного идеального, способного заменить все остальные. Это в полной мере относится и к инструментам сборки проектов. У каждого есть как достоинства, так и недостатки, которые нужно хорошо понять для обоснованного выбора.

Сравнительная таблица систем сборки

Система	Сложность	Скорость	Кроссплатформенность	Сообщество	Применение
Make	Средняя	Средняя	Unix-подобные	Огромное	Небольшие Unix-проекты
CMake	Высокая	Средняя (конфигурация медленная)	Полная	Огромное	Кроссплатформенные проекты
Ninja	Низкая (для генераторов)	Очень высокая	Полная	Среднее	Бэкенд для CMake/Meson
Meson	Низкая	Высокая	Полная	Среднее	Современные проекты, GNOME
Autotools	Очень высокая	Низкая (конфигурация)	Unix-подобные	Большое (legacy)	Legacy Unix-проекты
Bazel	Очень высокая	Очень высокая (инкрементально)	Полная	Растущее	Огромные monorepo

Резюме

Выбор системы сборки зависит от размера проекта, требований к кроссплатформенности и опыта команды:

- **Make** — для небольших Unix-проектов и автоматизации
- **CMake** — стандарт для кроссплатформенных проектов и open-source
- **Ninja** — для максимальной скорости сборки (обычно как бэкенд)
- **Meson** — современная альтернатива с простым синтаксисом
- **Autotools** — для legacy Unix-проектов
- **Bazel** — для огромных monorepo

Начинающим рекомендуется начать с Make для понимания базовых концепций, затем

перейти к CMake для работы с реальными проектами. Профессиональные разработчики обычно владеют несколькими системами сборки и выбирают инструмент в зависимости от конкретной задачи.

[1] С использованием соответствующих флагов. Кроме того, в GCC имеется множество полезных расширений, многие из которых впоследствии входят в стандарт языка.

[2] Начиная с GCC 12; в GCC 15 поддержка C23 практически полная.

[3] Возможность генерации кода для PC весьма важна при тестировании кода, поскольккку позволяет выполнять множество тестов непосредственно на компьютере разработчика

[4] https://www.st.com/en/development-tools/stm32cubeide.html

Chapter 5. Препроцессор и макрокоманды

Мы уже очень бегло познакомились с ролью препроцессора в главе, посвященной компиляции (TBD). Пришла пора познакомиться с языком макросов детально, поскольку они являются весьма важной частью языка C в целом.

Прежде всего необходимо понять, что препроцессор — это отдельная программа, выполняющаяся **до** передачи исходного текста компилятору. Его задача — выполнить некоторые предварительные преобразования текста программы: подключить заголовочные файлы, раскрыть макроопределения, удалить комментарии и обработать директивы условной компиляции.



Препроцессор **не анализирует** синтаксис языка и **не выполняет** проверку типов. Он работает исключительно с текстом, производя в нём механические замены. Это свойство определяет основные опасности макрокоманд.

5.1. Директивы препроцессора

Все директивы препроцессора начинаются со знака `#`, который должен располагаться в начале строки (допустимы только пробельные символы перед ним). Директива **не требует** точки с запятой в конце и действует до конца строки.

Основные директивы:

Директива	Назначение
`#define`	Определение макроса
`#undef`	Отмена определения макроса
`#include`	Включение содержимого файла
`#ifdef`, `#ifndef`, `#if`, `#elif`, `#else`, `#endif`	Условная компиляция
`#error`, `#warning`	Выдача сообщения на этапе препроцессинга
`#pragma`	Передача указаний компилятору

5.2. Объектные макросы (макросы без аргументов)

Объектный макрос — это именованная текстовая последовательность, которая подставляется вместо имени макроса везде, где оно встречается в исходном тексте после точки определения.

Пример:

```
#define MAX_STUDENTS 30
#define PI 3.14159265358979
```

```
int group[MAX_STUDENTS];
double area = PI * r * r;
```

После работы препроцессора текст примет вид:

```
int group[30];
double area = 3.14159265358979 * r * r;
```

Объектные макросы традиционно используются для задания именованных констант. Это устраняет так называемые **магические числа** — безымянные числовые литералы, смысл которых неочевиден из контекста.

Макрос может быть определён и без тела — такое определение используется как **флаг** для **директив условной компиляции** (см. ниже):

```
#define DEBUG
```

Отменить определение макроса позволяет директива `#undef`:

```
#undef DEBUG
```

5.3. Функциональные макросы (макросы с аргументами)

Функциональный макрос принимает аргументы и подставляет их в тело определения. Синтаксически такой макрос напоминает вызов функции, однако механизм его работы принципиально иной: происходит не вычисление, а **текстовая подстановка**.

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int y = SQUARE(5);       /* y = ((5) * (5)) */
int m = MAX(a + 1, b);   /* m = ((a + 1) > (b) ? (a + 1) : (b)) */
```

Обратите внимание на скобки вокруг каждого аргумента и вокруг всего тела макроса. Это не излишество, а необходимость, связанная с приоритетом операций. Посмотрим, что будет без таких скобок:

```
#define BAD_SUM(a, b) a + b
int x = BAD_SUM(2, 3) * 4;
```

Раскрытие даст `2 + 3 * 4`, то есть `14`, а не ожидаемые `20`. Правильное определение:

```
#define GOOD_SUM(a, b) ((a) + (b))
```

Его раскрытие даст `2) + (3 * 4`, в соответствии с ожидаемым результатом.

5.4. Операторы # и

В теле функционального макроса могут использоваться два специальных оператора.

5.4.1. Оператор # ("стрингификация")

Оператор `#`, стоящий перед параметром макроса, превращает фактический аргумент в строковый литерал.

```
#define PRINT_VAR(var, fmt) \
    printf(#var " = %" #fmt "\n", var)

int count = 42;
PRINT_VAR(count, d);
/* Раскрывается в:
   printf("count" " = %" "d" "\n", count);
   что эквивалентно:
   printf("count = %d\n", count);
*/
```

5.4.2. Оператор ## (конкатенация)

Оператор `##` склеивает два лексических элемента в единый идентификатор.

```
#define MAKE_FUNC(name) void func_##name(void)

MAKE_FUNC(init);   /* Раскрывается в: void func_init(void) */
MAKE_FUNC(cleanup); /* Раскрывается в: void func_cleanup(void) */
```

Эти операторы позволяют писать макросы, генерирующие имена идентификаторов и строк, что невозможно реализовать средствами самого языка C.

5.5. Предопределённые макросы

Компилятор предоставляет ряд макросов, значения которых определяются автоматически. Наиболее употребительные из них:

Макрос	Значение
`__LINE__`	Номер текущей строки исходного файла
`__FILE__`	Имя текущего исходного файла (строковый литерал)

Макрос	Значение
`__DATE__`	Дата компиляции в формате "Mmm dd yyyy"
`__TIME__`	Время компиляции в формате "hh:mm:ss"
`__STDC__`	Определён, если компилятор соответствует стандарту (C)
`__STDC_VERSION__`	Номер версии стандарта C (например, `201112L` для C11)

Эти макросы широко применяются в отладочных сообщениях:

```
#define LOG_ERROR(msg) \
    fprintf(stderr, "Ошибка в %s:%d — %s\n", __FILE__, __LINE__, msg)
```

5.6. Условная компиляция

Директивы условной компиляции позволяют включать или исключать фрагменты текста программы в зависимости от выполнения некоторых условий. Это необходимо, в частности, для:

- адаптации программы к различным операционным системам и архитектурам;
- включения отладочных фрагментов только в специальных сборках;
- предотвращения повторного включения заголовочных файлов.

5.6.1. Директива `#if`

Директива `#if` позволяет включать или исключать блоки кода из компиляции в зависимости от значения константного выражения.

Сначала директива `#if` вычисляет целочисленное константное выражение, составленное из литералов и макросов. Если указанное выражение после `#if` вычисляется как ненулевое (то есть истинно), то весь код, который находится между `#if` и соответствующей закрывающей директивой `#endif`, включается в итоговый исходный файл для компилятора. Если же выражение равно нулю (ложно), то этот код полностью игнорируется и в компиляцию не попадает.

Синтаксис:

```
#if выражение
    // код для компиляции, если выражение истинно
#endif
```



Условие должно быть константным выражением (то есть его значение вычисляется на этапе компиляции).



Директива `#if` всегда парная. Для каждой такой директивы должна быть

> соответствующая закрывающая `#endif`. Если этого не сделать, компилятор выдаст ошибку.

Внутри блока `#if` можно использовать директивы `#elif` (аналог `else if`), `#else` (альтернативный блок для случая, когда `#if` ложен) и вложенные `#if`. При этом `#else`, если он есть, должен быть последней директивой перед `#endif`.

Пример использования:

```
#define DEBUG 1

#if DEBUG
    printf("Отладка включена\n");
    // какие-то отладочные функции
#endif

// Нормальная работа
```

В этом примере, если макрос `DEBUG` определён и не равен нулю, в итоговый файл попадут вызов `printf` и `какие-то отладочные функции`; в противном случае они будут исключены из компиляции.

```
#define VERSION 2

#if VERSION >= 2
    void new_feature(void);
#else
    void old_feature(void);
#endif
```

В этом примере производится проверка номера версии. В случае, если номер версии 2 или выше, вызывается функция `new_feature`, в противном случае — `old_feature`.

5.6.2. Директивы `#ifdef` и `#ifndef`

Директива `#ifdef NAME` эквивалентна `#if defined(NAME)`, а `#ifndef NAME` — `#if !defined(NAME)`. Оператор `defined(NAME)` возвращает истинное значение, если макроопределение `#define NAME` было встречено ранее (возможно, без указания значения), и ложное — в противном случае.

```
#ifdef DEBUG
    printf("Отладочное сообщение\n");
#endif

#ifndef MAX_BUFFER
    #define MAX_BUFFER 1024
#endif
```

Оператор `defined` особенно удобен в сложных условиях:

```
#if defined(OS_LINUX) && !defined(DISABLE_NETWORKING)
    /* код для Linux с поддержкой сети */
#elif defined(OS_WINDOWS)
    /* код для Windows */
#else
    #error "Неподдерживаемая операционная система"
#endif
```

5.6.3. Защита от повторного включения

Директивы условной компиляции часто используются в идиоме, которая получила название **include guard**. Она предназначена для решения проблемы повторного включения заголовочного файла.

Эта проблема заключается в следующем. Предположим, в нашем проекте используются заголовочные файлы `a.h` и `b.h`. Каждый из них, в свою очередь, включает вложенный заголовочный файл `c.h` через директиву `#include` (многоуровневое вложение одних заголовочных файлов в другие — обычная практика.

Допустим, некоторый модуль `d.c` включает и `a.h`, и `b.h`:

```
#include "a.h" ①
#include "b.h" ②
```

Сначала препроцессор выполнит первую директиву `#include` (1). Он включит в исходный код модуля `d.c` содержимое заголовочного файла `a.h`, включая вложенный файл `c.h`. Затем будет выполнена вторая директива `#include` (2), в ходе которой в модуль `d.c` будет включено содержимое `b.h`, в том числе (повторно!) и `c.h`.

Когда компилятор дойдет до второй копии `c.h`, он встретит повторные определения типов, переменных и макросов, которые он уже обработал ранее. Это приведет к ошибке компиляции (повторное определение).

Эту проблему нельзя решить, просто удалив директиву `#include "c.h"` из файла `b.h`, поскольку в проекте могут быть и другие модули, которые включают только `b.h`. Теперь ошибка возникнет уже при их компиляции, поскольку необходимые определения из `c.h` теперь недоступны.

На помощь приходит **include guard**:

```
#ifndef MYHEADER_H ①
#define MYHEADER_H ②

/* содержимое заголовочного файла */
```

```
#endif /* MYHEADER_H */ ③
```

При первом включении заголовочного файла макрос `MYHEADER_H` не определен, поэтому условие (1) истинно, и выполняется остальной текст макроопределения вплоть до `#endif` (3). Будет опроеделен макрос `MYHEADER_H` (2), а затем обработано содержимое заголовочного файла.

При втором и последующих включениях макрос `MYHEADER_H` уже определен, поэтому условие (2) ложно, и содержимое заголовочного файла игнорируется.

Альтернативой является нестандартная, но широко поддерживаемая директива:

```
#pragma once
```

При ее использовании всегда есть (небольшой) риск, что в данной версии компилятора она не реализована.

5.7. Многострочные макросы

Длинное определение макроса можно разбить на несколько строк, экранируя перевод строки обратным слешем:

```
#define INIT_STRUCT(s) do { \
    (s).x = 0;              \
    (s).y = 0;              \
    (s).flags = 0;          \
} while (0)
```



Обратите внимание на конструкцию `do { ⋯ } while (0)`. Она необходима, чтобы макрос, раскрывающийся в несколько операторов, синтаксически вёл себя как **единый оператор**. Это позволяет безопасно использовать его в конструкциях `if-else`:

```
if (condition)
    INIT_STRUCT(obj);   /* без do-while здесь возникла бы ошибка */
else
    /* ... */
```

Компилятор распознаёт, что цикл выполняется ровно один раз, и оптимизирует его, не генерируя лишнего машинного кода.

5.8. Достоинства и недостатки макросов

Макрокоманды — мощный, но специфический механизм. Они стали неотъемлемой частью

программирования на C и оказываются очень полезными при правильном применении. Однако использовать их нужно с большой осторожностью и всегда отдавать себе отчет, чтьо вы делаете.

5.8.1. Достоинства

1. **Отсутствие накладных расходов.** Макрос раскрывается на этапе препроцессинга, поэтому вызов макроса не связан с передачей управления, формированием стекового кадра и возвратом из функции. Это важно в системах с ограниченными ресурсами.
2. **Работа с синтаксисом языка.** Макросы способны генерировать идентификаторы (через `#), преобразовывать их в строки (через`) и использовать информацию о месте вызова (`FILE`, `LINE`). Ни одна языковая конструкция C на это не способна.
3. **Условная компиляция.** Препроцессор позволяет включать и исключать фрагменты программы в зависимости от конфигурации сборки.
4. **Генерация шаблонного кода.** С помощью макросов можно создавать семейства однотипных функций и структур, что востребовано в системном программировании.

5.8.2. Недостатки

1. **Отсутствие проверки типов.** Препроцессор не различает типы данных. Ошибка, связанная с несовместимостью аргументов, проявится лишь на этапе компиляции раскрытого текста, что затрудняет локализацию проблемы.
2. **Многократное вычисление аргументов.** Если аргумент макроса используется в теле несколько раз, он вычисляется соответствующее число раз. Это порой может привести к неожиданному результату:

   ```
   #define SQUARE(X) ((X) * (X))
   ```

   Вызов `SQUARE(x++)` приведёт к двойному инкременту переменной `x`, что является источником трудноуловимых ошибок.

3. **Сложность отладки.** Отладчик работает с раскрытым текстом, а не с исходным макросом. Сообщения об ошибках указывают на строки раскрытия, а не на строки исходного кода.
4. **Нарушение области видимости.** Макрос действует глобально от точки определения до конца файла или до директивы `#undef`. Он не подчиняется правилам блочной области видимости языка C.

5.9. Когда следует применять макросы

Общее правило: **если задачу можно легко решить средствами самого языка C, макрос использовать не следует**.

5.9.1. Когда макросы оправданы

- **Условная компиляция** — для адаптации программы к различным платформам и конфигурациям сборки. Альтернатив здесь нет.
- **Защита заголовочных файлов** от повторного включения.
- **Отладочные и логгирующие макросы**, использующие `FILE` и `LINE`.
- **Генерация однотипного кода** — например, семейство функций для разных типов данных, когда обобщённые средства языка недостаточны.
- **Короткие выражения**, где важна каждая операция (в критичных по производительности участках встраиваемых систем).

5.9.2. Когда следует отказаться от макросов

Задача	Плохо (макрос)	Хорошо (средства C)
Именованные целочисленные константы	`#define SIZE 100`	`enum { SIZE = 100 };`
Простые функции	`#define SQUARE(x) x)*(x`	`static inline int square(int x) { return x * x; }`
Полиморфные операции	`#define MAX(a,b) …`	Отдельные функции `max_i`, `max_d` или `_Generic` (C11)
Строковые константы	`#define MSG "text"`	`static const char MSG[] = "text";`

Конструкция `static inline` (стандарт C99) позволяет получить функцию, тело которой встраивается в место вызова, то есть по сути достичь того же отсутствия накладных расходов на вызов функции, что и у макроса, но с полной проверкой типов и корректной работой отладчика.

Оператор `_Generic` (стандарт C11) даёт возможность выбирать реализацию в зависимости от типа аргумента, что позволяет создавать типобезопасные полиморфные конструкции без текстовых подстановок.

```
static inline int    max_i(int a, int b)       { return a > b ? a : b; }
static inline double max_d(double a, double b) { return a > b ? a : b; }

#define MAX(a, b) _Generic((a), \
    int:    max_i,              \
    double: max_d               \
)(a, b)
```

В этом определении макрос используется лишь как тонкая оболочка над типобезопасными функциями, что существенно снижает связанные с макросами риски.

5.10. Резюме

Макрокоманды — неотъемлемая часть языка C. Они предоставляют возможности, недоступные языковым средствам: условную компиляцию, работу с именами идентификаторов, генерацию кода. Однако их простая текстовая природа делает макросы источником труднодиагностируемых ошибок.

Современный стиль программирования на C предполагает **минимизацию** использования макросов. Там, где раньше применялись макросы для констант и простых функций, сегодня следует использовать `enum` и `static inline`. Макросы должны оставаться там, где они действительно незаменимы: в условной компиляции, в отладочных конструкциях и в задачах генерации кода.

Chapter 6. Типы в языке C

В этой главе мы рассмотрим типы объектов в языке C. Но сначала уточним, что следует понимать под типом. Это важно, поскольку понятие типа часто трактуется неверно.

В качестве эксперимента я задавал этот вопрос нескольким нейронным сетям и периодически получал некорректный ответ, как от новичка в языке C на собеседовании. Это значит, что велика вероятность войти в заблуждение при попытке прояснить этот вопрос самостоятельно при помощи сетевых ресурсов.

6.1. Понятие типа

Тип объекта определяется двумя фундаментальными характеристиками:

Домен значений

Область значений, которые объект данного типа способен представлять. Например:

- для типа `int` это целые числа;
- для типа `double` — числа с плавающей десятичной точкой, представленные в формате "мантисса + порядок";
- для типа `bool` — одно из двух значений, `false` либо `true`;
- для типа `struct { int x; int y; }` — все возможные пары целых чисел.

Множество операций

Совокупность операций, определённых над значениями данного типа, а также их семантика. Например:

- оператор `/` для целых типов выполняет деление **нацело** с отбрасыванием дробной части, для вещественных — обычное арифметическое деление;
- оператор `++` применительно к целому увеличивает значение на единицу, а применительно к указателю — смещает его на размер элемента.



Вопреки распространенному заблуждению, тип сам по себе не определяет размер объекта в памяти. Например, тип `int` имеет различный размер для разных архитектур (типичный размер 16 либо 32 бита); одна и та же структура может иметь различный размер даже на одной архитектуре , если используются разые варианты выравнивания ее компонентов. При этом домен и множество операций над этими типами остаются принципиально теми же. Бесполезно использовать предыдущий опыт в этом вопросе, поскольку простое изменение опций компиляции или атрибутов могут существенно изменить представление объекта в памяти, не говоря уже о переходе на другую платформу (вы же помните, одно из главных преимуществ языка C — это возможность писать переносимый код, одинаково работающий на разных платформах).

Примеры:

На многих платформах типы `int` и `float` имеют одинаковый размер в памяти (по 32 бита), однако они остаются совершенно разными типами, поскольку имеют разные домены значений, разные наборы операций и даже различную семантику одинаковых операций (см. пример с делением).

Одина и та же битовая последовательность может интерпретироваться по-разному. Например, 4 байта `0x48 0x65 0x61 0x64` может быть интерпретировано как:

1. целое число `0x64616548 = 1,684,104,520` (на машине с порядком байтов little-endian);
2. адрес ячейки оперативной памяти (указатель);
3. представление числа 1.6631×10^{22} в формате IEEE 754 single precision (32 бита);
4. текстовая строка "Head" (в кодировке Windows-1251 ;
5. часть объекта агрегатного типа (массива или структуры).

Выбор интерпретации бинарного представления полностью определяется типом объекта.

Пример разной семантики оператора деления в зависимости от типа операндов:

```
int   i = 1;
float f = 1.0f;

i / 2;    /* 0    – домен целых, деление с отбрасыванием */
f / 2;    /* 0.5f – домен вещественных, обычное деление */
```



При анализе типа целесообразно задавать три вопроса:

1. Значения из какого домена он способен хранить?
2. Каковы ограничения домена (что происходит при переполнении, когда значение выходит за пределы допустимого диапазона)?
3. Какие операции над типом определены стандартом и какова их семантика?

6.2. Тип, объект, значение

Следует разграничивать три связанных понятия:

- **Тип** — характеристика, определяющая домен значений и допустимые операции.
- **Объект** — область памяти, способная хранить значение определённого типа (т. е. экземпляр типа).
- **Значение** — конкретный элемент домена, хранящийся в объекте.



Функция обладает **типом**, но не является **объектом**.



Тип `void` существует, однако объект этого типа создать нельзя.

Пример:

```
int i = 42;
```

В данном примере объект `i` имеет целочисленный тип (`int`) и инициализируется значением этого типа (`42`).

6.3. Базовые и производные типы

Типы в языке C можно разделить на две большие категории: **базовые** (fundamental types) и **производные** (derived types). Базовые типы являются примитивными — они задаются самим языком и не строятся из других типов. Производные типы конструируются из уже существующих (базовых или других производных) по определённым правилам.

6.3.1. Базовые типы

К базовым типам относятся:

- `void` — тип без значения. Используется для обозначения отсутствия результата функции, отсутствия параметров, а также как базовый тип для указателя `void *` («указатель на объект неизвестного типа»).
- **Арифметические типы**, которые, в свою очередь, делятся на:
  - **целочисленные** (integer types):
    - символьные: `char`, `signed char`, `unsigned char`;
    - обычные целые: `short`, `int`, `long`, `long long` (каждый в знаковой и беззнаковой форме);
    - логический: `_Bool` (в C23 — также псевдоним `bool`);
    - расширенные символьные (C23): `char8_t`, `char16_t`, `char32_t`.
  - **типы с плавающей точкой** (floating types): `float`, `double`, `long double`;
  - **десятичные плавающие** (C23): `_Decimal32`, `_Decimal64`, `_Decimal128`;
  - **комплексные** (C99): `_Complex`-варианты для каждого плавающего типа.
- **Перечисления** (enumerated types, `enum`) — по стандарту относятся к целочисленным типам, но объявляются пользователем и поэтому часто рассматриваются отдельно.

```
int i;              /* базовый целочисленный тип */
double x;           /* базовый плавающий тип */
void f(void);       /* void как тип результата и тип параметров */
```

6.3.2. Производные типы (derived types)

Производные типы конструируются из других типов рекурсивно. К ним относятся:

- **указатели** — `T *` (указатель на объект типа `T`);
- **массивы** — `T[N]` (массив из `N` элементов типа `T`);
- **структуры** — `struct { ⋯ }` (агрегат полей различных типов);
- **объединения** — `union { ⋯ }` (поля, разделяющие одну область памяти);
- **функции** — `T(params)` (функция, возвращающая значение типа `T`);

Производные типы могут строиться из других производных типов, что позволяет описывать сколь угодно сложные структуры данных:

```
int *p;                    /* указатель на int */
int **pp;                  /* указатель на указатель на int */
int a[10];                 /* массив из 10 int */
int *ap[5];                /* массив из 5 указателей на int */
int (*pa)[5];              /* указатель на массив из 5 int */
int (*f)(double);          /* указатель на функцию (double) -> int */
int (*af[3])(double);      /* массив из 3 указателей на функцию */
```

Обратите внимание, что тип переменной в случае производных типов определяется не одним ключевым словом, а всей конструкцией объявления: в `int (*pa)[5]` типом переменной `pa` является «указатель на массив из 5 элементов типа `int`» — производный тип, построенный рекурсивно из базового `int`.

6.4. Квалифицированные типы

Любой тип (базовый или производный) может быть снабжён одним или несколькими квалификаторами:

- `const` — объект не может быть изменён через данное имя;
- `volatile` — объект может изменяться способами, невидимыми компилятору (например, аппаратурой);
- `restrict` (только для указателей) — указатель является единственным способом доступа к объекту;
- `_Atomic` (C11) — доступ к объекту производится атомарно.

```
const int *p;              /* указатель на const int */
volatile int * const p;    /* const-указатель на volatile int */
```

Такие типы называются **квалифицированными**.

Квалифицированный тип считается **отличным** от неквалифицированного, но **совместимым** с ним. В этом заключается важное различие между **идентичностью** типа и

совместимостью типов.

6.4.1. Что значит «отличный» (distinct)?

Когда мы говорим, что `const int` **отличен** от `int`, мы имеем в виду, что это **разные типы** с точки зрения системы типов языка. Они **не** являются одним и тем же типом.

```
/* Эти два объявления имеют РАЗНЫЕ типы */
int x;
const int y;

/* sizeof(int) и sizeof(const int) дают одинаковый результат,
   но типы всё равно считаются различными */
```

6.4.2. Что значит «совместимый» (compatible)?

Несмотря на то, что типы различны, они **совместимы** — это означает, что в большинстве контекстов они могут использоваться взаимозаменяемо. Совместимость — это более слабое отношение, чем идентичность.

Примеры совместимости:

```
int x = 5;
const int y = x;        /* OK: int -> const int */
int z = y;              /* OK: const int -> int (копирование значения) */

void f(int a);
const int c = 10;
f(c);                   /* OK: передача const int в параметр типа int */
```

6.4.3. Как проявляется это различие?

Различие между «отличным» и «совместимым» проявляется в нескольких ситуациях.

Присваивание указателей

```
int x = 5;
const int *p1 = &x;     /* OK: int* -> const int* (добавление ограничения const) */

const int y = 10;
int *p2 = &y;           /* ОШИБКА: const int* -> int* (снятие ограничения const
запрещено) */
```

Здесь `int` и `const int *` — `*различные` типы, но они **совместимы** только в одну сторону: можно добавить ограничение `const`, но нельзя его снять.

Совместимость типов в объявлениях

```
/* Эти два объявления совместимы */
extern int x;
extern const int x;     /* OK: типы совместимы */

/* А это разные типы! */
int a[10];
const int b[10];        /* Это ДРУГОЙ тип массива */
```

Передача параметров и возврат значений

```
int f(void);
const int g(void);      /* OK: типы совместимы, но g возвращает const int */

void h(int x);
void h(const int x);    /* ОШИБКА: повторное объявление с другим типом параметра */
```

6.4.4. Практические последствия

«Отличный, но совместимый» означает:

- типы **не идентичны** (это разные типы в системе типов);
- но они **могут использоваться вместе** в большинстве контекстов благодаря правилам неявного преобразования;
- однако в некоторых строгих контекстах (например, повторные объявления функций, совместимость указателей) различие становится важным.

Это различие позволяет языку быть гибким (можно передавать `const int` туда, где ожидается `int`), но при этом сохранять строгость типизации (нельзя модифицировать const-объект через указатель на не-const).

Когда проявляется «отличность»

```
/* Функции с разными типами параметров */
void f(int x);
void f(const int x);    /* ОШИБКА: конфликтующие объявления */

/* Указатели на функции */
void (*p1)(int);
void (*p2)(const int);  /* Это РАЗНЫЕ типы указателей на функции */
```

Когда проявляется «совместимость»

```
/* Присваивание значений */
int a = 5;
const int b = a;        /* OK: значения совместимы */

/* Передача в функцию */
void process(int value);
const int c = 10;
process(c);             /* OK: const int совместим с int при передаче по значению */
```

6.4.5. Типы объектов и типы функций

Типы объектов — это типы, которые могут описывать объекты в памяти и, следовательно, имеют размер (`sizeof`). Все базовые типы (кроме `void`), указатели, массивы, структуры, объединения и атомарные типы являются типами объектов.

Функции — единственный вид производных типов, не являющийся типом объекта.

6.4.6. Завершённые (complete) и незавершённые (incomplete) типы

Тип считается **незавершённым**, если его размер невозможно определить в текущей точке программы. К незавершённым типам относятся `void`, массивы без указания размера (`int a[]`), структуры и объединения без определения тела.

Незавершённый тип становится **завершённым**, когда появляется недостающая информация (например, после определения структуры).

```
struct S;                   /* S — незавершённый тип */
int a[];                    /* a — массив незавершённого типа */
extern int b[];             /* допустимо для внешних объявлений */

struct S { int x; };        /* теперь S — завершённый тип */
```

Использование незавершённого типа там, где требуется знание его размера (например, в `sizeof` или при объявлении объекта), является ошибкой.

6.5. Общая картина типов в C

Все типы языка можно разделить на несколько категорий:

Категория	Примеры	Домен значений
Скалярные	`int`, `double`, `bool`, `char *`	Одно значение
Массивы	`int[10]`, `double[3][4]`	Упорядоченный кортеж из N значений
Структуры	`struct Point`	Произведение доменов полей

Категория	Примеры	Домен значений
Объединения	`union Value`	Ровно одно из представлений полей
Функции	`int (const char *)`	Сигнатура[1]
`void`	`void`, `void *`	Пустой домен

[1] Понятие сигнатуры функции зависит от языка и, в частности, от возможности перегрузки функций в этом языке. В C перегрузка функций остутствует, поэтому принято считать, что возвращаемое функцией значение входит в состав ее сигнатуры наряду с ее именем и типами параметров. В других языках, например, C++, это не так.

Chapter 7. Скалярные типы

Скалярным называется тип, объекты которого содержат ровно одно значение, в отличие от **агрегатных** типов (массивов и структур), объекты которых содержат множество значений, организованных определённым образом.

Скалярные типы включают в себя **арифметические** типы и типы-**указатели**. Арифметические типы, в свою очередь, подразделяются на **целочисленные** и **вещественные**.

То, что скаляр содержит одно значение, определяет характер операций над ним: присваивание копирует значение целиком, сравнение сопоставляет два значения, передача в функцию передаёт одно значение.

7.1. Целочисленные типы (`int`)

К целочисленным типам относятся:

- **символьные типы**: `char`, `signed char`, `unsigned char`;
- **целые со знаком**: `signed char`, `short`, `int`, `long`, `long long`;
- **целые без знака**: `unsigned char`, `unsigned short`, `unsigned int`, `unsigned long`, `unsigned long long`;
- **булев тип**: `bool` (в C99/C11/C17 — `_Bool`);
- **перечисления**: `enum`;
- **произвольные целые**: `_BitInt(N)` и `unsigned _BitInt(N)` (с C23).

Тип `char` может быть как знаковым, так и беззнаковым; это определяет конкретная реализация.

Типы `char`, `signed char` и `unsigned char` совместно называются **символьными типами** (character types).



`char`, `signed char` и `unsigned char` — это три **разных** типа!

7.1.1. Домен

Ранее (TBD) упоминалось, что в языке C не специфицируется точный размер определенного типа в байтах. Вместо этого стандарт гарантирует лишь **минимальный** домен значений для каждого типа:

Тип	Минимальный знаковый домен	Беззнаковый аналог
`char`	−127 … 127 **или** 0 … 255	знак зависит от реализации
`signed char`	−127 … 127	`unsigned char`: 0 … 255
`short`	−32 767 … 32 767	0 … 65 535

Тип	Минимальный знаковый домен	Беззнаковый аналог
`int`	−32 767 … 32 767	0 … 65 535
`long`	−2 147 483 647 … 2 147 483 647	0 … 4 294 967 295
`long long`	$-2^{63} + 1 \dots 2^{63} - 1$	$0 \dots 2^{64} - 1$

На многих платформах домены шире минимума. Ниже приведены домены для некоторых распространенных **моделей данных**:

Модель	Применение	`char`	`short`	`int`	`long`	`long long`	`void*`
LP32	Win16 API	8	16	16	32	64	4
ILP32	Win32 API, Unix, Linux, macOS	8	16	32	32	64	4
LLP64	ARM (AArch64), Windows API (x64)	8	16	32	32	64	8
LP64	Unix, Linux, macOS (64 bit)	8	16	32	64	64	8

Для случаев, когда программисту требуется **точно** знать размер объекта в байтах, в заголовке `<stdint.h>` определены типы `int8_t`, `int16_t`, `int32_t`, `int64_t` и их беззнаковые аналоги, а также семейства `int_leastN_t` и `int_fastN_t` (TBD).

7.1.2. Операции



integer promotions (TBD)

Над целочисленными типами определены следующие операции.

Унарные операции

- `+` — унарный плюс (не влияет на значение операнда);
- `-` — унарный минус (меняет знак операнда на противоположный).

Арифметические операции

- `+` — сложение;
- `-` — вычитание;
- `*` — умножение;
- `/` — целочисленное деление (дробная часть отбрасывается);
- `%` — остаток от деления (взятие по модулю)

Побитовые операции

- `&` — поразрядное "И";

- `|` — поразрядное "ИЛИ";
- `^` — поразрядное "Исключающее ИЛИ";
- `~` — поразрядное дополнение (инверсия);
- `<<` — сдвиг влево;
- `>>` — сдвиг вправо.

Операции сравнения

- `<` — меньше;
- `⇐` — меньше или равно;
- `>` — больше;
- `>=` — больше или равно;
- `==` — равно;
- `!=` — не равно.

Инкремент и декремент

- `++` — инкремент (увеличение на единицу);
- `--` — декремент (уменьшение на единицу.

Логические операции

- `&&` — логическое "И";
- `||` — логическое "ИЛИ";
- `!` — отрицание.

Присваивание

- `=` — объекту, находящемуся слева от оператора присваивания, присваивается значение выражения, находящегося справа.



Существует также другая форма оператора присваивания: `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<⇐`, `>>=`.

Если представить его в общем виде как `?=`, где символ `?` заменяет любой из вышеперечисленных опереторов, то запись

```
X ?= Y;
```

почти эквивалентна

```
X = X ? Y;
```

> за исключением того, что `X` вычисляется только один раз[1].

7.1.3. Особенности поведения

Переполнение

При выполнении операций над целочисленными типами возможен выход за границы домена (переполнение). Поведение при переполнении принципиально различается для знаковых и беззнаковых типов:

```
unsigned int u = UINT_MAX;
u + 1;          /* 0: арифметика по модулю 2^N */

int i = INT_MAX;
i + 1;          /* неопределённое поведение */
```

исходить из того, что оно не происходит (что используется для оптимизаций). Для беззнаковых типов арифметика **всегда** выполняется по модулю 2^N **по определению. Для знаковых типов переполнение является *неопределённым поведением**, и компилятор не пытается отследить эту ситуацию.

Деление и остаток

Оператор `/` для целых **не округляет**, а просто **отбрасывает** дробную часть в сторону нуля:

```
 7 /  2;    /*  3 */
-7 /  2;    /* -3 */
 7 / -2;    /* -3 */
-7 / -2;    /*  3 */
```

Оператор `%` возвращает остаток такой, что `(a / b) * b + a % b == a`.

Деление на ноль и взятие остатка от деления на ноль приводят к неопределённому поведению.

Сдвиги

- Сдвиг на отрицательную величину или на величину, не меньшую ширины типа, приводит к неопределённому поведению.
- Сдвиг отрицательного знакового числа влево приводит к неопределённому поведению в C11/C17, определено в C23 при условии, что результат представим. (TBD)
- Сдвиг вправо знаковых отрицательных чисел вызывает поведение, определенное реализацией (обычно арифметический сдвиг с сохранением знака).

7.1.4. Продвижение целых

Перед выполнением большинства операций над целочисленными типами выполняется **продвижение целых** (integer promotions).

Если тип операнда имеет меньшую разрядность, чем `int`, он преобразуется к `int` (если `int` может представить все значения исходного типа) либо к `unsigned int` (в противном случае).

К операндам, к которым применяется продвижение, относятся:

- операнды унарных операторов `+`, `-`, `~`;
- операнды `char` и `short` (любого знака) в бинарных операторах;
- операнды битовых полей, если их тип имеет меньшую разрядность, чем `int`;
- условие оператора `?:`.

Примеры:

```
unsigned char a = 255;
unsigned char b = 1;

a + b;        /* 256, а не 0 */
```

Операнды `a` и `b` продвигаются до `int`, поэтому сложение выполняется в домене `int` и дает в результате значение 256. Приведение обратно к `unsigned char` не происходит.

```
unsigned char c = 0xFF;
~c;           /* не 0x00, а 0xFFFFFF00 (на 32-битном int) */
```

Оператор `~` применяется уже после продвижения, поэтому инвертируются все 32 бита.

```
unsigned char x = 0;
-x;           /* тип результата — int, значение 0 */
```

Унарный минус возвращает `int`, а не `unsigned char`.



Продвижение целых — одна из наиболее частых причин «неожиданного» поведения в программах на C. Внимательно разберитесь в этом.

7.2. Целочисленные типы произвольной разрядности (`_BitInt(N)`)

Стандарт C23 ввёл скалярный тип `_BitInt(N)`, представляющий целое число разрядностью N бит.

```
_BitInt(12)          small  = 123wb;
unsigned _BitInt(40) medium = 1000000000000ub;
_BitInt(73)          large  = 0wb;
```

7.2.1. Домен

- Знаковый `_BitInt(N)`: от $-2^{(N-1)}$ до $2^{(N-1)} - 1$;
- Беззнаковый `unsigned _BitInt(N)`: от 0 до $2^N - 1$.

Параметр N может быть любым от 1 до значения, установленного реализацией в макросе `BITINT_MAXWIDTH` заголовка `<limits.h>` (стандарт требует поддержки минимум 1…65535).

7.2.2. Операции

Те же, что и для обычных целых: арифметика, побитовые операции, сравнение.

7.2.3. Особенности поведения

- Ширина типа равна в точности N бит, а не «не менее N бит».
- Суффиксы литералов: `wb` / `WB` для знаковых, `ub` / `UB` для беззнаковых;
- продвижение целых для `_BitInt(N)` при N ≤ `sizeof int` выполняется до `int`, иначе — до `_BitInt(N)` (беззнаковый — до `unsigned _BitInt(N)`).

7.2.4. Область применения

- Криптография и хеш-функции, где нужны нестандартные ширины (256, 512 бит);
- сетевые протоколы с полями произвольной ширины;
- работа с аппаратными регистрами;
- ситуации, где требуется ширина, гарантированно не зависящая от платформы.

7.3. Перечисления

7.3.1. Семантика

Перечисление (`enum`) фактически представляет собой разновидность целочисленного типа, значения которого идентифицируются именованными константами-перечислителями.



Перечисление **не вводит** нового способа представления данных; оно вводит **имена** для целых чисел.

Главное назначение перечислений — сделать код читаемым и понятным. Вместо "магических чисел" используются символические имена.

```
enum Color { RED, GREEN, BLUE };
```

```
enum Color c = GREEN;   /* вместо «магического числа» 1 */
```

7.3.2. Синтаксис и правила нумерации

Общая форма объявления:

```
enum имя { список-перечислителей };
```

Правила вычисления значений:

1. Если у первого перечислителя нет явно заданного посредством `=` значения, его значение равно **0**.
2. Если у очередного перечислителя нет `=`, его значение на **единицу больше** значения предыдущего.
3. Перечислитель с `=` получает значение указанного константного выражения; дальнейший отсчёт продолжается от него.

```
enum Example {
    A,          /* 0 — правило 1 */
    B,          /* 1 — правило 2 */
    C = 10,     /* 10 — правило 3 */
    D,          /* 11 */
    E = -5,     /* -5: отрицательные значения допустимы */
    F,          /* -4 */
    G = A,      /* 0: дубликаты значений допустимы */
    H,          /* 1 */
                /* завершающая запятая разрешена начиная с C99 */
};
```

Выражение, задающее значение перечислителя, должно быть **целым константным выражением**. В нём допустимы литералы, ранее объявленные перечислители, оператор `sizeof`, приведения констант, `_Static_assert`-совместимые вычисления:

```
enum Limits {
    WORD_BYTES  = sizeof(long),         /* sizeof — константное выражение */
    BUF_WORDS   = 64,
    BUF_BYTES   = BUF_WORDS * WORD_BYTES
};
```

Поскольку значения перечислителей — целые константные выражения, их можно использовать всюду, где язык требует константы:

- в размерах массивов,
- в метках `case`,

- в размерах битовых полей,
- в статических инициализаторах.

7.3.3. Домен

Переменная перечислимого типа совместима с некоторым целочисленным типом (выбираемым реализацией).

Замкнутость домена перечислений

В контексте типов «замкнутость» означает, что тип определяет **строго ограниченное** множество допустимых значений, и выход за его границы является ошибкой или неопределённым поведением.

В C перечислимый тип **не является** замкнутым множеством. Это означает, что переменная перечислимого типа в C может хранить **любое значение**, представимое в совместимом целочисленном типе, а не только значения, перечисленные в списке перечислителей.

```
enum Color { RED, GREEN, BLUE };    // 0, 1, 2

enum Color c = RED;
c = 42;             // допустимо: значение представимо в совместимом типе
c = GREEN | BLUE;   // 3 — тоже допустимо, хотя такого перечислителя нет
c++;                // допустимо: инкремент целочисленного типа
c = -100;           // допустимо, если совместимый тип — знаковый
```

Практические последствия:

1. Нельзя полагаться на то, что переменная перечислимого типа содержит одно из перечисленных значений.
2. При чтении значений из файла, сети или разделяемой памяти необходимо явно проверять их допустимость.
3. Побитовые операции над перечислениями дают значения вне списка перечислителей, и это **законно**.
4. Следует использовать `switch` с `default` для обработки неожиданных значений.
5. Следует включать предупреждения компилятора `-Wswitch` и `-Wenum-conversion`, чтобы обнаруживать смешение разных перечислений.



Перечисление в C — это **именованный целочисленный тип с подсказками**, а не строгая проверка принадлежности множеству. Программист обязан сам следить за тем, чтобы значения оставались в ожидаемом диапазоне.



- Размер объекта перечислимого типа не задан стандартом и может отличаться на разных платформах и даже при разных ключах компиляции.
- Знаковость перечисления зависит от допустимого набора значений. Перечисление с отрицательными значениями не может быть совместимо с беззнаковым типом.



Распространённые реализации ведут себя по-разному.

- GCC и Clang по умолчанию используют `unsigned int` для перечислений с неотрицательными значениями и `int` при наличии отрицательных; ключ `-fshort-enums` заставляет выбирать наименьший вмещающий тип.
- MSVC всегда использует `int`.

7.3.4. Множество операций

Над перечислениями определены те же операции, что и над целыми: арифметика, сравнения, присваивания. При выполнении операций перечисления продвигаются до целого типа.

7.3.5. Анонимные перечисления

Перечисление может быть объявлено **без тега** — так называемое **анонимное перечисление**:

```
enum { RED, GREEN, BLUE };
```

Такое объявление не создаёт именованного типа, но **создаёт перечислители** в текущей области видимости. Они ведут себя как обычные целые константы:

```
enum {
    BUF_SIZE   = 1024,
    MAX_ERRORS = 10,
    VERSION    = 3
};

char buffer[BUF_SIZE];
int errors = 0;

if (errors > MAX_ERRORS) {
    /* ... */
}
```

Использование

Анонимные перечисления используются как **альтернатива** `#define` для групп связанных целочисленных констант. Такая альтернатива обладает рядом преимуществ:

- **Видимость отладчику**: перечислители сохраняются в отладочной информации, в отличие от макросов, которые исчезают после препроцессора.
- **Лексическая область видимости**: перечислители подчиняются обычным правилам области видимости C, а не действуют до конца файла, как макросы.
- **Последовательная нумерация**: если значения не заданы явно, они автоматически увеличиваются, что удобно для счётчиков и кодов.
- **Группировка**: несколько связанных констант объединяются в одно объявление, что улучшает читаемость кода.

```
/* Вместо множества #define */
#define RED   0
#define GREEN 1
#define BLUE  2

/* Лучше: анонимное перечисление */
enum { RED, GREEN, BLUE };
```

Ограничения

Поскольку анонимное перечисление не имеет имени, нельзя объявить переменную этого типа:

```
enum { LOW, MEDIUM, HIGH };

/* enum ??? priority = LOW; */   /* невозможно: у перечисления нет имени */
int priority = LOW;              /* допустимо: перечислитель — это int */
```

Если требуется создание типа, необходимо использовать именованное перечисление или `typedef`:

```
typedef enum { LOW, MEDIUM, HIGH } Priority;
Priority p = HIGH;
```

Сравнение с именованными перечислениями

Свойство	**Анонимное** `enum { ⋯ }`	**Именованное** `enum Name { ⋯ }`
Создаёт тип	нет	**да**
Можно объявить переменную	нет	**да**

Свойство	Анонимное `enum { ⋯ }`	Именованное `enum Name { ⋯ }`
Перечислители видны	**да**	**да**
Альтернатива `#define`	**да**	частично
Область видимости перечислителей	лексическая	лексическая
Фиксированный базовый тип (C23)	**да**	**да**

Практические правила

- Используйте **анонимные перечисления** для групп связанных целочисленных констант, когда типизация не требуется: коды ошибок, флаги, индексы.
- Используйте **именованные перечисления**, когда нужна типизация, проверка полноты в `switch` или передача значений как параметров функций.
- Избегайте смешения: если константы логически связаны с типом, используйте именованное перечисление; если это просто набор чисел — анонимное.

7.3.6. Особенности поведения

Типы перечисления и перечислителей

До стандарта C23 перечислители всегда имеют тип `int` **независимо** от того, с каким типом совместимо само перечисление. Это порождает неочевидное противоречие:

```
enum Small { S_A, S_B, S_C };
enum Small s;

sizeof(S_A);    /* sizeof(int) — перечислитель имеет тип int */
sizeof s;       /* размер перечислимого типа, выбираемый реализацией */
```

При компиляции с `-fshort-enums` эти два выражения могут дать разные результаты: например, `sizeof(S_A)` равно 4, а `sizeof s` 1.

Область видимости перечислителей

Перечислители — это **обычные идентификаторы** той области видимости, в которой объявлено перечисление. Тег перечисления их не ограничивает.

```
enum Color { RED, GREEN, BLUE };
enum Signal { RED, YELLOW };    /* ошибка: RED уже объявлен в этой области */
```

```
int RED;                        /* ошибка: то же имя в той же области */

void f(void) {
    int RED = 5;                /* допустимо: внутренний блок, имя скрыто */
}
```



Сам **тег** перечисления живёт в отдельном пространстве имён тегов, поэтому он не конфликтует с переменными:

```
enum Color { C_RED, C_GREEN };
int Color;                      /* допустимо: тег и переменная — разные
пространства */
```

На практике для сокращения записи широко применяется `typedef`:

```
typedef enum { LOW, MEDIUM, HIGH } Priority;
Priority p = HIGH;              /* слово enum не требуется */
```

Смешивание перечислений

Все перечислимые типы являются целочисленными. Поэтому присваивание значения одного перечисления переменной другого формально допустимо и даже не требует приведения:

```
enum Color  { RED, GREEN, BLUE };
enum Status { OK, FAIL };

enum Status st = RED;       /* допустимо в C, хотя смысл утрачен */
```

Это одно из слабых мест системы типов C. Некоторые компиляторы обнаруживают такие смешения при включённых соответствующих предупреждениях (например, `-Wenum-conversion` в GCC/Clang). При работе с перечислениями обязательно нужно использовать эти ключи.

Перечисления и оператор switch

Перечисления особенно удобны в `switch`: компилятор способен проверить, все ли перечислители обработаны.

```
enum Color { RED, GREEN, BLUE };

const char *name(enum Color c) {
    switch (c) {
        case RED:   return "red";
        case GREEN: return "green";
```

```
        case BLUE:  return "blue";
    }
    return "?";     /* значения вне списка всё же возможны */
}
```

При отсутствии одной из меток GCC и Clang с ключом `-Wswitch` выдают предупреждение. Это единственный механизм, отчасти компенсирующий открытость домена перечисления.



Наличие ветви `default` подавляет такое предупреждение. Если важна проверка полноты, `default` лучше не писать, а обработку «невозможных» значений вынести после `switch`.

Перечисления в стандарте C23

Стандарт C23 (ISO/IEC 9899:2024) существенно расширил возможности перечислений, устранив две давние проблемы: неопределённость представления и ограничение диапазона значений.

Фиксированный базовый тип

В C23 после имени перечисления можно указать **базовый тип** (fixed underlying type) через двоеточие:

```
enum Byte : unsigned char {
    B_MIN = 0,
    B_MAX = 255
};

_Static_assert(sizeof(enum Byte) == 1, "expected one byte");
```

Теперь перечислимый тип совместим не с «некоторым типом по выбору реализации», а с явно указанным типом. Это даёт:

- **предсказуемый размер** объекта;
- **предсказуемую знаковость**;
- **стабильный ABI**[2] — двоичное представление структур с перечислениями перестаёт зависеть от ключей компиляции;
- возможность использовать перечисления в протоколах и аппаратных регистрах без опасений.

```
struct Header {
    enum Byte  kind;        /* ровно 1 байт */
    enum Flags : unsigned int flags;
};
```

Базовым типом может быть `char`, знаковый или беззнаковый целый тип.

Значения вне диапазона int

До C23 значение перечислителя обязано было быть представимо в типе `int`. Это делало невозможными, например, перечисления 64-битных масок.

В C23 ограничение снято: допустимы значения, выходящие за диапазон `int`, что естественно в сочетании с соответствующим фиксированным базовым типом.

```
/* C17: ошибка — значение не представимо в int */
/* C23: допустимо */
enum Mask : unsigned long long {
    M_LOW  = 1ULL << 0,
    M_HIGH = 1ULL << 40,
    M_ALL  = ~0ULL
};
```

Соответственно изменились и правила определения типа самих перечислителей: он больше не обязан всегда быть `int`. Если базовый тип указан, перечислители соотносятся с ним; если не указан и значение представимо в `int`, сохраняется прежнее поведение.

Объявление без определения

Указание базового типа делает объявление перечисления **полным** даже без списка перечислителей. Это позволяет объявить тип заранее, а определить позже:

```
enum Handle : int;              /* C23: тип полон, размер известен */

struct Registry {
    enum Handle key;            /* допустимо: тип уже полный */
};

enum Handle : int {             /* определение */
    H_NONE  = -1,
    H_FIRST = 0
};
```

В C17 подобный приём был невозможен: перечислимый тип становился полным только после списка перечислителей.

Сводка изменений

Аспект	До C23	C23
Совместимый тип	Выбирается реализацией	Можно задать явно: `enum E : T`
Размер объекта	Не определён стандартом	Определён при заданном базовом типе

Аспект	До C23	C23
Диапазон значений перечислителей	Должен помещаться в `int`	Ограничение снято
Тип перечислителя	Всегда `int`	Зависит от значения и базового типа
Объявление без списка	Неполный тип, вне стандарта	`enum E : T;` — полный тип
Анонимные перечисления с базовым типом	нет	`enum : T { … }`
Область видимости перечислителей	Объемлющая область	Без изменений
Замкнутость домена	Домен открыт	Без изменений

7.3.7. Практические приёмы

Счётчик числа элементов

Завершающий перечислитель-«часовой» даёт количество значений, которое автоматически остаётся корректным при расширении списка:

```
enum Month {
    JAN, FEB, MAR, APR, MAY, JUN,
    JUL, AUG, SEP, OCT, NOV, DEC,
    MONTH_COUNT             /* 12 */
};

static const char *month_name[MONTH_COUNT] = {
    "январь", "февраль", "март",     "апрель", "май",    "июнь",
    "июль",   "август",  "сентябрь", "октябрь","ноябрь", "декабрь"
};
```



Приём работает корректно **только при последовательной нумерации без пропусков**. Если значения заданы явно, `MONTH_COUNT` перестаёт играть роль счетчика.

Битовые флаги

Перечисление со степенями двойки удобно для наборов битовых флагов.

Комбинация флагов даёт значение вне списка перечислителей (в C это законно):

```
enum Perm : unsigned int {      /* базовый тип — синтаксис C23 */
    PERM_NONE  = 0,
    PERM_READ  = 1u << 0,
    PERM_WRITE = 1u << 1,
    PERM_EXEC  = 1u << 2
```

```
};

enum Perm p = PERM_READ | PERM_WRITE;

if (p & PERM_WRITE) {
    /* запись разрешена */
}
p |= PERM_EXEC;
p &= ~PERM_READ;
```



Здесь лучше использовать беззнаковый базовый тип; побитовые операции над знаковыми значениями требуют осторожности.

X-макросы: единый источник имён

Иногда требуется получать доступ не только к значению перечислителя, но и к его идентификатору. Чтобы избежать рутинной ручной работы для создания таблицы имен (которую к тому же придется потом поддерживать), можно воспользоваться макросом:

```
#define COLORS(X)   X(RED) X(GREEN) X(BLUE)

#define AS_ENUM(n)   n,
#define AS_STRING(n) #n,

enum Color { COLORS(AS_ENUM) };
static const char *color_name[] = { COLORS(AS_STRING) };

#undef AS_ENUM
#undef AS_STRING
```

Добавление нового цвета в макрос `COLORS` автоматически расширяет и перечисление, и таблицу имён.

Перечисления, `#define` и `static const`

Три способа ввести именованную целую константу в C решают разные задачи:

Свойство	`enum`	`#define`	`static const int`
Виден препроцессору	нет	**да**	нет
Виден отладчику	**да**	обычно нет	**да**
Создаёт отдельный тип	**да**	нет	нет

Свойство	`enum`	`#define`	`static const int`
Проверка полноты в `switch`	**да**	нет	нет
Является константным выражением	**да**	**да**	нет
Пригоден для размера массива	**да**	**да**	нет (кроме VLA)
Занимает память	нет	нет	обычно да
Область видимости	лексическая	до `#undef` / конца файла	лексическая

Правила выбора:

- связанные наборы значений, состояния, коды ошибок — `enum`;
- константы, нужные препроцессору или сборке — `#define`;
- единичные типизированные константы, адресуемые через указатель — `static const`.

7.3.8. Резюме

Перечисление в языке C — это **именованный целочисленный тип**, а не строгое алгебраическое множество.

- **Семантика**: имена для целых констант-перечислителей плюс отдельный тег типа.
- **Домен**: значения перечислителей задают «ожидаемое» подмножество, но фактический домен определяется совместимым целочисленным типом и остаётся открытым.
- **Операции**: полный набор целочисленных операций; перечислители пригодны всюду, где нужно целое константное выражение.
- **Анонимные перечисления**: удобный способ ввести группу связанных целочисленных констант без создания отдельного типа — альтернатива `#define` с сохранением области видимости и видимости отладчику.
- **Особенности**:
  - продвижение целых,
  - тип перечислителей `int` (до C23),
  - отсутствие собственной области видимости,
  - невидимость препроцессору,

7.4. Вещественные типы

К вещественным типам относятся:

- `float` — 32-битный двоичный формат (IEC 60559 binary32);
- `double` — 64-битный двоичный формат (IEC 60559 binary64);
- `long double` — расширенный формат, зависящий от реализации (80-битный на x86, binary128 на ARM, иногда совпадает с `double`);
- с C23 в основном тексте — десятичные типы `_Decimal32`, `_Decimal64`, `_Decimal128`.

7.4.1. Домен

Домен включаен:

- нормализованные конечные числа;
- субнормальные (денормализованные) числа;
- два нуля: `+0` и `−0`;
- две бесконечности: `+∞` и `−∞`;
- множество NaN (quiet и signaling).



Важно помнить, что **домен вещественного типа включает не все вещественные числа**, а конечное множество представимых чисел вместе со специальными значениями (±0, ±∞, NaN). Это естественное следствие огранниченного числа разрядов для представления таких чисел. Десятичные типы обладают иным доменом: в них точно представимы десятичные дроби вида `0.1`, что делает их пригодными для финансовых вычислений.

```
0.1 + 0.2 == 0.3;       /* false: 0.1 и 0.2 не представимы точно */
NAN == NAN;             /* false: NaN не равен ничему */
+0.0 == -0.0;           /* true: нули равны при сравнении */
1.0 / 0.0;              /* +inf, а не аварийное завершение */
```

7.4.2. Операции

Над вещественными типами определены следующие операции.

Унарные операции

- `+` — унарный плюс (не влияет на значение операнда);
- `-` — унарный минус (меняет знак операнда на противоположный).

Арифметические операции

- `+` — сложение;
- `-` — вычитание;
- `*` — умножение;
- `/` — деление.

Операции сравнения

- `<` — меньше;
- `⇐` — меньше или равно;
- `>` — больше;
- `>=` — больше или равно;
- `==` — равно;
- `!=` — не равно.

Присваивание

- `=` — объекту, находящемуся слева от оператора присваивания, присваивается значение выражения, находящегося справа.



Как и для целочисленных типов, поддерживаются формы оператора присваивания `+=, `-=`, `*=`, `/=`.

Функции стандартной библиотеки

Следующие стандартные функции определены в заголовке `<math.h>`: (TBD)

- классификация:
  - `fpclassify`,
  - `isnan`,
  - `isinf`;
- округление:
  - `trunc`,
  - `floor`,
  - `ceil`,
  - `round`;
- манипуляция: **`copysign`,
  - `nextafter`;
- математические:
  - `sqrt`,

- `sin`,
- `exp`,
- `log`.

Не определены

- оператор остатка `%` (вместо него используется `fmod` или `remainder`);
- побитовые операции;
- инкремент и декремент.

7.4.3. Особенности поведения

- Операция сложения не ассоциативна: `(a + b) + c` может отличаться от `a + (b + c)`.
- Функция `fma(x, y, z)` вычисляет `x * y + z` с **одним промежуточным округлением**, что полезно для точных вычислений.
- Режимы округления и исключения плавающей точки управляются через заголовок `<fenv.h>`.

7.5. Булев тип

Булев тип предназначен для представления логических значений.

7.5.1. Домен

Домен булева типа минимален; он включает лишь два значения: `false` и `true`.

7.5.2. Операции

Над булевым типом определены следующие операции.

Логические операции

- `&&` — логическое "И";
- `||` — логическое "ИЛИ";
- `!` — логическое "НЕ" (отрицание).

(TBD) Замыкание

Операции сравнения

- `==` — равно;
- `!=` — не равно;
- `<` — меньше;
- ⇐-- меньше или равно;

- `>` — больше;
- `>=`-- больше или равно.

Операция присваивания

- `=` — присваивание.



Язык C имеет давнюю и заслуженную репутацию одного из лучших инструментов для того, чтобы "отстрелить себе ногу"[3]. Немалый вклад в такую славу внесла вечная путаница между равенством (`==`) и присваиванием (`=`), которая может привести к неожиданным (особенно для новичков) последствиям.

7.5.3. Особенности поведения

При применении побитовых операций (как и арифметических) значение операндов продвигается до `int`, что приводит к неочевидным на первый взгляд последствиям:

```
true + true;    /* 2, тип int */
~false;         /* -1 (все биты инвертированы в promoted int) */
```

Любое скалярное значение при приведении к `bool` принимает значение 0 или 1:

```
bool a = 42;        /* true  */
bool b = 0.0;       /* false */
bool c = NULL;      /* false */
```

7.5.4. Эволюция

- В ранних версиях языка в качестве логических значений использовались целочисленные (`int`).
- В C99 введён тип `_Bool`; идентификаторы `bool`, `true`, `false` определены как макросы в заголовке `<stdbool.h>`.
- В C23 `bool`, `true`, `false` стали ключевыми словами. Заголовок `<stdbool.h>` сохранён для совместимости с предыдущими версиями.

7.6. Указатели

Указатель — это скалярный производный тип, значение которого обеспечивает **доступ** к объекту (или функции) некоторого другого типа.



Очень часто можно встретить мнение, что указатель представляет собой **адрес** объекта, на который он указывает. Это ошибочное мнение. Хотя для многих современных архитектур с плоской моделью памяти указатель действительно реализуется как адрес объекта в памяти, в общем случае это

> не так. Например, в реальном режиме микропроцессора i8086 20-разрядный физический адрес вычисляется как сумма 16-разрядного базового адреса сегмента, сдвинутого влево на 4 бита, и 16-разрядного смещения в пределах сегмента, при этом **разные** пары сегмент-смещение могут указывать на **один и тот же** физический адрес. Существуют и другие варианты (например, для вычисления адреса используется индекс в таблице дескрипторов страниц памяти).
>
> Поскольку мы решили учиться писать **портабельный** код на C, который корректно работает на **любой** архитектуре, мы не должны закладываться на знание частных особенностей конкретной архитектуры. Следует рассматривать указатель как **абстракцию**, обеспечивающую доступ к сущности некоторым способом, зависящим от реализации на данной платформе. Поэтому корректно говорить, что указатель **указывает** (points to) на объект, а не "равен его адресу".

Объявление указателя имеет вид:

```
T *p;
```

где `T` — тип объекта, на который он указывает.

7.6.1. Домен

Домен указателя `T *` — это множество значений, каждое из которых обеспечивает **доступ** (reference) к сущности типа `T` согласно семантике языка C.

Указатель `T *` может принимать следующие значения:

1. Значение, обеспечивающее доступ к объекту типа `T` (или функции, если `T` — функциональный тип). Это значение указывает на существующий объект или функцию.
2. Значение, указывающее на позицию **непосредственно за последним элементом массива** объектов типа `T`. Это специальное значение; его можно использовать в операциях арифметики указателей и сравнения, но запрещено **разыменовывать**.
3. Нулевое значение указателя (null pointer value) — значение, которое **гарантированно** не обеспечивает доступ ни к какому объекту или функции.

Значения, обеспечивающие доступ к объектам других типов, **не входят** в домен `T`; однако они могут быть *приведены к типу `T *` (с использованием операции приведения).

7.6.2. Операции

Присваивание

```
`p = q`;
```

где `p` и `q` — указатели.

Указатель может быть присвоен:

- указателю того же типа;
- указателю типа `void *` и обратно — неявно, без приведения (поэтому `void *` может использоваться как "указатель общего назначения");
- указателю на **совместимый** тип;
- специальному значению **нулевого указателя** — константе `NULL` (макрос из `<stddef.h>`) или целочисленной константе `` `0` ``[4].



Начиная со стандарта C23 в язык введено значение `nullptr` (см. TBD), имеющее специальный тип `nullptr_t`.

Разыменование

Разыменование (`*p`) предоставляет доступ к объекту, на который указывает `p`.

Доступ к полю структуры

Если `p` указывает на структуру, в состав которой входит поле `member`, то операция `p->member` предоставляет доступ к этому полю.



Фактически это эквивалентно `(*p).member`.

Арифметика

Арифметические операции над указателями тесно связаны с понятием **массива**. TBD

- Если `K` — значение, которое можно привести к целому типу, то `p + K` и `p - K` указывают на объект, смещенный относительно `p` на `K` **элементов** соответственно вправо или влево (то есть на `n * sizeof(T)` **байт**).
- `p++`, `++p`, `--p`, `p--` — частный случай смещения на **один** элемент (при `K == 1`).
- `p - q` — расстояние в **элементах** между двумя указателями (определено только в случае, если оба указателя ссылаются на элементы **одного** массива или на элемент сразу за его концом). Для представления разности указателей используется специально созданный для этой цели знаковый целочисленный тип, который определён в заголовочном файле `<stddef.h>`.

Сравнение

- `p == NULL` — сравнение с нулевым указателем (используется для проверки валидности указателя);
- `p == q` — сравнение указателей на **равенство**, то есть проверка, указывают ли два указателя на один и тот же объект (или оба являются нулевыми); , `p != q` — сравнение указателей на **неравенство**.



Для следующих операций сравнения результат определен только в следующих случаях:

- когда оба указывают на элементы одного массива (или на элемент сразу за его концом);
- когда оба указывают на поля одной структуры.

В остальных случаях результат не определён.

- `p < q`;
- `p > q`;
- `p <= q`;
- `p >= q`

Приведение к другим типам

Приведение к другим типам указателей

Указатели можно явно приводить к другим типам указателей:

```
A *pa;
...
B *pb = (B*)pa;
```

При этом следует отдавать себе отчет о проблемах, к которым может привести такое приведение.

Во-первых, довольно легко получить неопределенное поведение. Например, в приведенном выше примере может оказаться, что значение `*pa` имеет битовый паттерн, недопустимый для типа `B`. В этом случае при попытке разыменовать `pb` возникнет неопределенное поведение.

Во-вторых, даже если удастся избежать неопределенного поведения, остается вероятность получить поведение, зависящее от платформы. Например:

```
void process_byte(const uint8_t *);
...
uint32_t i = 0x12345678L;
...
uint8_t *pb = (uint8_t *)&i;
for (size_t n = 0; n < 4; ++n)
{
    process_byte(pb++);
}
```

В данном случае мы предприняли наивную попытку интерпретировать 32-битовое целое

беззнаковое число как массив из четырех смежных байтов. Проблема состоит в том, что на машинах little-endian и big-endian этот код будет работать по-разному, что будет неожиданным и крайне неприятным сюрпризом для его автора.

Приведение к целым типам

Довольно распространенная ошибка — попытка привести значение указателя к типу `int`. Она обусловлена, по-видимому, тем, что на 8- и 16-разрядных платформах, с которых многие начинают свой путь в эмбеддинг, диапазон адресуемой памяти не превышает диапазон целого типа, поэтому подобные трюки обычно сходят с рук и закрепляются в памяти. Однако на 64-разрядных платформах это уже не пройдет (причина указана в TBD). Получаем проблему непереносимости кода между платформами.

Для решения этой проблемы в заголовочном файле `<stdint.h>` из стандартной библиотеки предусмотрены два специальных целочисленных типа достаточной для представления указателя на данной платформе ширины:

- `intptr_t` — знаковый,
- `uintptr_t` — беззнаковый.

Их использование дает безопасный переносимый код.

Примеры операций с указателями

```
int a[10];
int *p = a;       /* присваивание: указатель на массив -> указатель на int */
int *q = p;       /* присваивание указателей одного типа */
void *v = p;      /* неявное преобразование в void * */

p + 1;            /* смещение на sizeof(int) байт */
p == q;           /* истинно: оба указывают на a[0] */
p < &a[5];        /* истинно: оба внутри одного массива */
p == NULL;        /* ложно: p не является нулевым указателем */
```

7.6.3. Указатель на объект неизвестного типа

Среди разных типов указателей есть один совершенно особый случай: это `void *` — указатель на объект неизвестного типа. Он неявно преобразуется в любой объектный указатель и обратно.

В стандарте языка размер "пустого" типа `void` (т. е. `sizeof(void)`) не определен. Как неизбежное следствие этого, арифметика над `void *` также не определена, поскольку для вычисления величины смещения необходимо знать размер объекта, на который указывает исходный указатель. Однако некоторые реализации используют расширение стандарта, трактуя `sizeof(void)` как 1). Использование подобных расширений крайне не рекомендуется без настоятельной необходимости, поскольку использующий их код теряет переносимость.

7.7. Обычные арифметические преобразования

Когда в бинарной операции участвуют два операнда разных целочисленных или вещественных типов, сначала к обоим применяются продвижения целых, а затем — **обычные арифметические преобразования** (usual arithmetic conversions), приводящие операнды к общему типу.

7.7.1. Правила (упрощённые)

1. Если один операнд имеет тип `long double`, другой приводится к `long double`.
2. Иначе если один — `double`, другой — к `double`.
3. Иначе если один — `float`, другой — к `float`.
4. Иначе применяются продвижения целых.
5. Если после продвижения типы совпадают — операция выполняется в этом типе.
6. Иначе если оба операнда одной знаковости — операнд меньшего ранга приводится к типу операнда большего ранга.
7. Иначе (операнды разных знаковостей):
   - если беззнаковый тип имеет ранг не ниже знакового — знаковый приводится к беззнаковому;
   - если знаковый тип может представить все значения беззнакового — беззнаковый приводится к знаковому;
   - иначе оба приводятся к беззнаковому типу, соответствующему знаковому.

7.7.2. Практические следствия

```
int    i = -1;
unsigned u = 1;

i < u;      /* false: i приводится к unsigned, становится UINT_MAX */
```

Смешение знаковых и беззнаковых типов — классический источник ошибок. Компиляторы обычно выдают предупреждение при таком сравнении.

```
int    a = 1;
double b = 2.0;

a + b;      /* тип результата — double, значение 3.0 */
```

При смешении целого и вещественного типов целое приводится к вещественному.

Оператор `sizeof` к указателю возвращает размер самого указателя, а не объекта, на который он ссылается.

7.8. Преобразования между скалярными типами

7.8.1. Неявные приведения типа

Неявные приведения происходят:

- при присваивании значения одного скалярного типа переменной другого типа;
- при передаче аргумента функции (если есть прототип);
- при возврате значения из функции;
- в бинарных операциях — через обычные арифметические преобразования; TBD
- в условии `if`, `while`, `for`, `?:` — приведение к `bool`.

```
int    i = 3.7;       /* 3: отбрасывание дробной части */
double d = 42;        /* 42.0 */
bool   b = -1;        /* true */
```

7.8.2. Явные приведения типа

Оператор `(T)x` выполняет явное приведение. Он не меняет правил преобразования, но подавляет предупреждения компилятора и делает намерения программиста явными.

```
int i = (int)3.7;                /* явно: 3 */
unsigned u = (unsigned)-1;       /* явно: UINT_MAX */
```

7.8.3. Потеря информации

При приведении вещественного типа к целочисленному:

- дробная часть отбрасывается;
- если значение не помещается в домен целочисленного типа, это вызывает неопределённое поведение (в C23 — зависящее от реализации). TBD

При приведении целого к вещественному:

- значение округляется до ближайшего представимого;
- если значение слишком велико — поведение не определено.

При приведении между целыми типами:

- беззнаковое → знаковое: битовое представление сохраняется, интерпретация меняется;
- знаковое → беззнаковое: битовое представление сохраняется, интерпретация меняется;
- более широкий → более узкий: отбрасываются старшие биты.

7.9. Сводная таблица скалярных типов

Тип	Домен	Ключевые операции	Особенности
Целые со знаком	Минимум −32 767 … 32 767	арифметика, побитовые	переполнение — UB
Целые без знака	Минимум 0 … 65 535	арифметика, побитовые	модулярная арифметика
`bool`	`false`, `true`	логические, сравнения	продвижение до `int`
`float`, `double`, `long double`	Конечное подмножество вещественных	арифметика, `<math.h>`	нет `%`, нет побитовых
Указатели	Адреса + нулевой указатель	разыменование, арифметика	шаг зависит от типа
`nullptr_t` (C23)	Нулевой указатель	присваивание указателям	только одно значение
`enum`	Именованные целые	как у целых	не замкнутый домен
`_BitInt(N)` (C23)	Ровно N бит	как у целых	произвольная ширина

7.10. Резюме

Все скалярные типы языка C объединяет общее свойство: каждый объект такого типа содержит ровно одно значение. Однако между ними существуют значительные различия:

- **Целочисленные типы** имеют модулярную арифметику для беззнаковых и неопределённое поведение при переполнении для знаковых. Перед операциями над ними применяется продвижение целых, которое часто приводит к неочевидным для начинающих результатам.
- **Вещественные типы** имеют конечный домен со специальными значениями, неассоциативную арифметику и значительную часть операций, вынесенных в функции `<math.h>`.
- **Булев тип** имеет минимальный домен из двух значений, но при арифметических операциях продвигается до `int`.
- **Указатели** имеют собственный специфический домен, а их арифметика зависит от типа указуемого объекта.
- **Перечисления** в C не образуют замкнутого множества значений.
- `_BitInt(N)` в C23 добавляет целые произвольной ширины.
- `nullptr` в C23 даёт отдельный тип для нулевого указателя.

[1] Это различие существенно, если при вычислении значения `X` возникают побочные эффекты.

[2] ABI (Application Binary Interface) — это «контракт» на уровне скомпилированного машинного кода о том, как разные части программы (например, ваше приложение и системная библиотека) взаимодействуют друг с другом. Не путать с API!

[3] Примерно в 1986 году Бьярн Страуструп, создатель языка C, сказал: «C makes it easy to shoot yourself in the foot; C makes

it harder, but when you do it blows your whole leg off» («C позволяет легко выстрелить себе в ногу; с C++ это сделать сложнее, но если уж вам это удастся, вы снесете всю ногу напрочь»)

[4] Как мы уже знаем, указатель не является адресом, поэтому в общем случае производится неявное преобразование нуля в представление нулевого указателя для данной платформы (слабая типизация языка это позволяет).